



Facultad de Ingeniería
Obligatorio
Inteligencia Artificial

Delfina Olaso (281981)

Martin Abzaradel (282958)

Proyecto LOST

El proyecto LOST consta de diseñar, implementar y evaluar un sistema de control autónomo capaz de guiar al rover mencionado en la letra en la exploración del terreno.

El objetivo primordial del agente es dirigir el rover desde el fondo de un valle profundo hasta alcanzar la cima de la elevación. Desde la perspectiva del desarrollo algorítmico, el objetivo es poblar y converger una matriz de valores de acción (Tabla Q) que permita guiar al agente hacia el estado terminal.

Para que el rover pueda navegar de forma autónoma tal como sugiere la letra utilizamos Q-learning para que este aprenda. El agente fue desarrollado con asistencia de inteligencia artificial generativa. El agente incluye métodos internos que nos facilitan el entrenamiento y la visualización de resultados. Como se trata de un espacio continuo, buscamos discretizar para poder usar el algoritmo, debido a que la tabla de Q crece exponencialmente, por lo que se discretiza utilizando bins, o sea rangos o contenedores, que reflejan la resolución del ambiente. Inicialmente probamos nuestra implementación de Q-Learning con valores arbitrarios, siendo los parámetros: alfa, la tasa de aprendizaje; gamma, el factor de descuento; epsilon start, la exploración, la cual se va a ir reduciendo a medida que ocurren los pasos con epsilon decay en base al epsilon end; max steps, la cantidad máxima de pasos y la cantidad de episodios. [Anexo 1.1](#).

En este primer intento limitamos el entrenamiento en 1000 pasos y a lo largo de 300 episodios, buscando un tiempo de ejecución corto en una primera instancia. Esta ejecución nos muestra algunos valores por consola mostrándonos el avance del entrenamiento y finalmente 2 gráficas, una con la recompensa por episodio y otra con la cantidad de pasos dada por episodio. Al evaluar nuestros resultados observamos que el agente no logró una convergencia satisfactoria, obteniendo una recompensa con promedio deficiente. Vemos como va “mejorando”, cada vez obteniendo recompensas menos negativas, pero esto quiere decir que nunca encontró la bandera y por eso fue aprendiendo que no moverse es mejor que moverse, porque así por lo menos reduce el castigo. También el agente se detuvo en el límite de pasos permitidos por episodio por lo que se podría aumentar este para mejorar la exploración y que no se conforme con máximos locales. Si algún episodio se corta antes de los pasos máximos quiere decir que llegó a la bandera, lo cual no es el caso.

Para un segundo intento aumentamos los valores tanto de los bins como episodios y máxima cantidad de pasos. Con la finalidad de analizar el impacto luego de modificar estos valores. También se aumentó levemente el parámetro de epsilon para favorecer mayormente a la exploración. Los parámetros y resultados se ven en el [1.2](#).

En la etapa de entrenamiento nuevamente se detiene en la cantidad máxima de pasos y tampoco se obtiene un valor de recompensa promedio positivo. Se utilizó la misma función para evaluar el agente por lo que la cantidad de pasos se forzó a que se detuviera en 1000. Y se obtuvo un promedio más cercano a un valor positivo.

En el tercer intento similarmente al caso anterior volvimos a aumentar los valores tanto de la discretización del espacio como la cantidad de pasos y episodios. Sin embargo seguíamos sin encontrar el éxito. [Anexo 1.3](#)

Recién en la cuarta instancia de entrenamiento vemos los primeros éxitos con los valores del [anexo 1.4](#). Los picos de reward representan los episodios exitosos, pero aparecen al principio y casi no vuelven a aparecer, significa que el agente encontró una buena política por casualidad mientras exploraba, pero luego la perdió cuando bajó la exploración. Es decir: halló por suerte cómo llegar a la meta temprano, pero no aprendió a hacerlo consistentemente. La tabla Q no tuvo tiempo de consolidar lo que aprendió.

La realidad a su vez, es que no estábamos dejando suficiente tiempo como para una buena experimentación, el entrenamiento se ejecutaba en minutos. Para la siguiente vamos a probar con una exploración más larga y después ejecutar la celda del test. [1.5](#).

En la sexta iteración aumentamos la cantidad de episodios y redujimos el epsilon end. También lo corrimos varias veces seguidas para que refuerce. Vemos constantemente una mayor cantidad de episodios exitosos en el [1.6](#).

En este momento fue cuando decidimos pasar al desarrollo del agente basado en Dyna-Q. Tiene una estructura similar al agente anterior y fue realizada complementariamente con inteligencia artificial. A la declaración de este agente se agrega el parámetro planning steps, la cual representa cuantos pasos va a simular el agente para mejorar el modelo y el planning por paso, la gran diferencia entre el algoritmo de Q-Learning y el de Dyna-Q. Usando los valores que se pueden ver en el [1.7](#), entrenamos por primera vez a este agente, sin conseguir ningún caso de éxito.

Para la segunda vez del entrenamiento aumentamos drásticamente la cantidad de episodios, redujimos el alfa para estabilizar más el aprendizaje, aumentamos el epsilon end para que mantenga un mayor grado de exploración y duplicamos los planning steps, para que pueda conseguir un aprendizaje significativo a partir de esto. Estos cambios aumentaron considerablemente el tiempo de ejecución, terminando de correr en 2 horas y media, pero dando muy buenos resultados, especialmente al final, confirmando que el aprendizaje se establecía correctamente. [Anexo 1.8](#). A su vez comenzamos a guardar los modelos.

En el anexo [1.9](#) podemos ver los resultados de cargar el modelo y volverlo a entrenar, encontrando un gran porcentaje de aciertos desde el principio.

A su vez ejecutamos la celda del test, la cual por primera vez de manera consistente nos mostraba al auto llegando a la bandera.

En la entrega se incluyó dos videos que son capturas de pantalla de las pruebas renderizadas en distintas etapas del proyecto.

Análisis de los bins

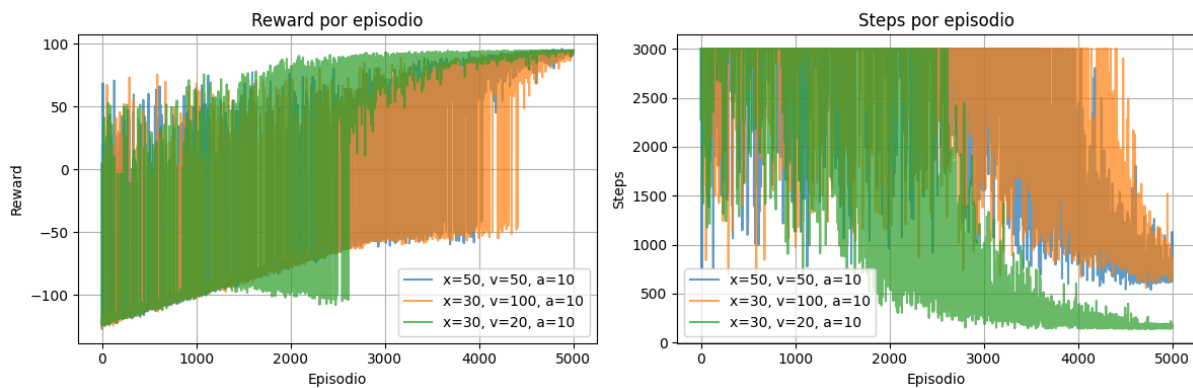
Dado que para aplicar Q-learning se necesita un espacio de acciones discreto y el espacio en Mountain Car es continuo lo adaptamos a la tabla Q. Las dimensiones de esta tabla están determinadas por los valores que llamamos x_bins, vel_bins y action_bins. Con la finalidad de buscar mejoras en nuestros resultados tanto en velocidad de ejecución y aprendizaje del modelo probamos con distintas combinaciones de ellos.

Comenzamos por evaluar con valores más pequeños para ver si estos favorecen la convergencia. Para reducir la cantidad de combinaciones en nuestra evaluación primero probamos cambiar únicamente los action_bins. Evaluamos los valores 5, 10 y 20 resultando en que se obtenían mayores recompensas con el valor de 10. Luego realizamos pruebas probando cambiar los otros dos parámetros con valores menores y con los que se había experimentado anteriormente. Nuevamente resultamos en que la disminución de estos generaba mayores recompensas.

De nuestra investigación obtuvimos los siguiente (resultados en detalle en [Anexo 1.10](#)):

=== TOP 5 CONFIGURACIONES GENERALES ===

x	v	a	avg_reward	std_reward	avg_steps	success	q_size
50	50	10	93.47	0.76	733.90	1.00	26010
30	100	10	92.63	1.08	750.70	1.00	31310
30	20	10	92.56	3.22	189.90	1.00	6510
50	50	5	0.00	0.00	3000.00	0.00	13005
50	50	20	-0.84	0.00	3000.00	0.00	52020



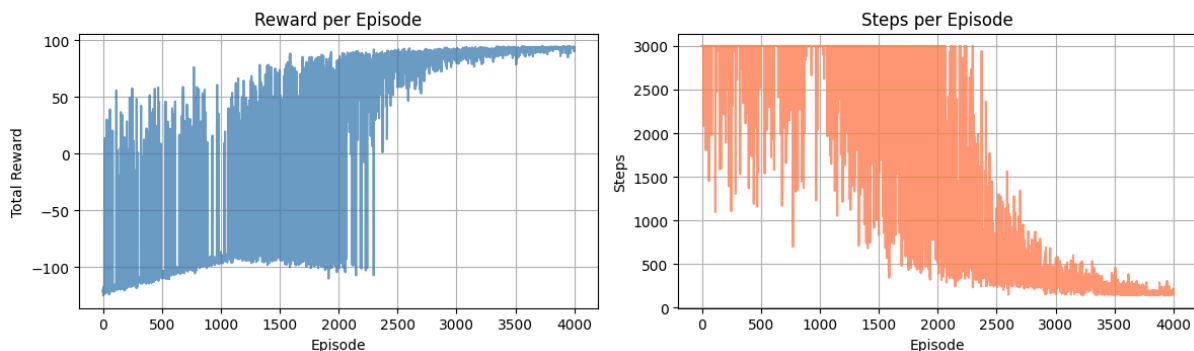
A pesar de que los valores de la recompensa promedio son mejores para las configuraciones de 50 50 10 y 30 100 10 elegimos la configuración 30 20 10. Por un lado, las recompensas promedio son bastante similares entre ellos. Pero al analizar la gráfica podemos observar que la elegida (la gráfica verde) tiene mayor velocidad de convergencia pasando los 3000 episodios tiene menos outliers y menor rango de valores de recompensa, donde estos tienden y se limitan a valores positivos. Esto implica que la política aprendida es más consistente. Con las otras dos configuraciones representadas por el naranja y el azul se visualiza que ambos al aproximarse a los 4000 episodios continúan teniendo recompensas negativas. Las otras dos configuraciones al tener tablas de mayores dimensiones requieren más episodios de entrenamiento.

Así resulta el entrenamiento con los valores de bins seleccionados:

```
[4k_episodes_eps015] Iniciando experimento con Q-Table de tamaño:
(31, 21, 10)
[4k_episodes_eps015] Ep 800/4000 - Avg Reward: -94.61, Steps: 3000
[4k_episodes_eps015] Ep 1600/4000 - Avg Reward: -64.33, Steps:
2111
[4k_episodes_eps015] Ep 2400/4000 - Avg Reward: 28.20, Steps: 426
[4k_episodes_eps015] Ep 3200/4000 - Avg Reward: 84.59, Steps: 379
```

[4k_episodes_eps015] Ep 4000/4000 - Avg Reward: 91.73, Steps: 149
--> Experimento '4k_episodes_eps015' guardado con éxito en el historial.

Progreso del experimento: 4k_episodes_eps015

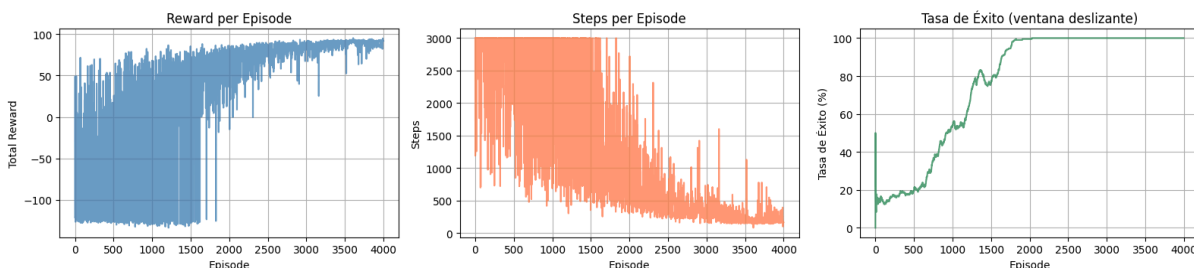


Experimentación con Dyna Q

Continuamos nuestra exploración evaluando Dyna Q para lograr resultados que se equiparen a nivel de tiempos de cómputo con los de Q-Learning.

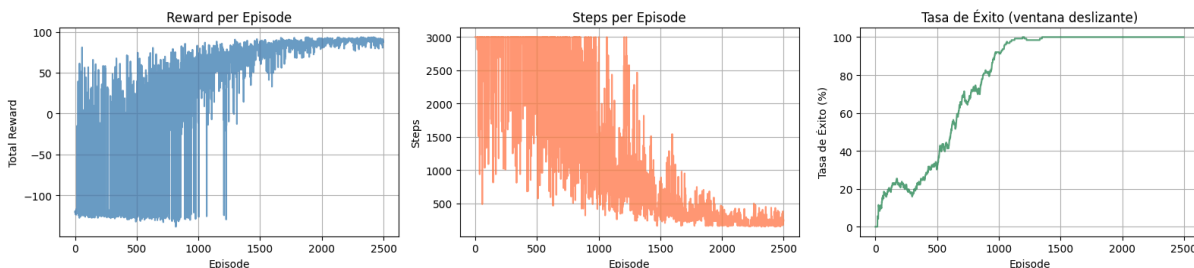
Primero evaluamos con distinta cantidad de planning steps 30 y 4000 episodios obteniendo un tiempo de cómputo de 16m 0.2s mientras que el aprendizaje con Q-Learning rondaba el minuto y medio.

Progreso: dyna_4k_episodes_eps015 | Bins: 30x20x10, planning_steps=30

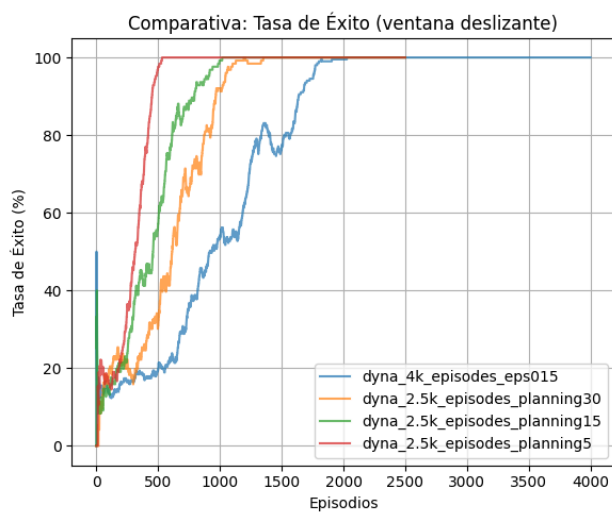
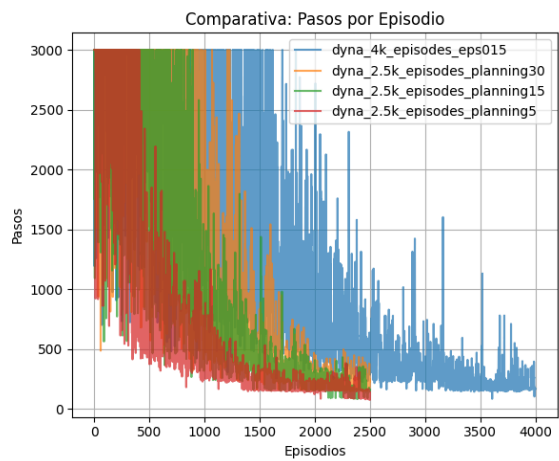
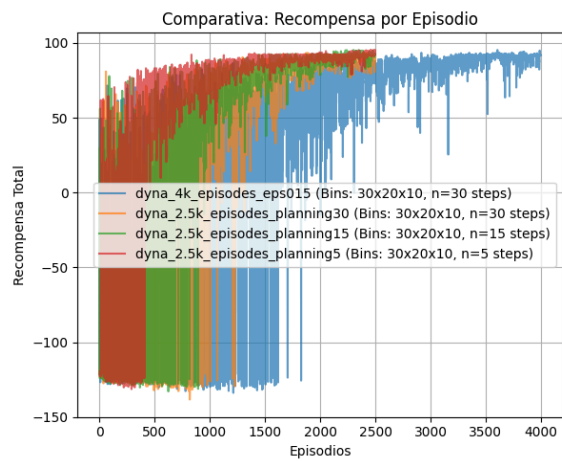


Al notar que Dyna-Q requería menos iteraciones para aprender, reducimos a 2500 la cantidad de episodios manteniendo los mismos pasos en un tiempo de 10m 7.4s.

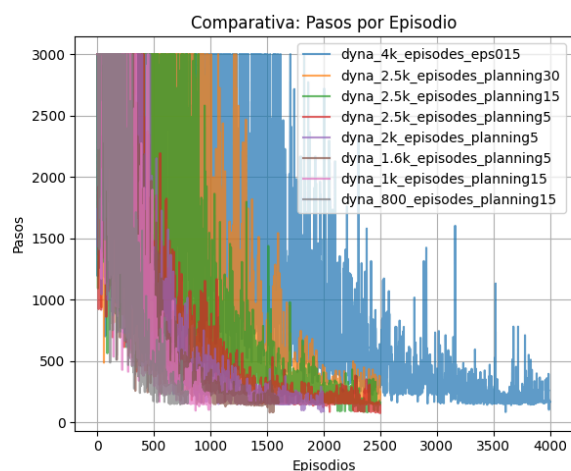
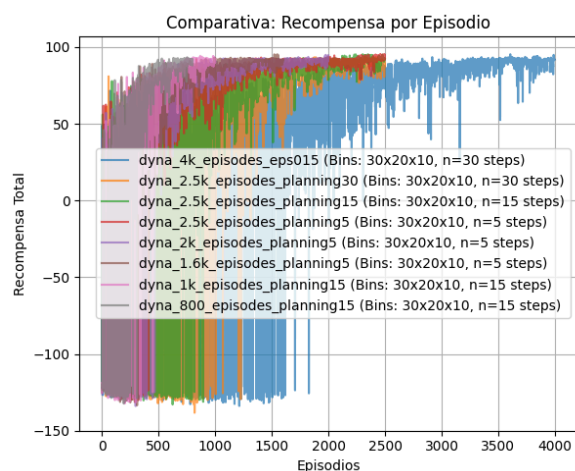
Progreso: dyna_2.5k_episodes_planning30 | Bins: 30x20x10, planning_steps=30

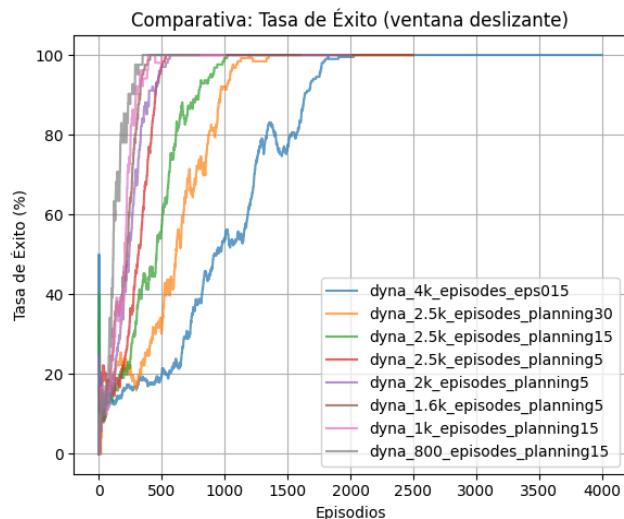


Luego optamos por reducir la cantidad de planning steps para reducir aún más los tiempos. Experimentando con los valores en 15 y 5. Para finalmente conseguir un tiempo que se equipare con el entrenamiento del agente de Q-Learning con el valor de 5 planning steps.



En base a los resultados obtenidos notamos que si tomamos el dyna q con los planning steps en 5 podríamos reducir la cantidad de episodios sin sacrificar las recompensas y disminuir aún más el tiempo de ejecución. Por lo que procedimos a entrenar con 2000, 1600, 1000 y hasta 800 episodios. [Ver en el anexo](#)





Mejores Resultado obtenidos

Adicionalmente experimentamos también con distintos valores de epsilon end y alfa en Q-Learning. Y en el caso de Dyna Q retomamos a las dimensiones anteriores para también poder analizar el efecto de haber optado por los bins elegidos. Aunque se tienen unos tiempos de ejecución extremadamente altos al observar las recompensas se obtiene un rendimiento significativamente mayor en relación a la cantidad de episodios.

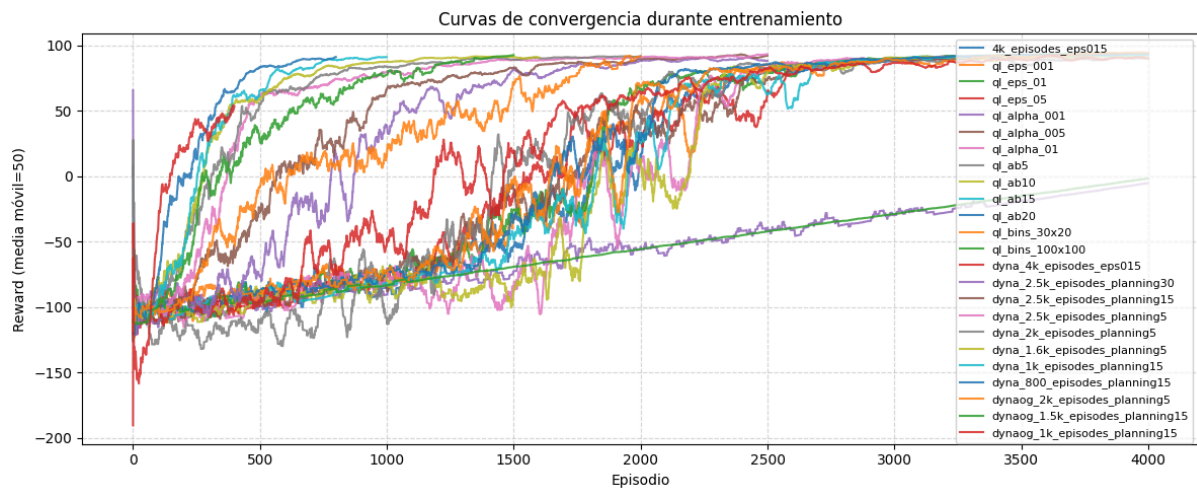
En base a todos los experimentos, los cuales analizan las recompensas obtenidas y pasos dados, incluyendo comparación de resultados entre los agentes y cuyos detalles están en el [anexo 1.13](#), se concluye que para el caso de Q-Learning tomando en cuenta tanto la recompensa promedio, la tasa de éxito, la convergencia y la estabilidad en el aprendizaje que el agente con los siguientes parámetros es el de mejor desempeño. Internamente llamado “ql_bins_30x20”.

```
manager.train_run(
    env=env,
    run_id=id_qb1,
    x_bins=30,
    vel_bins=20,
    action_bins=20,
    episodes=4000,
    alpha=0.1,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.001,
    max_steps=3000
)
```

Aunque podría mejorarse aún más si se optara por un epsilon end y un alfa de 0.05 de acuerdo con las experimentaciones realizadas con ambos parámetros.

Para el caso de Dyna Q la mayor con las dimensiones de bins en 100 100 20 se obtuvo una velocidad de convergencia en 274 episodios pero a costo de un tiempo de ejecución de 98m

con 36.4s. En código referenciado con el nombre “dynaog_1.5k_episodes_planning15”. Pero al reducir a 30 20 10 se logran tiempos menores al minuto y medio, esto no sacrifica la calidad final de las recompensas y fueron más eficientes en el test final. Por lo que optamos por “dyna_2.5k_episodes_planning5” Que reduce los planning steps sin comprometer el rendimiento al evaluarlo consigue un promedio de 113 pasos y a diferencia de la version de 200 episodios tiene una tasa de éxito de 100%. Y a nivel de tiempo de ejecución se reduce un 98%.



--- Top 5 por avg_reward ---

run_id	avg_reward	std_reward	success	avg_steps
params				
ql_bins_30x20	94.71	0.26	100%	152
x_bins=30 vel_bins=20 action_bins=20 alpha=0.1				
ql_eps_05	94.10	0.53	100%	162
x_bins=30 vel_bins=20 action_bins=10 alpha=0.05				
ql_ab20	93.79	0.90	100%	170
x_bins=30 vel_bins=20 action_bins=20 alpha=0.1				
4k_episodes_eps015	93.57	1.36	100%	171
x_bins=30 vel_bins=20 action_bins=10 alpha=0.05				
ql_eps_001	93.53	0.40	100%	171
x_bins=30 vel_bins=20 action_bins=10 alpha=0.05				

--- Velocidad de convergencia (umbral reward > 0) ---

4k_episodes_eps015	episodio 1802
ql_eps_001	episodio 1703
ql_eps_01	episodio 1812
ql_eps_05	episodio 1539
ql_alpha_001	episodio no convergió
ql_alpha_005	episodio 1725
ql_alpha_01	episodio 1755
ql_ab5	episodio 1415
ql_ab10	episodio 1840
ql_ab15	episodio 1821
ql_ab20	episodio 1652
ql_bins_30x20	episodio 1644
ql_bins_100x100	episodio no convergió
dyna_4k_episodes_eps015	episodio 1186
dyna_2.5k_episodes_planning30	episodio 781
dyna_2.5k_episodes_planning15	episodio 566
dyna_2.5k_episodes_planning5	episodio 372
dyna_2k_episodes_planning5	episodio 310

dyna_1.6k_episodes_planning5	episodio 280
dyna_1k_episodes_planning15	episodio 254
dyna_800_episodes_planning15	episodio 201
dynaog_2k_episodes_planning5	episodio 520
dynaog_1.5k_episodes_planning15	episodio 274

dynaog_1k_episodes_planning15	episodio 147
-------------------------------	--------------

=== Evaluando 11 experimento(s), 10 episodios c/u ===

```
[dyna_4k_episodes_eps015] avg_reward=91.89 ± 0.28 | success=100%
| avg_steps=165
[dyna_2.5k_episodes_planning30] avg_reward=90.75 ± 1.33 |
success=100% | avg_steps=180
[dyna_2.5k_episodes_planning15] avg_reward=92.28 ± 0.80 |
success=100% | avg_steps=164
[dyna_2.5k_episodes_planning5] avg_reward=93.01 ± 0.98 |
success=100% | avg_steps=113
[dyna_2k_episodes_planning5] avg_reward=64.92 ± 54.70 |
success=80% | avg_steps=901
[dyna_1.6k_episodes_planning5] avg_reward=90.97 ± 0.41 |
success=100% | avg_steps=168
[dyna_1k_episodes_planning15] avg_reward=90.77 ± 8.64 |
success=100% | avg_steps=149
[dyna_800_episodes_planning15] avg_reward=71.94 ± 38.44 |
success=80% | avg_steps=939
[dynaog_2k_episodes_planning5] avg_reward=-1.13 ± 0.02 |
success=0% | avg_steps=4000
[dynaog_1.5k_episodes_planning15] avg_reward=93.54 ± 1.46 |
success=100% | avg_steps=1121
[dynaog_1k_episodes_planning15] avg_reward=-1.11 ± 0.00 |
success=0% | avg_steps=4000
```

Estas 2 configuraciones fueron las elegidas para entrenar a los modelos incluidos en la entrega “dyna_agent_trained_final.npy” y “ql_agent_trained_final.npy”. También se incluyen “dyna_agent_trained.npy” y “dyna_agent_trained_v2.npy” que fueron desarrollados durante los distintos entrenamientos.

Proyecto MATE

Para esta parte del trabajo comenzamos viendo los agentes ya implementados y los juegos entre ellos. A partir de ahí implementamos primero el agente basado en minimax, incluyendo Alpha-Beta Pruning, el cual busca podar nodos del árbol de decisiones basado en dos valores que funcionan como cotas.

Ya en la primera implementación decidimos acotarlo a una profundidad máxima de 3 turnos a la hora de analizar el futuro, es un valor que nos pareció que le daba la suficiente libertad para explorar pero sin exigir un tiempo de cómputo exagerado. Como función de evaluación elegimos arrancar con una simple como la diferencia entre los movimientos posibles del agente menos los movimientos posibles del rival. Refleja una realidad del juego que a mayor movimientos posibles tuyos y menor del rival, más cerca de la victoria estás.

Una vez ya implementado el algoritmo, ejecutamos pruebas individuales como tenían los otros, enfrentándolo contra el random agent y el stratagem dado, probándolo tanto como player 1 y 2. Ya ejecutando estas pruebas puntuales pudimos notar su eficacia.

Posteriormente desarrollamos el agente basado en expectimax. Este agente considera todas las posibilidades y el valor promedio de la evaluación de las opciones de cada rama. Al no descartar ninguna opción no puede podar tal como el minimax. Esto lo vemos reflejado en la práctica, ya que al empezar a probar con los partidos individuales notamos como era el agente que más demoraba para cada paso. Al igual que en el agente anterior decidimos usar profundidad máxima de 3 y la misma heurística. Ambos agentes fueron desarrollados con asistencia de inteligencia artificial generativa.

Una vez hechos ambos agentes, pasamos a una fase de experimentación más global y profunda, haciéndolos jugar varios partidos entre sí y extrayendo datos a partir de ello. Iteramos con un doble for para que cada agente juegue contra otro (incluyéndose a sí mismo) y también varíe el orden de jugador 1 o 2, lo cual hicimos para comprobar si tener el primer movimiento implicaba una ventaja o no. Por cada enfrentamiento entre agentes, simulamos 20 partidos en una primera instancia y los datos que recopilamos en todo este proceso fueron: resultado de la serie, cantidad de victorias del player 1 a nivel global, victorias como player 1 y como player 2 del agente en particular y victorias en general del agente.

Esta simulación nos llevó a los gráficos del [anexo 2.1](#), donde vemos como salió cada serie entre agentes, la tabla general de victorias y la diferencia de victorias por cada agente según si era player 1 o 2 y en total de todos los agentes según si jugaban primero o segundo.

Después de jugarse los 320 partidos en total (tras casi 2 horas de ejecución, las cuales se alargaron porque a veces la computadora apagó la pantalla y quedó en modo suspendida) las primeras conclusiones fueron que el agente minimax resultó el mejor, seguido del stratagem.

Parecería ser que el empezar moviendo es una ventaja considerable pero no circunstancial. Lo que más nos llamó la atención es el desempeño del agente expectimax, siendo el de peores resultados, siendo especialmente llamativo su bajo nivel en las series contra el random agent, donde debería destacarse. Este bajo rendimiento nos hizo revisar el código y encontrar un bug: cuando iteraba por cada acción posible, clonaba el tablero pero no ejecutaba la acción. Agregamos la línea de `next_board.play(action, player)`, que debería

haber estado desde un principio. También vimos un detalle en el de minimax que calculaba 2 veces la lista de movimientos porque el código estaba duplicado.

Después de estas correcciones repetimos el experimento, esta vez jugando 25 partidos por serie, lo cual nos dejó las gráficas del [2.2](#). Ahí podemos ver cómo se consolidan el minimax y el stratagem como los mejores agentes. A su vez repunta el expectimax, que ahora sí funcionando correctamente supera al random y queda en 3era posición. El expectimax también cuenta con la curiosidad de ser el único agente que cosecha más victorias como player 2 que como 1, esto igualmente se podía deber a que cada agente jugaba 200 partidos en total, si bien es un buen número, podría ser sensible a pequeñas desviaciones.

Una vez visto un despliegue general de los agentes, buscamos mejorarlos y probar distintas variaciones, comenzando con la profundidad. Para esto decidimos, empezando con minimax, enfrentarlos como player 1 y 2 al resto de los agentes, controlando su cantidad de victorias y su tiempo de ejecución; variando la profundidad en un rango desde 1 a 4. Elegimos este rango porque nos pareció que después de 4 iba a ser muy costoso sin mejorar significativamente los resultados. Poco menos de una hora después obtuvimos los resultados del anexo [2.3](#). Ahí vemos una mejora lenta pero constante entre profundidades, siendo en promedio 6,67 victorias mejor por cada profundidad, pero la clara diferencia de tiempo entre el de profundidad 3 y 4 nos hizo elegir por seguir con profundidad 3.

En el caso de expectimax, redujimos la cantidad de partidos por serie a 20 y, tras casi 4 horas de ejecución, nos sorprendieron los resultados, en el [2.4](#). Los agentes con profundidad menor fueron claramente mejores que los de profundidad mayor, siendo el de profundidad 2 nuestro elegido. Esta situación, que resulta contraria a lo que uno esperaría, puede deberse a que el algoritmo de expectimax le agrega una capa de azar a cada turno, la cual en realidad no está ahí, al ser un juego determinístico y teniendo dos rivales que en principio no deciden mediante el azar. Esta decisión de elegir la profundidad máxima 2 para el agente de expectimax nos va a aliviar temporalmente para futuras pruebas.

Tras realizar estos ajustes volvimos a correr la celda de prueba, esta vez corriendo 100 partidos por serie y terminando alrededor de en una hora, con los resultados de [2.5](#). Hay una clara mejoría del expectimax, el cual ahora se pone ligeramente atrás del minimax, que sigue siendo el mejor de los agentes. A su vez seguimos chequeando la ventaja que impone ser el que realiza el primer movimiento, obteniendo casi el doble de victorias en general de este modo.

Ahora nos vamos a dedicar a analizar el valor de la función de evaluación, probando variaciones para encontrar la mejor. Basándonos en un artículo encontrado en internet (<https://www.rpubs.com/primaryobjects/isolation>), agregamos a la firma del agente un argumento de texto el cual determina cual va a ser la heurística, ya implementada en el agente. Incluimos:

- baseline: la que veníamos usando, mis posibles movimientos - movimientos del rival
- defensive: prioriza tener movimientos nosotros, $2 * \text{mis posibles movimientos} - \text{movimientos del rival}$
- ofensive: prioriza limitar los movimientos del rival $\text{mis posibles movimientos} - 2 * \text{movimientos del rival}$
- simple: solo tiene en cuenta mis movimientos

-d2o: defensiva a ofensiva, comienza el partido con la estrategia defensiva mencionada previamente y una vez que ya fueron eliminadas la mitad de las celdas posibles, pasa a la estrategia ofensiva.

-o2d: ofensiva a defensiva, al contrario que la anterior.

El artículo mencionado como inspiración también menciona una estrategia basada en dominar el centro, pero al ver que hay varias interpretaciones a lo que significa “dominar el centro” (podría ser ser más defensivo en caso de ocupar el centro, o ser más ofensivo ya que nos encontramos en una posición mejor por ejemplo) decidimos no implementarla.

Para la prueba hicimos que juegue minimax con cada heurística 400 partidos contra el stratagem, 200 como player 1 y 200 como player 2, durando la ejecución total de la prueba 2 horas. Las gráficas resultantes, en el [anexo 2.6](#), nos muestran que si bien todos resultaron bastante parejos, la función de evaluación que se impone como la más victoriosa es la defensiva.

Lo que correspondía posteriormente era comprobar si este era el caso también con expectimax, simulando ahora 100 y 100 partidos contra el stratagem, usando esas mismas distintas heurísticas. En este caso la de mayor número de victorias fue la baseline, seguido muy de cerca de la ofensiva y la ofensiva to defensiva. [Anexo 2.7](#).

El último cambio que decidimos probar fue el efecto de la poda Alfa-Beta en minimax. Para esto implementamos un agente idéntico al que teníamos, pero sin las partes de la poda. Luego hicimos que jueguen 100 partidos contra el agente random cada una de las dos versiones de minimax, controlando su tiempo y sus victorias en caso de que haya un cambio en el rendimiento, el cual supusimos que no iba a pasar porque Alpha-Beta pruning afecta a la velocidad y no a la efectividad. Obtuvimos los resultados del anexo 2.8, donde vemos que no hay una diferencia considerable en la cantidad de victorias (99 con a 98 sin), pero sí un incremento notable de tiempo de ejecución para el agente sin poda, demorando 800,86 segundos, más de 13 minutos. En comparación, la versión con poda demoró 83,47 segundos, o sea 717 segundos menos, lo que vendría a ser 9,59 veces más. En definitiva, es como si cada partida sin poda durase 8 segundos, mientras que usando la poda la partida se desarrolla en menos de 1 segundo.

Una vez ya probadas todas estas variaciones, decidimos volver a probar las pruebas generales, para ver cómo era el rendimiento de los agentes finalmente, jugando esta vez 200 partidos por serie siendo la última instancia. Cada agente jugó 1600 partidos (200 por serie por 4 agentes a enfrentar por 2 porque se juega como player 1 y como player 2), dando los resultados de [2.9](#). Volvimos a notar a minimax como el mejor, con 1075 victorias y una efectividad de 67,2%, seguido del expectimax con 1031 partidos ganados y 64,4%, Stratagem con 829 y 51,8% y por último el random con 265 victorias y 16,6%. Otro aspecto interesante que vemos, es que se cumple algo estudiado, cuando no conocemos la implementación y la policy del rival, como en sus enfrentamientos contra el stratagem, vemos que es peor que el minimax.

Todos vieron una ventaja de ser el player 1, el random ganó el 60,8% de sus encuentros en ese contexto, stratagem 67,8%, minimax 63,3% y expectimax 67,3%.

ANEXO

Anexo 1.1

```
agent = QLearningAgent(x_bins=100, vel_bins=100, action_bins=10)
```

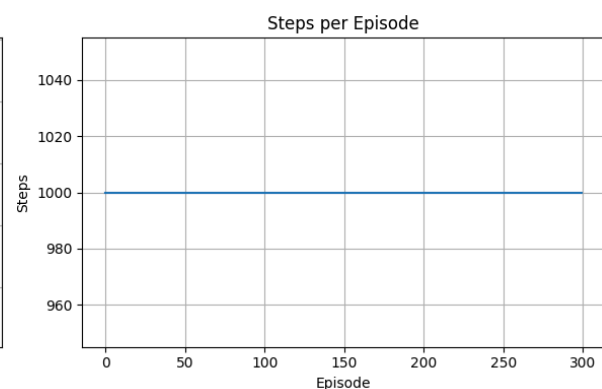
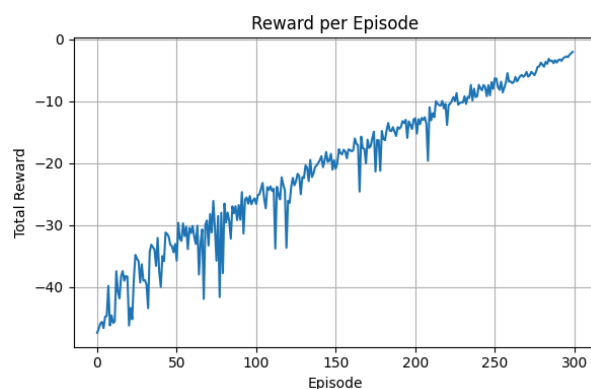
```
env,
  episodes=300,
  alpha=0.1,
  gamma=0.99,
  epsilon_start=0.9,
  epsilon_end=0.01,
  max_steps=1000
)
env.close()
```

```
# Visualizar progreso de entrenamiento
agent.plot_training_history()
```

✓ 5.1s

Python

```
Entrenando agente Q-Learning...
Episode 30/300 - Avg Reward: -41.66, Steps: 1000
Episode 60/300 - Avg Reward: -34.02, Steps: 1000
Episode 90/300 - Avg Reward: -31.10, Steps: 1000
Episode 120/300 - Avg Reward: -25.92, Steps: 1000
Episode 150/300 - Avg Reward: -21.44, Steps: 1000
Episode 180/300 - Avg Reward: -18.02, Steps: 1000
Episode 210/300 - Avg Reward: -14.19, Steps: 1000
Episode 240/300 - Avg Reward: -10.26, Steps: 1000
Episode 270/300 - Avg Reward: -7.16, Steps: 1000
Episode 300/300 - Avg Reward: -3.94, Steps: 1000
```



```
# Testear el agente entrenado (sin exploración)
print("\nTesteando agente entrenado...")
env = gym.make('MountainCarContinuous-v0', render_mode='human')
test_rewards, test_steps = agent.test_agent(env, episodes=3, render=True)
env.close()
```

```
print(f"\nPromedio de rewards en testing: {np.mean(test_rewards):.2f}")
print(f"Promedio de steps en testing: {np.mean(test_steps):.2f}")
```

✓ 3m 25.2s

```
Testeando agente entrenado...
Test Episode 1 - Reward: -1.64, Steps: 1000
Test Episode 2 - Reward: -1.60, Steps: 1000
Test Episode 3 - Reward: -1.62, Steps: 1000
```

```
Promedio de rewards en testing: -1.62
Promedio de steps en testing: 1000.00
```

Anexo 1.2

```
agent = QLearningAgent(x_bins=200, vel_bins=200, action_bins=20)
```

```
agent.train_agent(  
    env,  
    episodes=1000,  
    alpha=0.05,  
    gamma=0.99,  
    epsilon_start=0.9,  
    epsilon_end=0.05,  
    max_steps=1500  
)
```

Entrenando agente Q-Learning...

Episode 100/1000 - Avg Reward: -60.43, Steps: 1500
Episode 200/1000 - Avg Reward: -51.68, Steps: 1500
Episode 300/1000 - Avg Reward: -43.75, Steps: 1500
Episode 400/1000 - Avg Reward: -38.26, Steps: 1500
Episode 500/1000 - Avg Reward: -31.46, Steps: 1500
Episode 600/1000 - Avg Reward: -26.53, Steps: 1500
Episode 700/1000 - Avg Reward: -21.13, Steps: 1500
Episode 800/1000 - Avg Reward: -16.11, Steps: 1500
Episode 900/1000 - Avg Reward: -10.42, Steps: 1500
Episode 1000/1000 - Avg Reward: -5.78, Steps: 1500

Testeando agente entrenado...

Test Episode 1 - Reward: -0.42, Steps: 1000
Test Episode 2 - Reward: -1.48, Steps: 1000
Test Episode 3 - Reward: -0.52, Steps: 1000

Promedio de rewards en testing: -0.81

Promedio de steps en testing: 1000.00

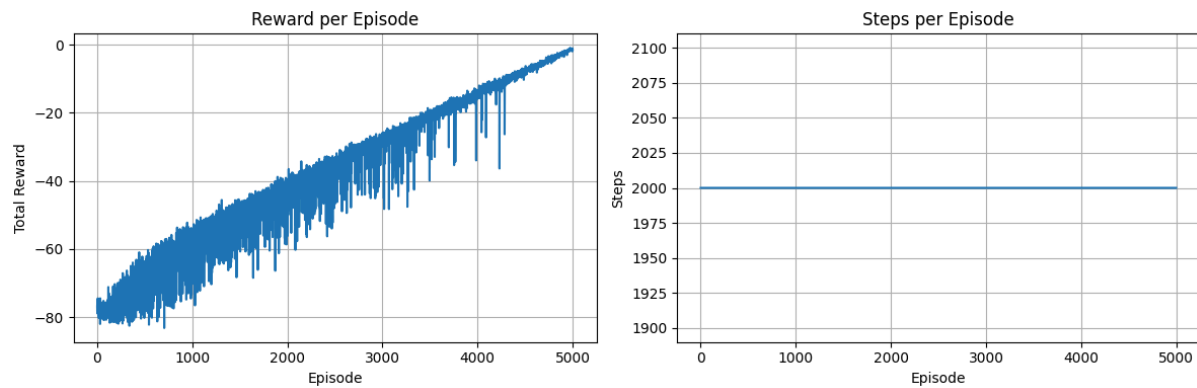
Anexo 1.3

```
agent = QLearningAgent(x_bins=300, vel_bins=300, action_bins=30)  
agent.train_agent(  
    env,  
    episodes=5000,  
    alpha=0.02,  
    gamma=0.99,  
    epsilon_start=0.95,  
    epsilon_end=0.01,  
    max_steps=2000  
)
```

Entrenando agente Q-Learning...

Episode 500/5000 - Avg Reward: -74.91, Steps: 2000
Episode 1000/5000 - Avg Reward: -64.62, Steps: 2000
Episode 1500/5000 - Avg Reward: -55.23, Steps: 2000
Episode 2000/5000 - Avg Reward: -47.08, Steps: 2000

Episode 2500/5000 - Avg Reward: -39.84, Steps: 2000
 Episode 3000/5000 - Avg Reward: -32.16, Steps: 2000
 Episode 3500/5000 - Avg Reward: -25.25, Steps: 2000
 Episode 4000/5000 - Avg Reward: -18.05, Steps: 2000
 Episode 4500/5000 - Avg Reward: -11.27, Steps: 2000
 Episode 5000/5000 - Avg Reward: -4.50, Steps: 2000

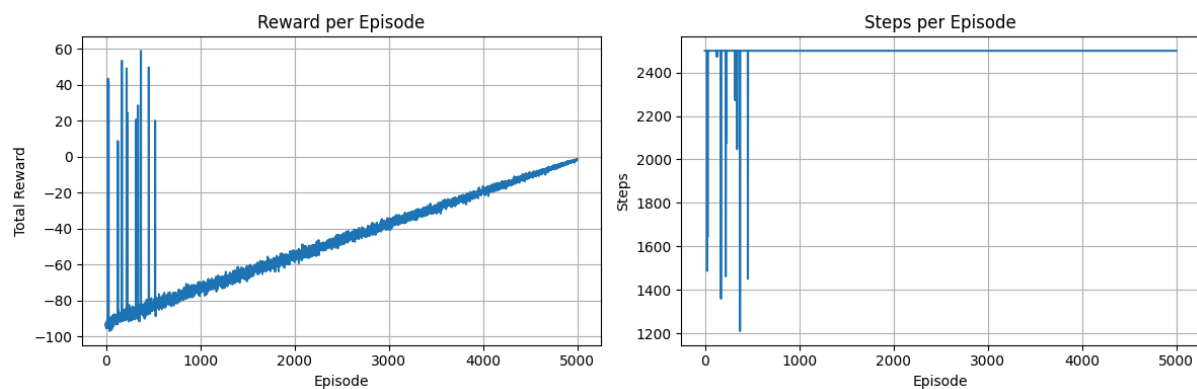


Anexo 1.4

```

agent = QLearningAgent(x_bins=100, vel_bins=100, action_bins=20)
agent.train_agent(
    env,
    episodes=5000,
    alpha=0.02,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.005,
    max_steps=2500
)

```



Anexo 1.5

```

agent.train_agent(
    env,
    episodes=5000,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,

```

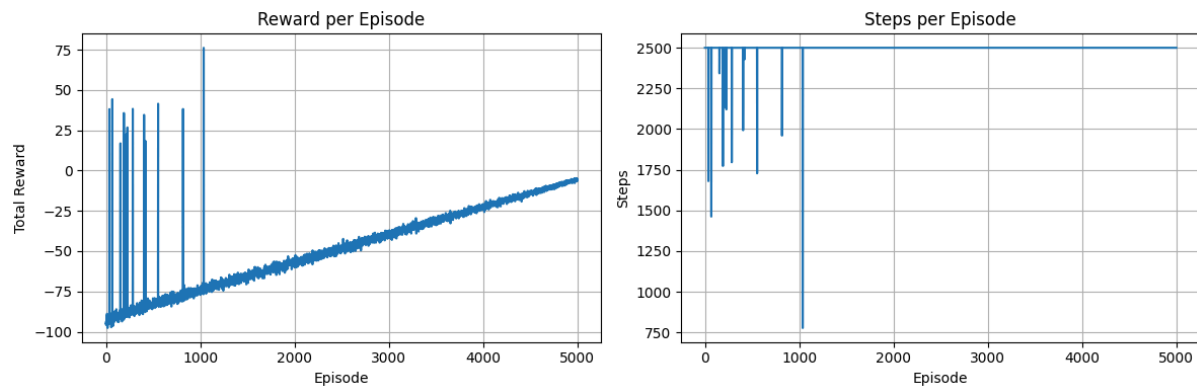
```

    epsilon_end=0.05,
    max_steps=2500
)

```

Entrenando agente Q-Learning...

Episode 500/5000 - Avg Reward: -85.62, Steps: 2500
 Episode 1000/5000 - Avg Reward: -77.78, Steps: 2500
 Episode 1500/5000 - Avg Reward: -69.49, Steps: 2500
 Episode 2000/5000 - Avg Reward: -61.04, Steps: 2500
 Episode 2500/5000 - Avg Reward: -52.60, Steps: 2500
 Episode 3000/5000 - Avg Reward: -43.95, Steps: 2500
 Episode 3500/5000 - Avg Reward: -35.38, Steps: 2500
 Episode 4000/5000 - Avg Reward: -26.84, Steps: 2500
 Episode 4500/5000 - Avg Reward: -18.30, Steps: 2500
 Episode 5000/5000 - Avg Reward: -9.81, Steps: 2500



Testeando agente entrenado...

Test Episode 1 - Reward: -0.28, Steps: 1000
 Test Episode 2 - Reward: -0.28, Steps: 1000
 Test Episode 3 - Reward: -0.28, Steps: 1000

Promedio de rewards en testing: -0.28
 Promedio de steps en testing: 1000.00

Anexo 1.6

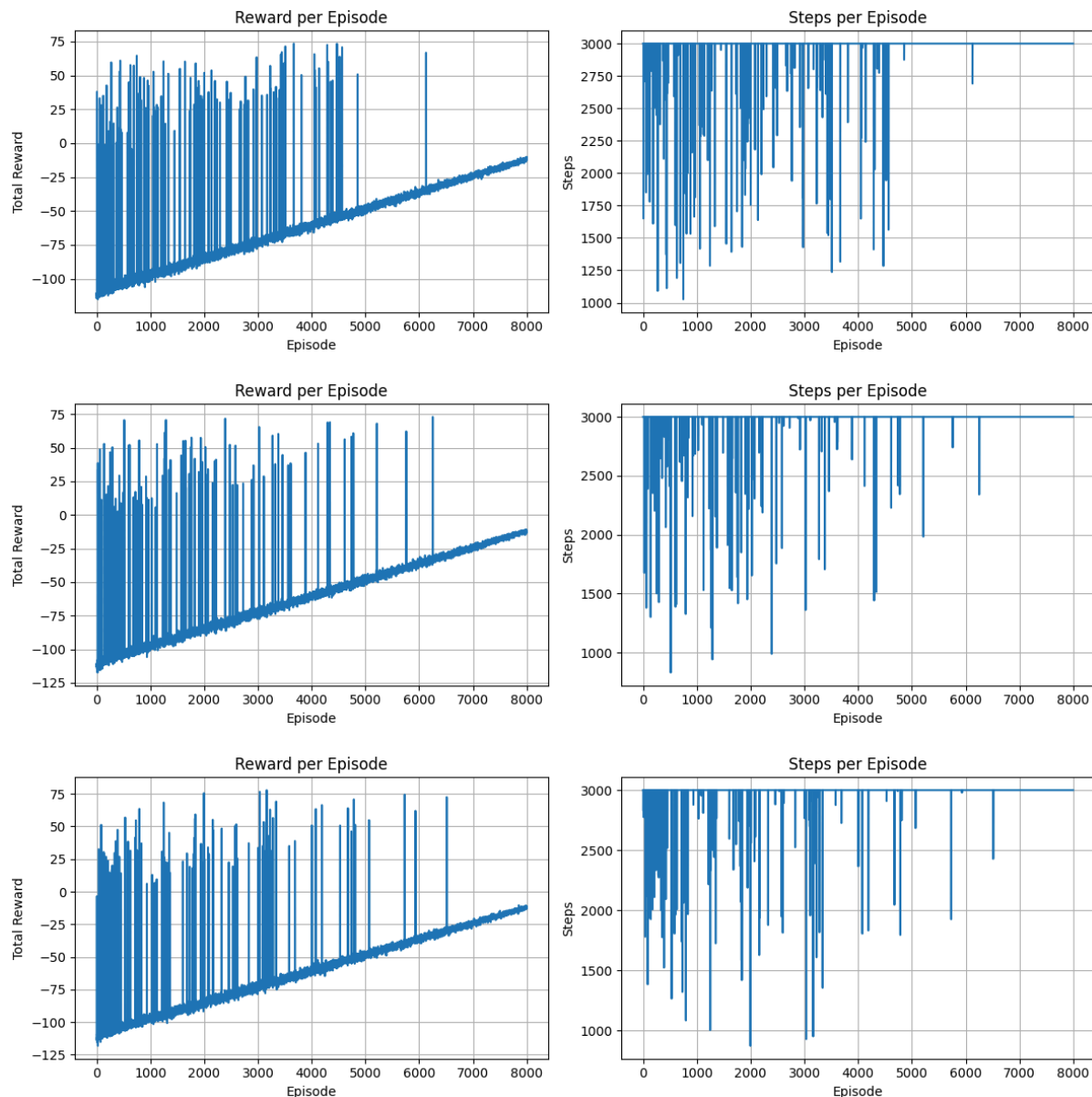
```

agent.train_agent(
    env,
    episodes=8000,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.1,
    max_steps=2500
)

```

Episode 800/8000 - Avg Reward: -99.25, Steps: 3000
 Episode 1600/8000 - Avg Reward: -91.31, Steps: 3000

Episode 2400/8000 - Avg Reward: -81.53, Steps: 3000
 Episode 3200/8000 - Avg Reward: -73.30, Steps: 3000
 Episode 4000/8000 - Avg Reward: -63.50, Steps: 3000
 Episode 4800/8000 - Avg Reward: -54.10, Steps: 3000
 Episode 5600/8000 - Avg Reward: -45.80, Steps: 3000
 Episode 6400/8000 - Avg Reward: -36.15, Steps: 3000
 Episode 7200/8000 - Avg Reward: -26.57, Steps: 3000
 Episode 8000/8000 - Avg Reward: -16.80, Steps: 3000



Anexo 1.7

```
agent = DynaQAgent(x_bins=100, vel_bins=100, action_bins=20)
```

```

agent.train_agent(
    env,
    episodes=2000,

```

```

alpha=0.05,
gamma=0.99,
epsilon_start=0.99,
epsilon_end=0.15,
max_steps=1000,
planning_steps=10
)

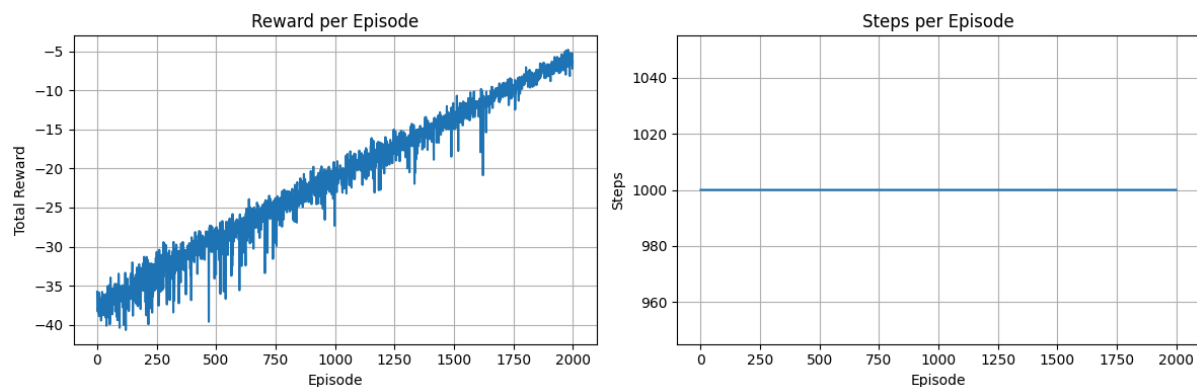
```

Entrenando agente Dyna-Q...

```

Episode 200/2000 - Avg Reward: -36.14, Steps: 1000
Episode 400/2000 - Avg Reward: -32.71, Steps: 1000
Episode 600/2000 - Avg Reward: -29.29, Steps: 1000
Episode 800/2000 - Avg Reward: -26.16, Steps: 1000
Episode 1000/2000 - Avg Reward: -22.85, Steps: 1000
Episode 1200/2000 - Avg Reward: -19.63, Steps: 1000
Episode 1400/2000 - Avg Reward: -16.66, Steps: 1000
Episode 1600/2000 - Avg Reward: -13.51, Steps: 1000
Episode 1800/2000 - Avg Reward: -10.45, Steps: 1000
Episode 2000/2000 - Avg Reward: -7.30, Steps: 1000

```



Anexo 1.8

```

agent = DynaQAgent(x_bins=100, vel_bins=100, action_bins=20)
agent.train_agent(
    env,
    episodes=5000,
    alpha=0.03,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.2,
    max_steps=1000,
    planning_steps=20
)

```

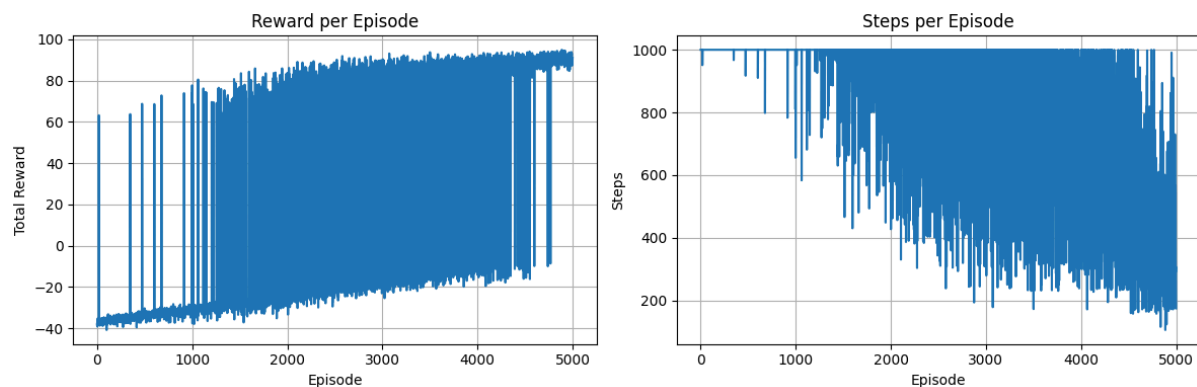
Entrenando agente Dyna-Q...

```

Episode 500/5000 - Avg Reward: -34.92, Steps: 1000, Success: 0.6%

```

Episode 1000/5000 - Avg Reward: -31.41, Steps: 655, Success: 1.0%
 Episode 1500/5000 - Avg Reward: -22.84, Steps: 1000, Success: 6.8%
 Episode 2000/5000 - Avg Reward: -3.76, Steps: 1000, Success: 22.8%
 Episode 2500/5000 - Avg Reward: 20.28, Steps: 721, Success: 42.8%
 Episode 3000/5000 - Avg Reward: 40.02, Steps: 1000, Success: 59.0%
 Episode 3500/5000 - Avg Reward: 52.36, Steps: 589, Success: 69.0%
 Episode 4000/5000 - Avg Reward: 67.82, Steps: 376, Success: 82.2%
 Episode 4500/5000 - Avg Reward: 77.24, Steps: 234, Success: 89.4%
 Episode 5000/5000 - Avg Reward: 88.10, Steps: 174, Success: 98.2%
 Modelo guardado en dyna_agent_trained.npy



Anexo 1.9

Continuar entrenamiento a partir del modelo guardado

```
from dyna_q_agent import DynaQAgent
```

```
print("\nContinuando entrenamiento desde el modelo guardado...")
```

```
agent = DynaQAgent(x_bins=100, vel_bins=100, action_bins=20)
```

```
agent.load_model('dyna_agent_trained.npy')
```

```
env = gym.make('MountainCarContinuous-v0', render_mode='rgb_array')
```

```
agent.train_agent(
```

```
    env,
```

```
    episodes=1000,
```

```
    alpha=0.02,
```

```
    gamma=0.995,
```

```
    epsilon_start=0.2,
```

```
    epsilon_end=0.05,
```

```
    max_steps=700,
```

```
    planning_steps=30
```

```
)
```

```
env.close()
```

```
agent.save_model('dyna_agent_trained_v2.npy')
```

```
agent.plot_training_history()
```

```
print("Modelo actualizado guardado en dyna_agent_trained_v2.npy")
```

Continuando entrenamiento desde el modelo guardado...

Modelo cargado desde dyna_agent_trained.npy (Q shape: (101, 101, 20))

Episode 100/1000 - Avg Reward: 90.12, Steps: 181, Success: 99.0%

Episode 200/1000 - Avg Reward: 91.02, Steps: 258, Success: 100.0%

Episode 300/1000 - Avg Reward: 91.07, Steps: 250, Success: 100.0%

Episode 400/1000 - Avg Reward: 91.29, Steps: 188, Success: 100.0%

Episode 500/1000 - Avg Reward: 91.25, Steps: 108, Success: 100.0%

Episode 600/1000 - Avg Reward: 91.96, Steps: 163, Success: 100.0%

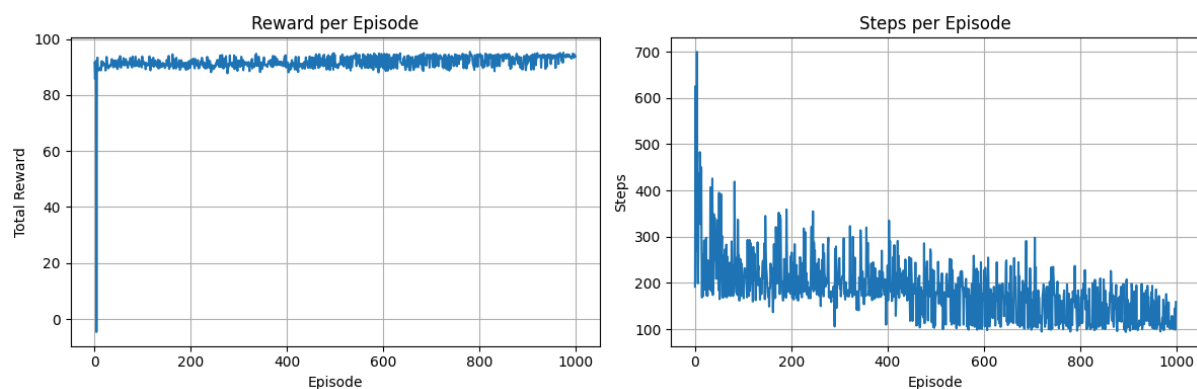
Episode 700/1000 - Avg Reward: 91.99, Steps: 99, Success: 100.0%

Episode 800/1000 - Avg Reward: 92.63, Steps: 110, Success: 100.0%

Episode 900/1000 - Avg Reward: 92.67, Steps: 164, Success: 100.0%

Episode 1000/1000 - Avg Reward: 93.45, Steps: 159, Success: 100.0%

Modelo guardado en dyna_agent_trained_v2.npy



Continuando entrenamiento desde el modelo guardado...

Modelo cargado desde dyna_agent_trained.npy (Q shape: (101, 101, 20))

Episode 100/1000 - Avg Reward: 91.40, Steps: 169, Success: 100.0%

Episode 200/1000 - Avg Reward: 91.28, Steps: 263, Success: 100.0%

Episode 300/1000 - Avg Reward: 90.86, Steps: 104, Success: 100.0%

Episode 400/1000 - Avg Reward: 91.13, Steps: 174, Success: 100.0%

Episode 500/1000 - Avg Reward: 91.63, Steps: 164, Success: 100.0%

Episode 600/1000 - Avg Reward: 92.37, Steps: 178, Success: 100.0%

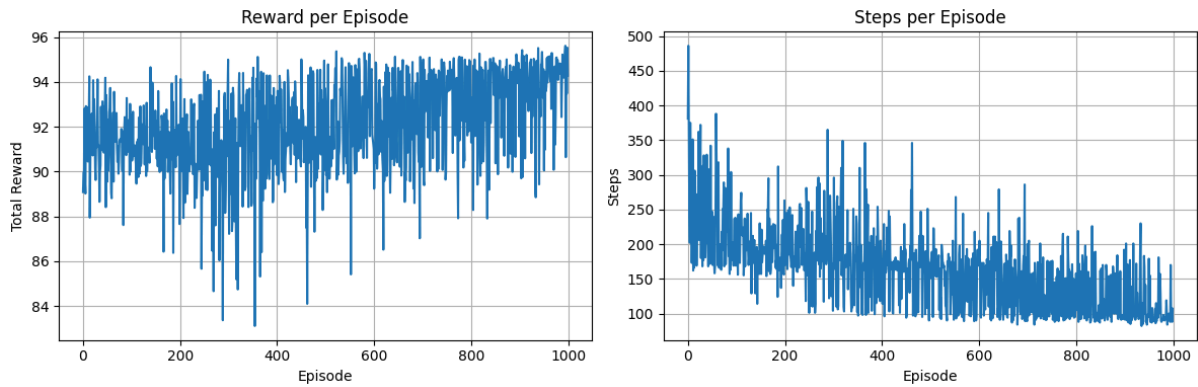
Episode 700/1000 - Avg Reward: 92.19, Steps: 89, Success: 100.0%

Episode 800/1000 - Avg Reward: 93.19, Steps: 109, Success: 100.0%

Episode 900/1000 - Avg Reward: 93.20, Steps: 192, Success: 100.0%

Episode 1000/1000 - Avg Reward: 93.84, Steps: 107, Success: 100.0%

Modelo guardado en dyna_agent_trained_v2.npy



Anexo 1.10

```
train_kwargs = {
    'episodes': 5000,
    'alpha': 0.05,
    'gamma': 0.99,
    'epsilon_start': 0.99,
    'epsilon_end': 0.0005,
    'max_steps': 3000,
}
```

```
evaluator.evaluate_group(
    env=env,
    x_bins_list=[50],
    vel_bins_list=[50],
    action_bins_list=[5, 10, 20],
    train_kwargs=train_kwargs,
    test_episodes=10,
    test_max_steps=3000,
    prefix="G1_Acciones"
)
```

=== Iniciando Tanda de 3 combinaciones ===

[1/3] Evaluando: G1_Acciones_x50_v50_a5

[G1_Acciones_x50_v50_a5] Iniciando experimento con Q-Table de tamaño: (51, 51, 5)

[G1_Acciones_x50_v50_a5] Ep 1000/5000 - Avg Reward: -114.13, Steps: 3000

[G1_Acciones_x50_v50_a5] Ep 2000/5000 - Avg Reward: -90.73, Steps: 3000

[G1_Acciones_x50_v50_a5] Ep 3000/5000 - Avg Reward: -67.28, Steps: 3000

[G1_Acciones_x50_v50_a5] Ep 4000/5000 - Avg Reward: -43.08, Steps: 3000

[G1_Acciones_x50_v50_a5] Ep 5000/5000 - Avg Reward: -14.87, Steps: 3000
--> Experimento 'G1_Acciones_x50_v50_a5' guardado con éxito en el historial.

-> Resultado: Avg Reward=0.00 | Success Rate=0.00

[2/3] Evaluando: G1_Acciones_x50_v50_a10

[G1_Acciones_x50_v50_a10] Iniciando experimento con Q-Table de tamaño: (51, 51, 10)

[G1_Acciones_x50_v50_a10] Ep 1000/5000 - Avg Reward: -97.41, Steps: 3000

[G1_Acciones_x50_v50_a10] Ep 2000/5000 - Avg Reward: -76.52, Steps: 3000

[G1_Acciones_x50_v50_a10] Ep 3000/5000 - Avg Reward: -47.80, Steps: 3000

[G1_Acciones_x50_v50_a10] Ep 4000/5000 - Avg Reward: 32.54, Steps: 1136

[G1_Acciones_x50_v50_a10] Ep 5000/5000 - Avg Reward: 87.82, Steps: 800

--> Experimento 'G1_Acciones_x50_v50_a10' guardado con éxito en el historial.

-> Resultado: Avg Reward=93.47 | Success Rate=1.00

[3/3] Evaluando: G1_Acciones_x50_v50_a20

[G1_Acciones_x50_v50_a20] Iniciando experimento con Q-Table de tamaño: (51, 51, 20)

[G1_Acciones_x50_v50_a20] Ep 1000/5000 - Avg Reward: -93.86, Steps: 3000

[G1_Acciones_x50_v50_a20] Ep 2000/5000 - Avg Reward: -73.76, Steps: 3000

[G1_Acciones_x50_v50_a20] Ep 3000/5000 - Avg Reward: -53.60, Steps: 3000

[G1_Acciones_x50_v50_a20] Ep 4000/5000 - Avg Reward: -33.49, Steps: 3000

[G1_Acciones_x50_v50_a20] Ep 5000/5000 - Avg Reward: -11.87, Steps: 3000

--> Experimento 'G1_Acciones_x50_v50_a20' guardado con éxito en el historial.

-> Resultado: Avg Reward=-0.84 | Success Rate=0.00

=== Tanda completada. Historial global cuenta con 3 combinaciones registradas. ===

```

evaluator.evaluate_group(
    env=env,
    x_bins_list=[30, 100],      # <--- Baja y alta resolución de
    posición
    vel_bins_list=[20, 100],    # <--- Baja y alta resolución de
    velocidad
    action_bins_list=[10],      # Fijamos el ganador tentativo del
    grupo anterior
    train_kwargs=train_kwargs,
    test_episodes=10,
    test_max_steps=3000,
    prefix="G2_Estados"
)

env.close()

```

=== Iniciando Tanda de 4 combinaciones ===

[1/4] Evaluando: G2_Estados_x30_v20_a10

[G2_Estados_x30_v20_a10] Iniciando experimento con Q-Table de tamaño:
(31, 21, 10)

[G2_Estados_x30_v20_a10] Ep 1000/5000 - Avg Reward: -98.14, Steps: 3000

[G2_Estados_x30_v20_a10] Ep 2000/5000 - Avg Reward: -42.42, Steps: 2026

[G2_Estados_x30_v20_a10] Ep 3000/5000 - Avg Reward: 53.00, Steps: 492

[G2_Estados_x30_v20_a10] Ep 4000/5000 - Avg Reward: 87.55, Steps: 181

[G2_Estados_x30_v20_a10] Ep 5000/5000 - Avg Reward: 92.74, Steps: 173

--> Experimento 'G2_Estados_x30_v20_a10' guardado con éxito en el historial.

-> Resultado: Avg Reward=92.56 | Success Rate=1.00

[2/4] Evaluando: G2_Estados_x30_v100_a10

[G2_Estados_x30_v100_a10] Iniciando experimento con Q-Table de tamaño:
(31, 101, 10)

[G2_Estados_x30_v100_a10] Ep 1000/5000 - Avg Reward: -98.62, Steps: 3000

[G2_Estados_x30_v100_a10] Ep 2000/5000 - Avg Reward: -75.82, Steps: 3000

[G2_Estados_x30_v100_a10] Ep 3000/5000 - Avg Reward: -47.35, Steps: 3000

[G2_Estados_x30_v100_a10] Ep 4000/5000 - Avg Reward: 6.00, Steps: 3000

[G2_Estados_x30_v100_a10] Ep 5000/5000 - Avg Reward: 81.03, Steps: 703

--> Experimento 'G2_Estados_x30_v100_a10' guardado con éxito en el historial.

-> Resultado: Avg Reward=92.63 | Success Rate=1.00

[3/4] Evaluando: G2_Estados_x100_v20_a10

[G2_Estados_x100_v20_a10] Iniciando experimento con Q-Table de tamaño: (101, 21, 10)

[G2_Estados_x100_v20_a10] Ep 1000/5000 - Avg Reward: -95.39, Steps: 3000

[G2_Estados_x100_v20_a10] Ep 2000/5000 - Avg Reward: -74.70, Steps: 3000

[G2_Estados_x100_v20_a10] Ep 3000/5000 - Avg Reward: -56.44, Steps: 3000

[G2_Estados_x100_v20_a10] Ep 4000/5000 - Avg Reward: -33.63, Steps: 3000

[G2_Estados_x100_v20_a10] Ep 5000/5000 - Avg Reward: -15.37, Steps: 3000

--> Experimento 'G2_Estados_x100_v20_a10' guardado con éxito en el historial.

-> Resultado: Avg Reward=-2.55 | Success Rate=0.00

[4/4] Evaluando: G2_Estados_x100_v100_a10

[G2_Estados_x100_v100_a10] Iniciando experimento con Q-Table de tamaño: (101, 101, 10)

[G2_Estados_x100_v100_a10] Ep 1000/5000 - Avg Reward: -102.21, Steps: 3000

[G2_Estados_x100_v100_a10] Ep 2000/5000 - Avg Reward: -80.29, Steps: 3000

[G2_Estados_x100_v100_a10] Ep 3000/5000 - Avg Reward: -60.02, Steps: 3000

[G2_Estados_x100_v100_a10] Ep 4000/5000 - Avg Reward: -38.36, Steps: 3000

[G2_Estados_x100_v100_a10] Ep 5000/5000 - Avg Reward: -16.58, Steps: 3000

--> Experimento 'G2_Estados_x100_v100_a10' guardado con éxito en el historial.

-> Resultado: Avg Reward=-3.71 | Success Rate=0.00

=== Tanda completada. Historial global cuenta con 7 combinaciones registradas. ===

*** PROCESANDO RESULTADOS DE LOS GRUPOS EJECUTADOS ***

```

=== MEJOR CONFIGURACIÓN EN EL HISTORIAL ACUMULADO ===
x_bins=50, vel_bins=50, action_bins=10
avg_reward=93.47, std_reward=0.76
avg_steps=733.90, success_rate=1.00
q_table_size=26010

```

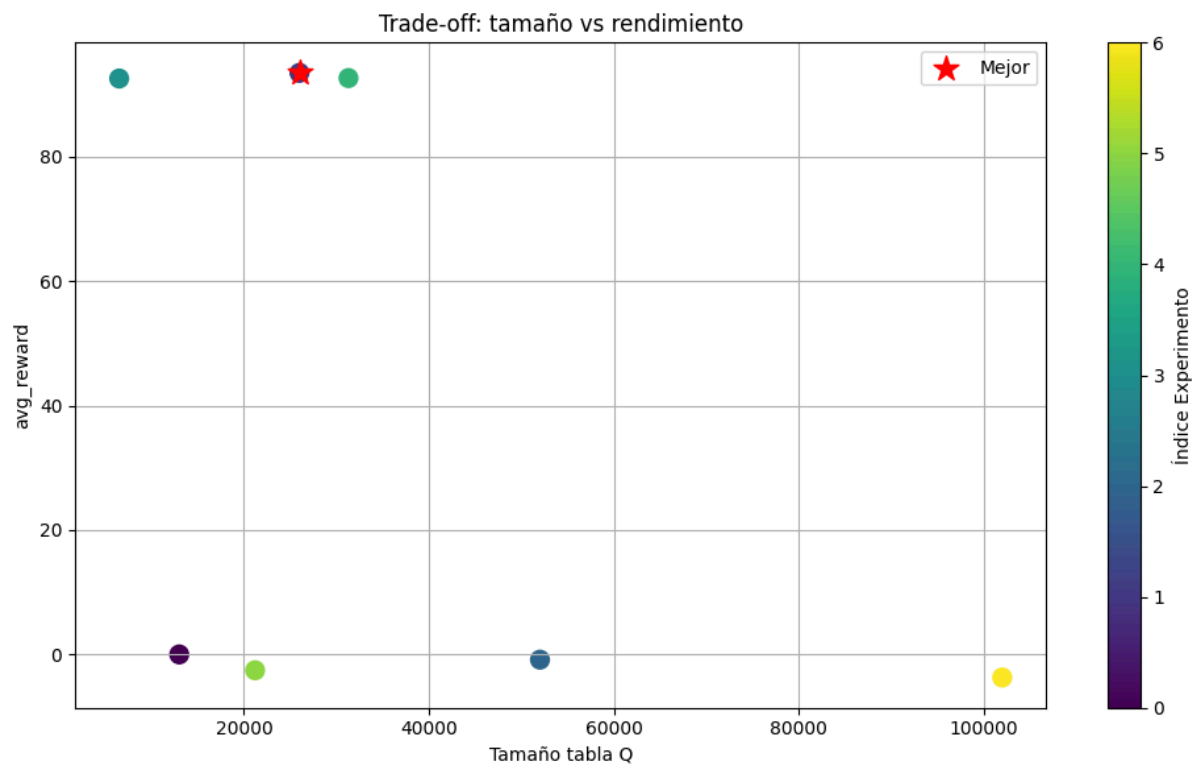
```

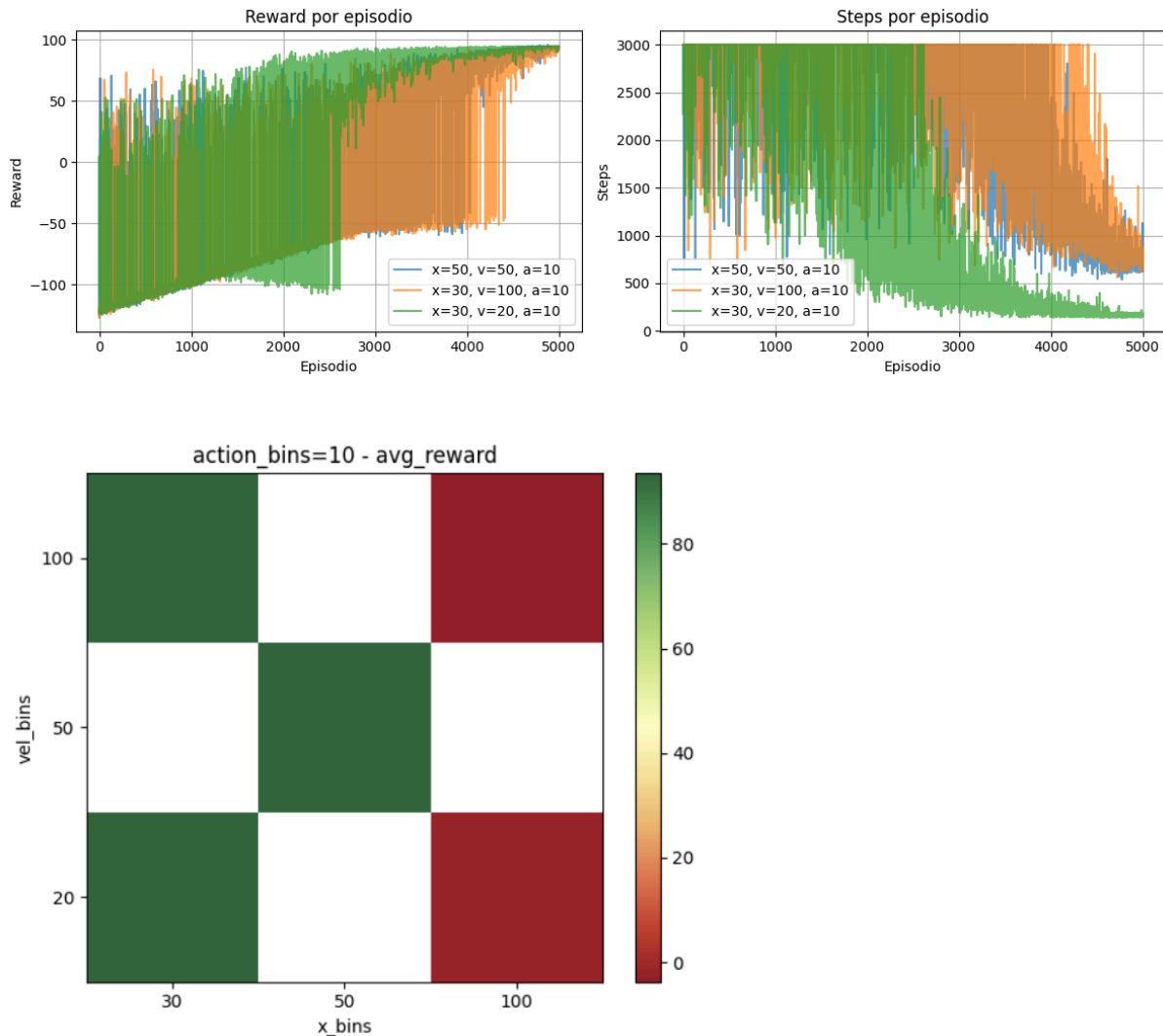
=== TOP 5 CONFIGURACIONES GENERALES ===

```

x	v	a	avg_reward	std_reward	avg_steps	success
q_size						

50	50	10	93.47	0.76	733.90	1.00
26010						
30	100	10	92.63	1.08	750.70	1.00
31310						
30	20	10	92.56	3.22	189.90	1.00
6510						
50	50	5	0.00	0.00	3000.00	0.00
13005						
50	50	20	-0.84	0.00	3000.00	0.00
52020						



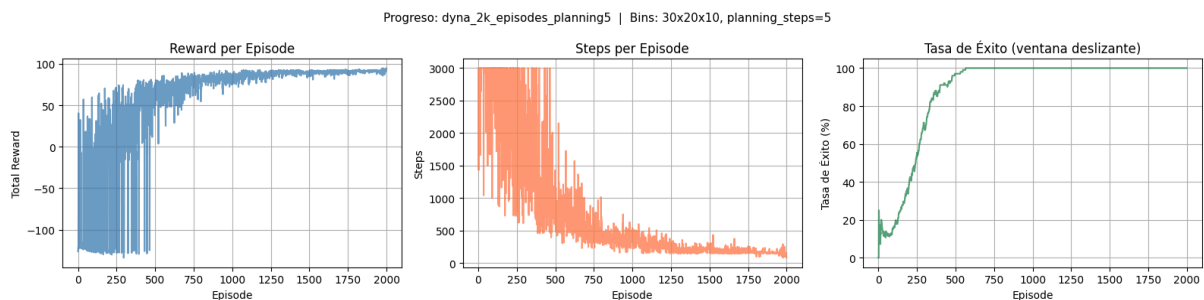


Anexo 1.11

Con el de mejor resultado re evaluamos pero con menor cantidad de episodios esta vez con un tiempo de 1m 15.8s que es una mejora respecto del Q learning

```
dyna_manager.train_run(
    env=env,
    run_id=id_d5,
    x_bins=30, vel_bins=20, action_bins=10,
    planning_steps=5,
    episodes=2000,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.0005,
    max_steps=3000,
)
```

```
[dyna_2k_episodes_planning5] Iniciando experimento con Q-Table de
tamaño: (31, 21, 10), planning_steps=5
[dyna_2k_episodes_planning5] Ep 400/2000 - Avg Reward: -42.56,
Steps: 686, Success: 53.2%
[dyna_2k_episodes_planning5] Ep 800/2000 - Avg Reward: 65.90,
Steps: 700, Success: 99.2%
[dyna_2k_episodes_planning5] Ep 1200/2000 - Avg Reward: 84.08,
Steps: 387, Success: 100.0%
[dyna_2k_episodes_planning5] Ep 1600/2000 - Avg Reward: 89.22,
Steps: 151, Success: 100.0%
[dyna_2k_episodes_planning5] Ep 2000/2000 - Avg Reward: 90.92,
Steps: 91, Success: 100.0%
--> Experimento 'dyna_2k_episodes_planning5' guardado con éxito en
el historial.
```



Seguimos reduciendo la cantidad de episodios:

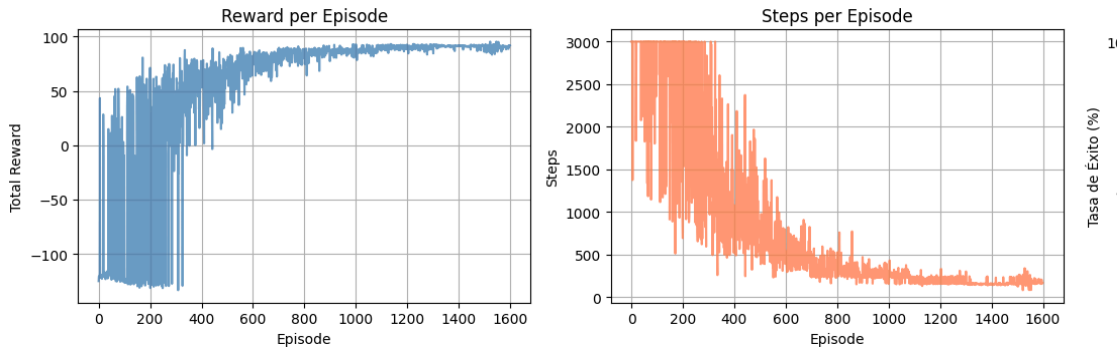
Con 1600 episodios obteniendo un tiempo de 1m 2.9s

```
dyna_manager.train_run(
    env=env,
    run_id=id_d6,
    x_bins=30, vel_bins=20, action_bins=10,
    planning_steps=5,
    episodes=1600,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.0005,
    max_steps=3000,
)
```

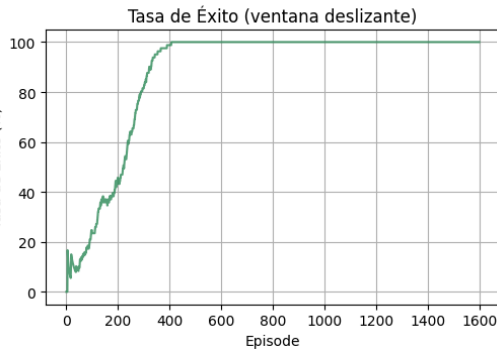
```
[dyna_1.6k_episodes_planning5] Iniciando experimento con Q-Table
de tamaño: (31, 21, 10), planning_steps=5
[dyna_1.6k_episodes_planning5] Ep 320/1600 - Avg Reward: -46.75,
Steps: 1719, Success: 50.3%
[dyna_1.6k_episodes_planning5] Ep 640/1600 - Avg Reward: 62.39,
Steps: 316, Success: 99.7%
[dyna_1.6k_episodes_planning5] Ep 960/1600 - Avg Reward: 83.20,
Steps: 230, Success: 100.0%
```

[dyna_1.6k_episodes_planning5] Ep 1280/1600 - Avg Reward: 89.07,
Steps: 226, Success: 100.0%
[dyna_1.6k_episodes_planning5] Ep 1600/1600 - Avg Reward: 90.82,
Steps: 166, Success: 100.0%
--> Experimento 'dyna_1.6k_episodes_planning5' guardado con éxito
en el historial.

Progreso: dyna_1.6k_episodes_planning5 | Bins: 30x20x10, planning_steps=5



5



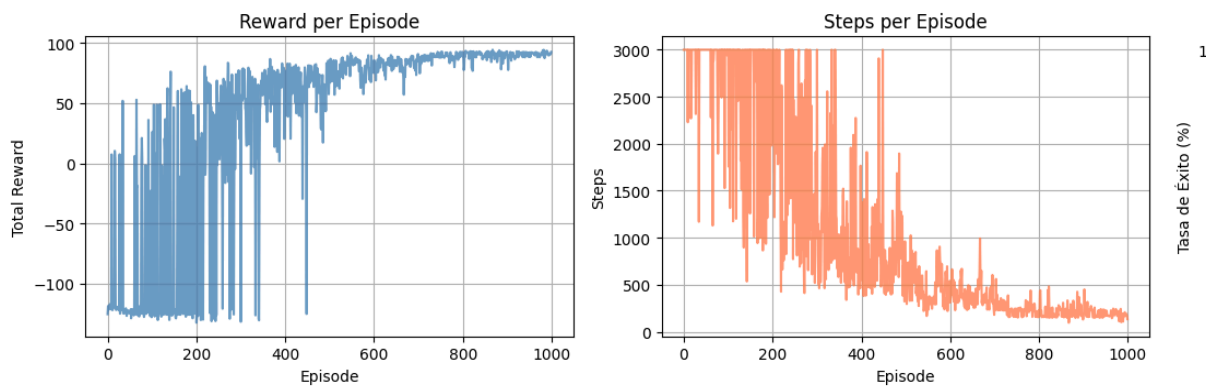
Evaluamos el efecto de incrementar los planning steps a 15
T = 1m 50.9s

```
dyna_manager.train_run(
    env=env,
    run_id=id_d7,
    x_bins=30, vel_bins=20, action_bins=10,
    planning_steps=15,
    episodes=1000,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.0005,
    max_steps=3000,
)
```

[dyna_1k_episodes_planning15] Iniciando experimento con Q-Table de
tamaño: (31, 21, 10), planning_steps=15

```
[dyna_1k_episodes_planning15] Ep 200/1000 - Avg Reward: -81.16,
Steps: 2048, Success: 29.5%
[dyna_1k_episodes_planning15] Ep 400/1000 - Avg Reward: 28.57,
Steps: 552, Success: 89.5%
[dyna_1k_episodes_planning15] Ep 600/1000 - Avg Reward: 70.08,
Steps: 541, Success: 99.5%
[dyna_1k_episodes_planning15] Ep 800/1000 - Avg Reward: 85.30,
Steps: 224, Success: 100.0%
[dyna_1k_episodes_planning15] Ep 1000/1000 - Avg Reward: 90.08,
Steps: 134, Success: 100.0%
--> Experimento 'dyna_1k_episodes_planning15' guardado con éxito
en el historial.
```

Progreso: dyna_1k_episodes_planning15 | Bins: 30x20x10, planning_steps=15



i=15



```

dyna_manager.train_run(
    env=env,
    run_id=id_d8,
    x_bins=30, vel_bins=20, action_bins=10,
    planning_steps=15,
    episodes=800,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.0005,
    max_steps=3000,
)

```

[dyna_800_episodes_planning15] Iniciando experimento con Q-Table de tamaño: (31, 21, 10), planning_steps=15

[dyna_800_episodes_planning15] Ep 160/800 - Avg Reward: -65.11, Steps: 3000, Success: 38.8%

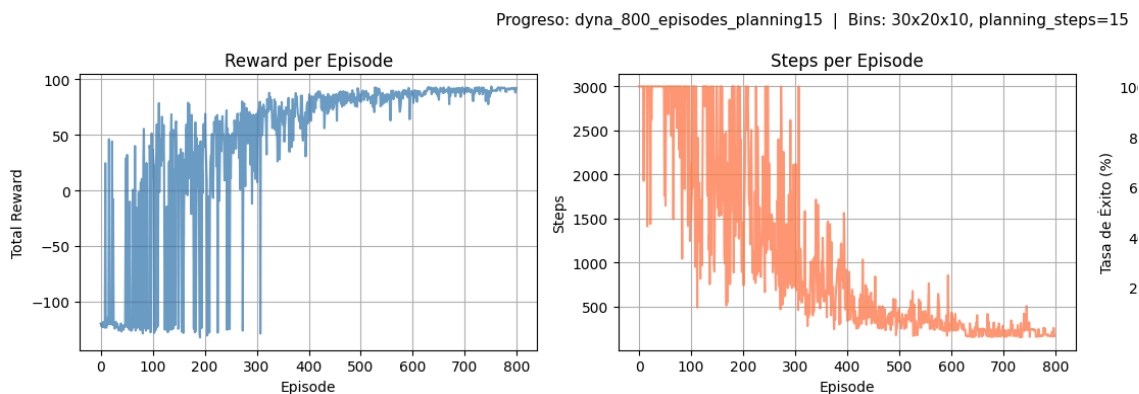
[dyna_800_episodes_planning15] Ep 320/800 - Avg Reward: 25.79, Steps: 1582, Success: 90.6%

[dyna_800_episodes_planning15] Ep 480/800 - Avg Reward: 72.74, Steps: 402, Success: 100.0%

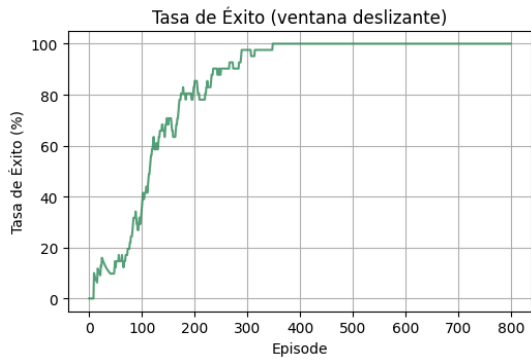
[dyna_800_episodes_planning15] Ep 640/800 - Avg Reward: 85.03, Steps: 151, Success: 100.0%

[dyna_800_episodes_planning15] Ep 800/800 - Avg Reward: 89.67, Steps: 161, Success: 100.0%

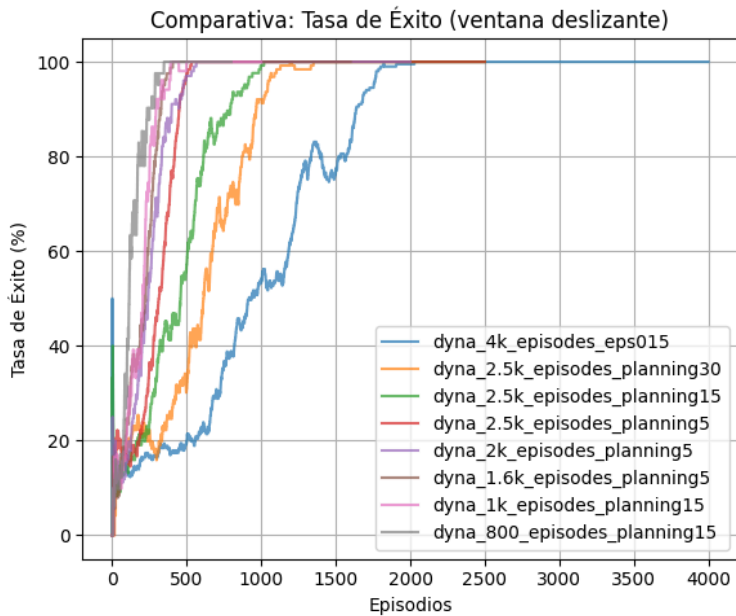
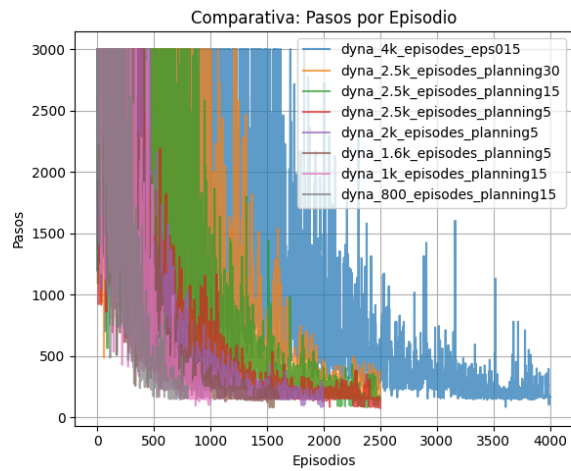
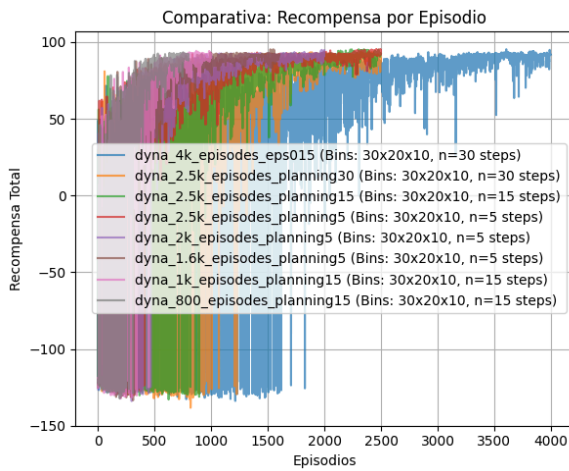
--> Experimento 'dyna_800_episodes_planning15' guardado con éxito en el historial.



s=15



Nuevamente graficamos todos los resultados



Anexo 1.12

Exploración Dyna Q sin mejora en los valores de ejecución

Para poder comparar recompensas y tiempos de ejecución

T=25m 8.0s

```

dyna_manager.train_run(
    env=env,
    run_id=id_d9,
    x_bins=100, vel_bins=100, action_bins=20,
    planning_steps=5,
    episodes=2000,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.0005,
    max_steps=3000,
)

```

[dynaog_2k_episodes_planning5] Iniciando experimento con Q-Table de tamaño: (101, 101, 20), planning_steps=5

[dynaog_2k_episodes_planning5] Ep 400/2000 - Avg Reward: -78.61, Steps: 3000, Success: 17.0%

[dynaog_2k_episodes_planning5] Ep 800/2000 - Avg Reward: 0.19, Steps: 1328, Success: 61.5%

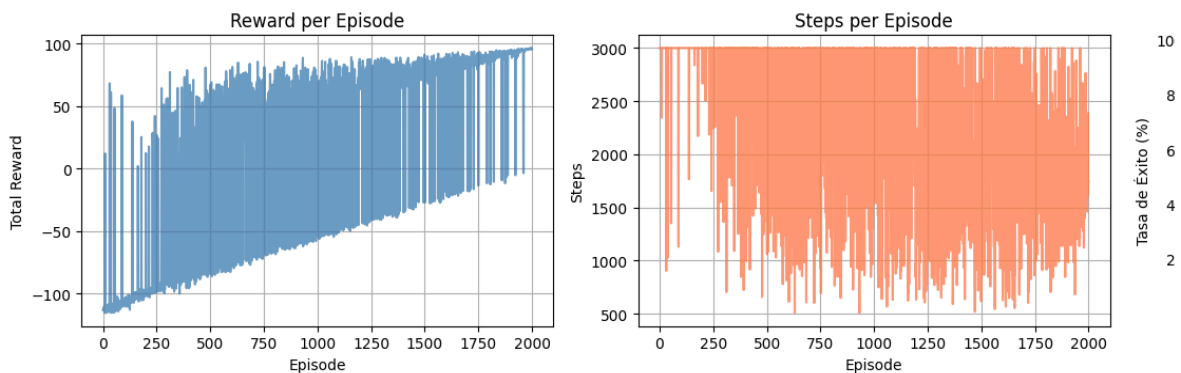
[dynaog_2k_episodes_planning5] Ep 1200/2000 - Avg Reward: 23.84, Steps: 1707, Success: 66.0%

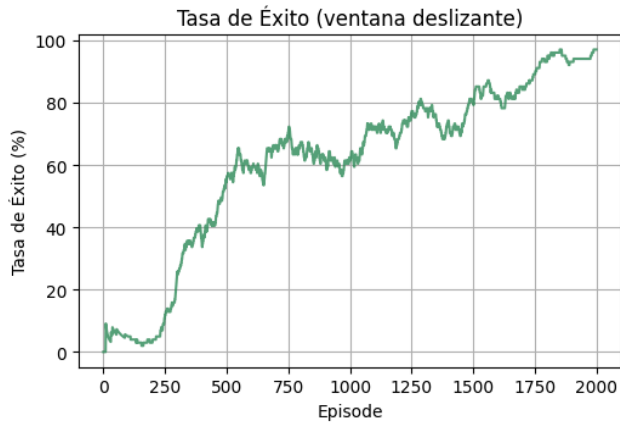
[dynaog_2k_episodes_planning5] Ep 1600/2000 - Avg Reward: 53.50, Steps: 3000, Success: 78.2%

[dynaog_2k_episodes_planning5] Ep 2000/2000 - Avg Reward: 81.60, Steps: 1742, Success: 92.2%

--> Experimento 'dynaog_2k_episodes_planning5' guardado con éxito en el historial.

Progreso: dynaog_2k_episodes_planning5 | Bins: 100x100x20, planning_steps=5



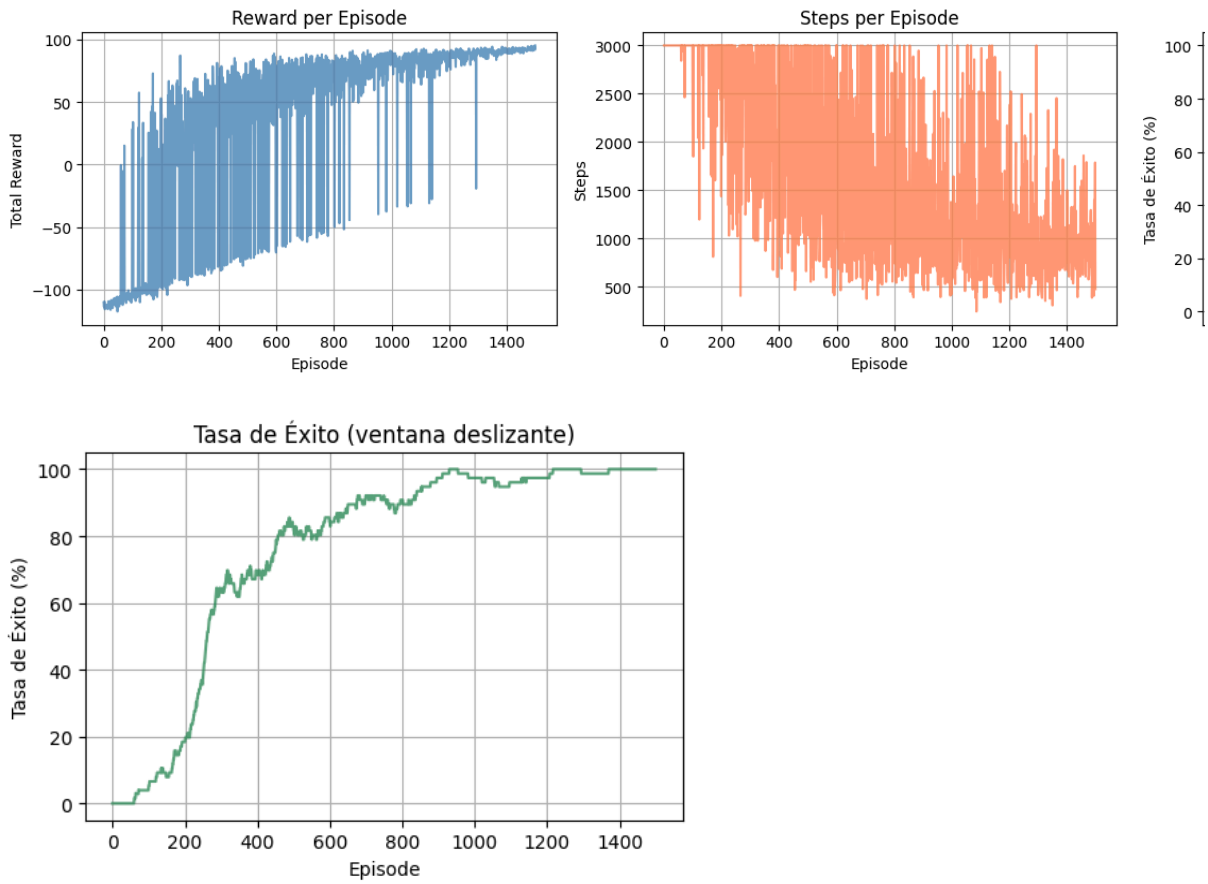


```
dyna_manager.train_run(
    env=env,
    run_id=id_d10,
    x_bins=100, vel_bins=100, action_bins=20,
    planning_steps=15,
    episodes=1500,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.0005,
    max_steps=3000,
)
```

T = 98m 36.4s

```
[dynaog_1.5k_episodes_planning15] Iniciando experimento con
Q-Table de tamaño: (101, 101, 20), planning_steps=15
[dynaog_1.5k_episodes_planning15] Ep 300/1500 - Avg Reward:
-68.13, Steps: 3000, Success: 25.3%
[dynaog_1.5k_episodes_planning15] Ep 600/1500 - Avg Reward: 25.11,
Steps: 3000, Success: 77.7%
[dynaog_1.5k_episodes_planning15] Ep 900/1500 - Avg Reward: 61.30,
Steps: 515, Success: 92.3%
[dynaog_1.5k_episodes_planning15] Ep 1200/1500 - Avg Reward:
78.86, Steps: 1671, Success: 97.3%
[dynaog_1.5k_episodes_planning15] Ep 1500/1500 - Avg Reward:
89.51, Steps: 478, Success: 99.7%
--> Experimento 'dynaog_1.5k_episodes_planning15' guardado con
éxito en el historial.
```

Progreso: dynaog_1.5k_episodes_planning15 | Bins: 100x100x20, planning_steps=15

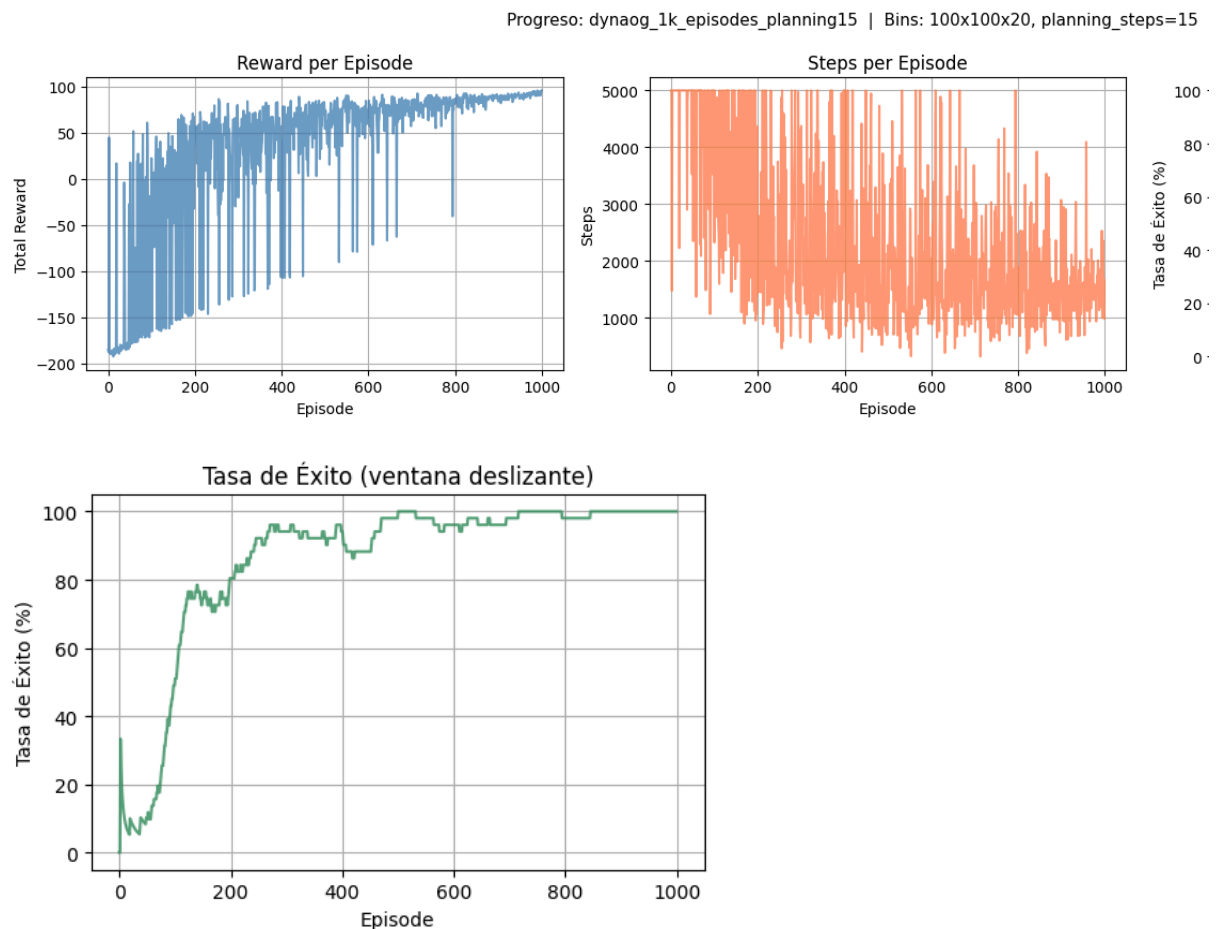


```
dyna_manager.train_run(  
    env=env,  
    run_id=id_d11,  
    x_bins=100, vel_bins=100, action_bins=20,  
    planning_steps=15,  
    episodes=1000,  
    alpha=0.05,  
    gamma=0.99,  
    epsilon_start=0.99,  
    epsilon_end=0.0005,  
    max_steps=5000,  
)
```

T = 90m 49.3s

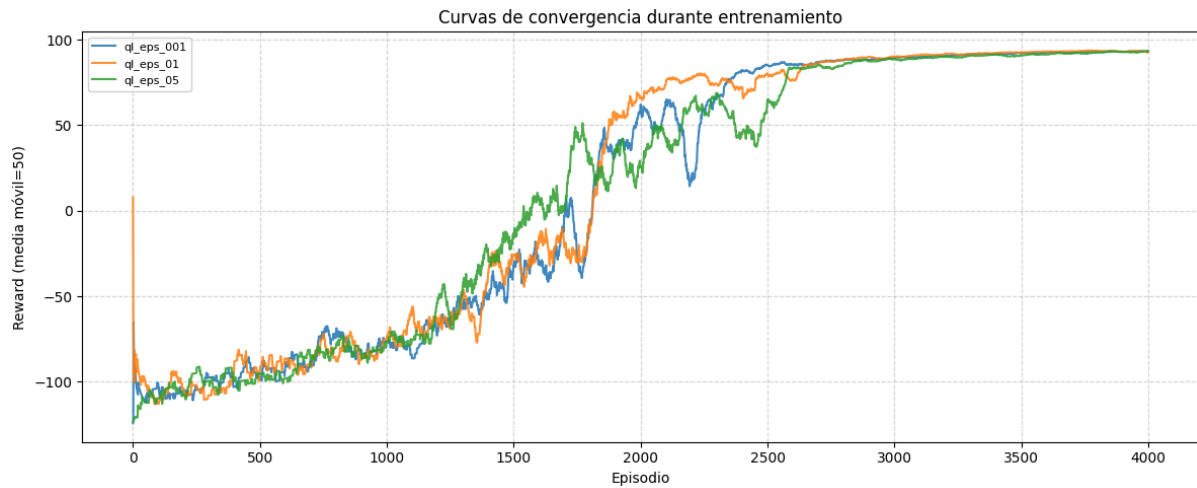
[dynaog_1k_episodes_planning15] Iniciando experimento con Q-Table de tamaño: (101, 101, 20), planning_steps=15
[dynaog_1k_episodes_planning15] Ep 200/1000 - Avg Reward: -79.19, Steps: 1406, Success: 53.5%

```
[dynaog_1k_episodes_planning15] Ep 400/1000 - Avg Reward: 31.11,
Steps: 2601, Success: 93.0%
[dynaog_1k_episodes_planning15] Ep 600/1000 - Avg Reward: 57.02,
Steps: 1935, Success: 95.5%
[dynaog_1k_episodes_planning15] Ep 800/1000 - Avg Reward: 73.52,
Steps: 1830, Success: 98.0%
[dynaog_1k_episodes_planning15] Ep 1000/1000 - Avg Reward: 87.99,
Steps: 986, Success: 100.0%
--> Experimento 'dynaog_1k_episodes_planning15' guardado con éxito
en el historial.
```



Anexo 1.13

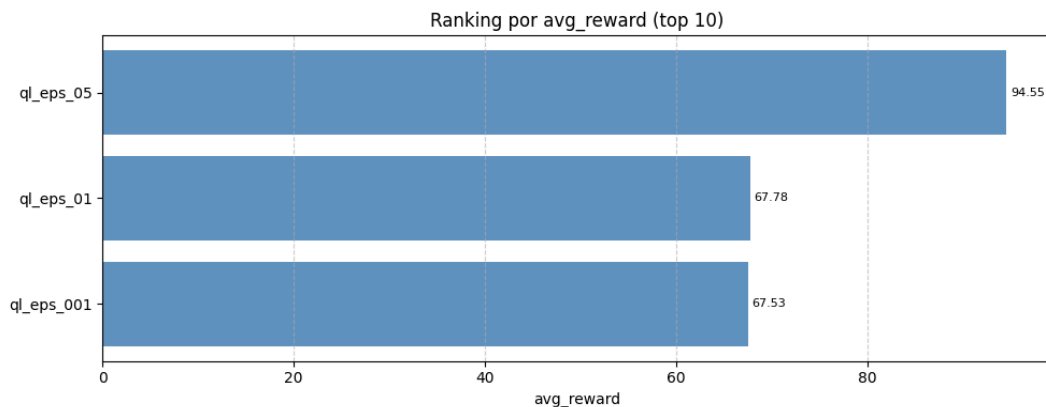
```
manager.train_run(
    env=env,
    run_id=id_qe1,
    x_bins=30,
    vel_bins=20,
    action_bins=10,
    episodes=4000,
    alpha=0.05,
    gamma=0.99,
    epsilon_start=0.99,
    epsilon_end=0.001,
    max_steps=3000
)
```



```

=== Evaluando 3 experimento(s), 10 episodios c/u ===
[ql_eps_001] avg_reward=67.53 ± 78.15 | success=90% |
avg_steps=539
[ql_eps_01] avg_reward=67.78 ± 78.47 | success=90% |
avg_steps=554
[ql_eps_05] avg_reward=94.55 ± 0.35 | success=100% |
avg_steps=150
=== Evaluación completada ===

```



Luego utilizamos nuestro método para generar los reportes:

```

=====
REPORTE DE EVALUACIÓN DE HIPERPARÁMETROS
=====

--- Top 5 por avg_reward ---
run_id                                avg_reward  std_reward  success  avg_steps  params
-----
ql_eps_05                             94.55       0.35       100%     150       x_bins=30
vel_bins=20  action_bins=10  alpha=0.05
ql_eps_01                             67.78       78.47       90%      554       x_bins=30
vel_bins=20  action_bins=10  alpha=0.05
ql_eps_001                            67.53       78.15       90%      539       x_bins=30
vel_bins=20  action_bins=10  alpha=0.05

--- Velocidad de convergencia (umbral reward > 0) ---
ql_eps_001                             episodio 1703
ql_eps_01                              episodio 1812

```

ql_eps_05

episodio 1539

--- Estabilidad final (std últimos 200 ep de entrenamiento) ---

ql_eps_001 std=1.733

ql_eps_01 std=1.910

ql_eps_05 std=1.944

===== MEJOR CONFIGURACIÓN =====

run_id : ql_eps_05

Parámetros : {'x_bins': 30, 'vel_bins': 20, 'action_bins': 10, 'alpha': 0.05}

avg_reward : 94.55 ± 0.35

success_rate : 100%

avg_steps : 150

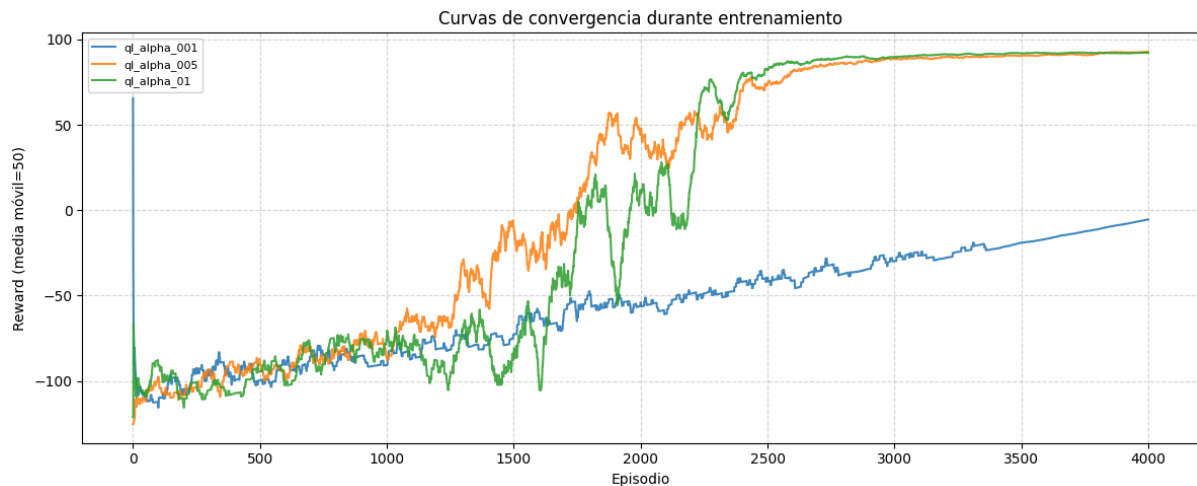
Convergencia : episodio 1539

Estabilidad : std=1.944 (últimos 200 ep)

=====

epsilon_end	Tiempo de ejecución
0.001	3m 23.5s
0.01	3m 19.1s
0.05	3m 22.9s

Exploración con alpha 0.01, 0.05 y 0.1



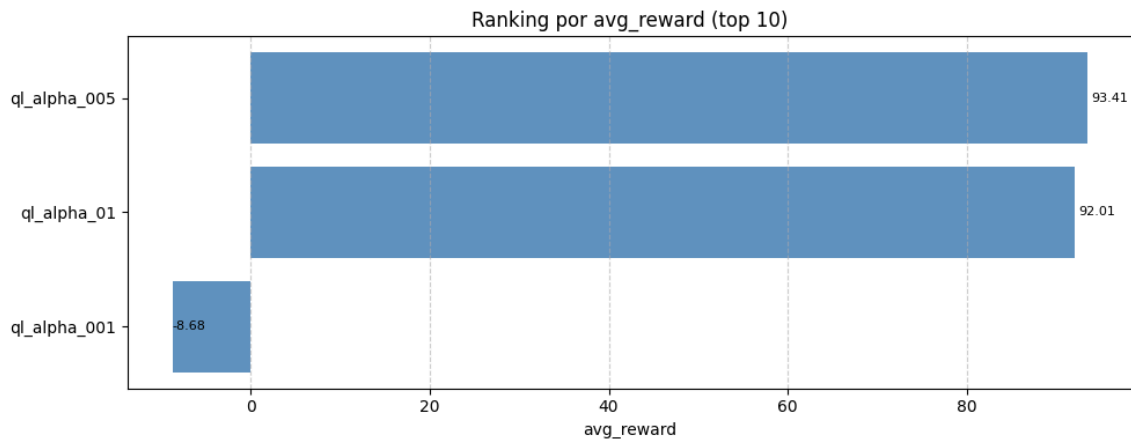
=== Evaluando 3 experimento(s), 10 episodios c/u ===

[ql_alpha_001] avg_reward=-8.68 ± 2.45 | success=0% | avg_steps=4000

[ql_alpha_005] avg_reward=93.41 ± 0.61 | success=100% | avg_steps=155

[ql_alpha_01] avg_reward=92.01 ± 1.04 | success=100% | avg_steps=141

=== Evaluación completada ===



=====

REPORTE DE EVALUACIÓN DE HIPERPARÁMETROS

=====

--- Top 5 por avg_reward ---

run_id		avg_reward	std_reward	success	avg_steps	params

ql_alpha_005		93.41	0.61	100%	155	x_bins=30
ql_alpha_01		92.01	1.04	100%	141	x_bins=30
ql_alpha_001		-8.68	2.45	0%	4000	x_bins=30
ql_alpha_005	vel_bins=20 action_bins=10 alpha=0.05					
ql_alpha_01	vel_bins=20 action_bins=10 alpha=0.1					
ql_alpha_001	vel_bins=20 action_bins=10 alpha=0.01					

--- Velocidad de convergencia (umbral reward > 0) ---

ql_alpha_001	episodio no convergió
ql_alpha_005	episodio 1725
ql_alpha_01	episodio 1755

--- Estabilidad final (std últimos 200 ep de entrenamiento) ---

ql_alpha_01	std=1.021
ql_alpha_005	std=1.570
ql_alpha_001	std=1.676

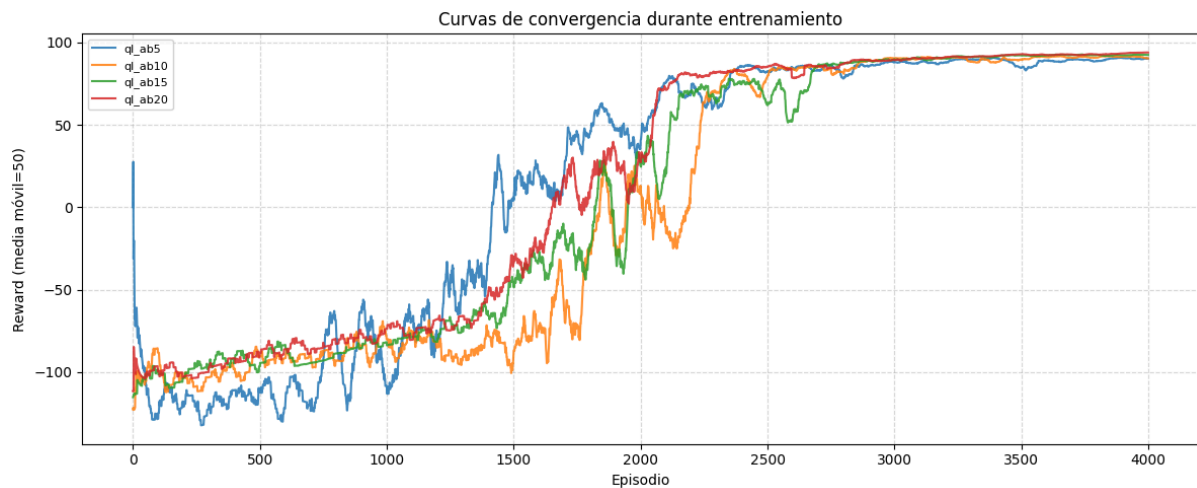
===== MEJOR CONFIGURACIÓN =====

run_id	: ql_alpha_005
Parámetros	: {'x_bins': 30, 'vel_bins': 20, 'action_bins': 10, 'alpha': 0.05}
avg_reward	: 93.41 ± 0.61
success_rate	: 100%
avg_steps	: 155
Convergencia	: episodio 1725
Estabilidad	: std=1.570 (últimos 200 ep)

=====

alpha	Tiempo de ejecución
0.01	6m 27.9s
0.05	3m 19.9s
0.1	3m 16.2s

Action Bins 5, 10, 15, 20



```

=== Evaluando 4 experimento(s), 10 episodios c/u ===
[ql_ab5] avg_reward=90.38 ± 1.42 | success=100% | avg_steps=128
[ql_ab10] avg_reward=90.73 ± 2.97 | success=100% | avg_steps=138
[ql_ab15] avg_reward=54.35 ± 114.72 | success=90% |
avg_steps=531
[ql_ab20] avg_reward=93.89 ± 0.19 | success=100% | avg_steps=165
=== Evaluación completada ===

```

=====

REPORTE DE EVALUACIÓN DE HIPERPARÁMETROS

=====

```

--- Top 5 por avg_reward ---
run_id                                avg_reward  std_reward  success  avg_steps  params
-----
ql_ab20                                93.89        0.19      100%      165  x_bins=30
vel_bins=20  action_bins=20  alpha=0.1
ql_ab10                                90.73        2.97      100%      138  x_bins=30
vel_bins=20  action_bins=10  alpha=0.1
ql_ab5                                 90.38        1.42      100%      128  x_bins=30
vel_bins=20  action_bins=5   alpha=0.1
ql_ab15                                54.35       114.72      90%      531  x_bins=30
vel_bins=20  action_bins=15  alpha=0.1

--- Velocidad de convergencia (umbral reward > 0) ---
ql_ab5                                episodio 1415
ql_ab10                                episodio 1840
ql_ab15                                episodio 1821
ql_ab20                                episodio 1652

--- Estabilidad final (std últimos 200 ep de entrenamiento) ---
ql_ab15                                std=0.955

```

ql_ab20	std=1.069
ql_ab5	std=1.975
ql_ab10	std=2.473

===== MEJOR CONFIGURACIÓN =====

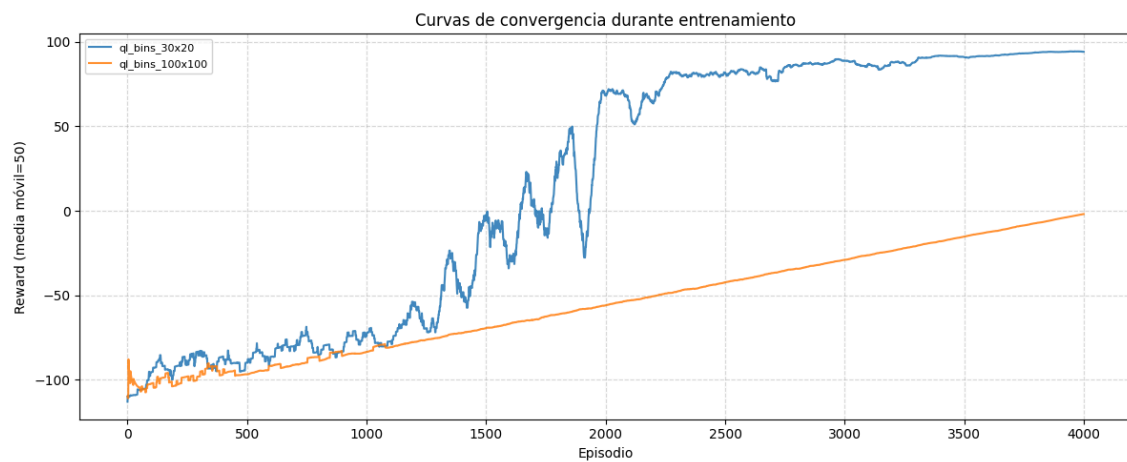
```

run_id      : ql_ab20
Parámetros  : {'x_bins': 30, 'vel_bins': 20, 'action_bins': 20, 'alpha': 0.1}
avg_reward   : 93.89 ± 0.19
success_rate : 100%
avg_steps    : 165
Convergencia : episodio 1652
Estabilidad  : std=1.069 (últimos 200 ep)

```

=====

action_bins	Tiempo de ejecución
5	2m 28.8s
10	3m 19.8s
15	3m 28.4s
20	3m 23.9s

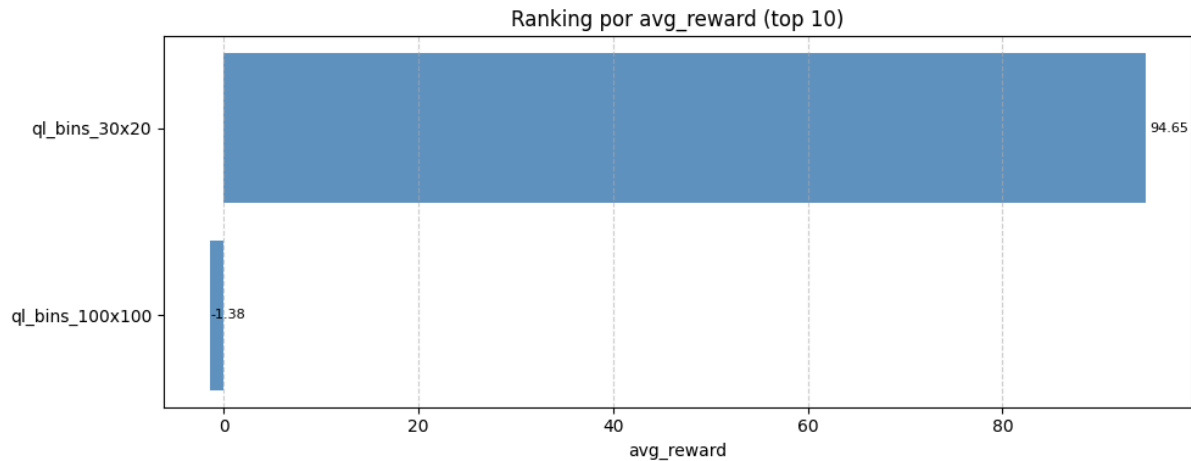


=== Evaluando 2 experimento(s), 10 episodios c/u ===

[ql_bins_30x20] avg_reward=94.65 ± 0.38 | success=100% | avg_steps=153

[ql_bins_100x100] avg_reward=-1.38 ± 0.08 | success=0% | avg_steps=4000

=== Evaluación completada ===



REPORTE DE EVALUACIÓN DE HIPERPARÁMETROS

--- Top 5 por avg_reward ---

run_id	avg_reward	std_reward	success	avg_steps
params				

ql_bins_30x20	94.65	0.38	100%	153
x_bins=30 vel_bins=20 action_bins=20 alpha=0.1				
ql_bins_100x100	-1.38	0.08	0%	4000
x_bins=100 vel_bins=100 action_bins=20 alpha=0.1				

--- Velocidad de convergencia (umbral reward > 0) ---

ql_bins_30x20	episodio 1644
ql_bins_100x100	episodio no convergió

--- Estabilidad final (std últimos 200 ep de entrenamiento) ---

ql_bins_30x20	std=0.829
ql_bins_100x100	std=1.644

===== MEJOR CONFIGURACIÓN =====

run_id	: ql_bins_30x20
Parámetros	: {'x_bins': 30, 'vel_bins': 20, 'action_bins': 20, 'alpha': 0.1}
avg_reward	: 94.65 ± 0.38
success_rate	: 100%
avg_steps	: 153
Convergencia	: episodio 1644
Estabilidad	: std=0.829 (últimos 200 ep)

bins(x_bins y vel_bins)	Tiempo de ejecución
30x20	3m 15.2s

100x100	7m 6.7s
---------	---------

Evaluación en conjunto de los Dyna Q ejecutados anteriormente

=== Evaluando 11 experimento(s), 10 episodios c/u ===

[dyna_4k_episodes_eps015] avg_reward=91.89 ± 0.28 | success=100%
| avg_steps=165

[dyna_2.5k_episodes_planning30] avg_reward=90.75 ± 1.33 |
success=100% | avg_steps=180

[dyna_2.5k_episodes_planning15] avg_reward=92.28 ± 0.80 |
success=100% | avg_steps=164

[dyna_2.5k_episodes_planning5] avg_reward=93.01 ± 0.98 |
success=100% | avg_steps=113

[dyna_2k_episodes_planning5] avg_reward=64.92 ± 54.70 |
success=80% | avg_steps=901

[dyna_1.6k_episodes_planning5] avg_reward=90.97 ± 0.41 |
success=100% | avg_steps=168

[dyna_1k_episodes_planning15] avg_reward=90.77 ± 8.64 |
success=100% | avg_steps=149

[dyna_800_episodes_planning15] avg_reward=71.94 ± 38.44 |
success=80% | avg_steps=939

[dynaog_2k_episodes_planning5] avg_reward=-1.13 ± 0.02 |
success=0% | avg_steps=4000

[dynaog_1.5k_episodes_planning15] avg_reward=93.54 ± 1.46 |
success=100% | avg_steps=1121

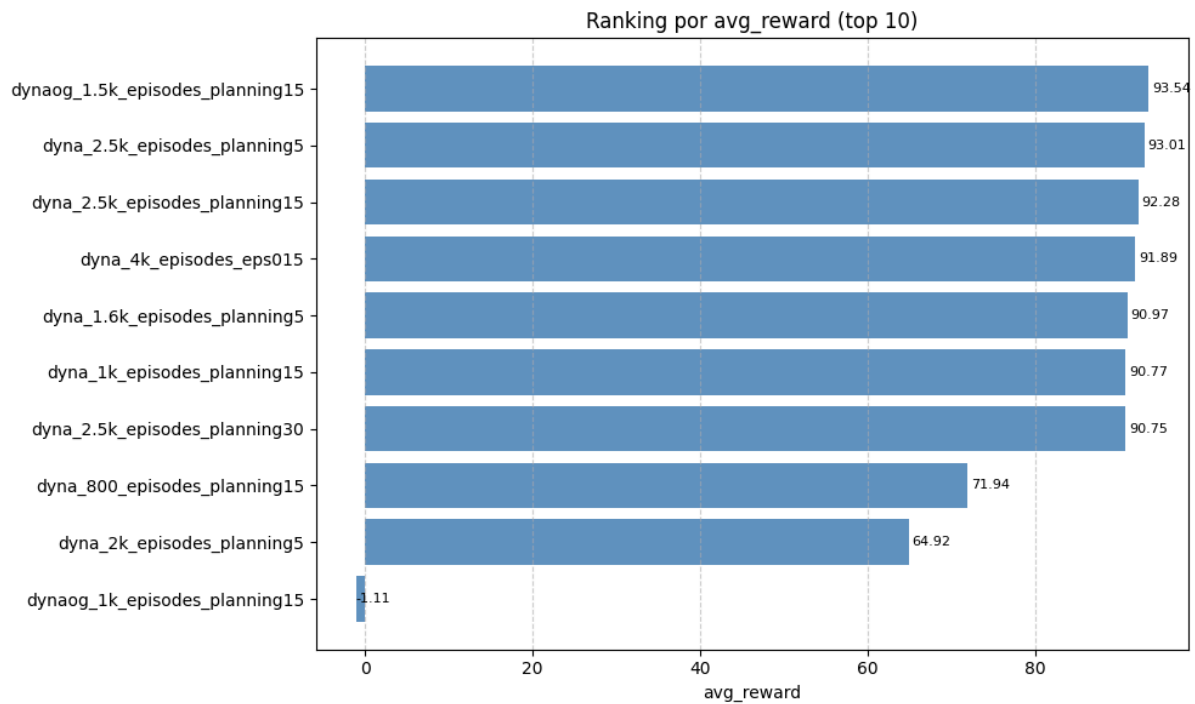
[dynaog_1k_episodes_planning15] avg_reward=-1.11 ± 0.00 |
success=0% | avg_steps=4000

=== Evaluación completada ===

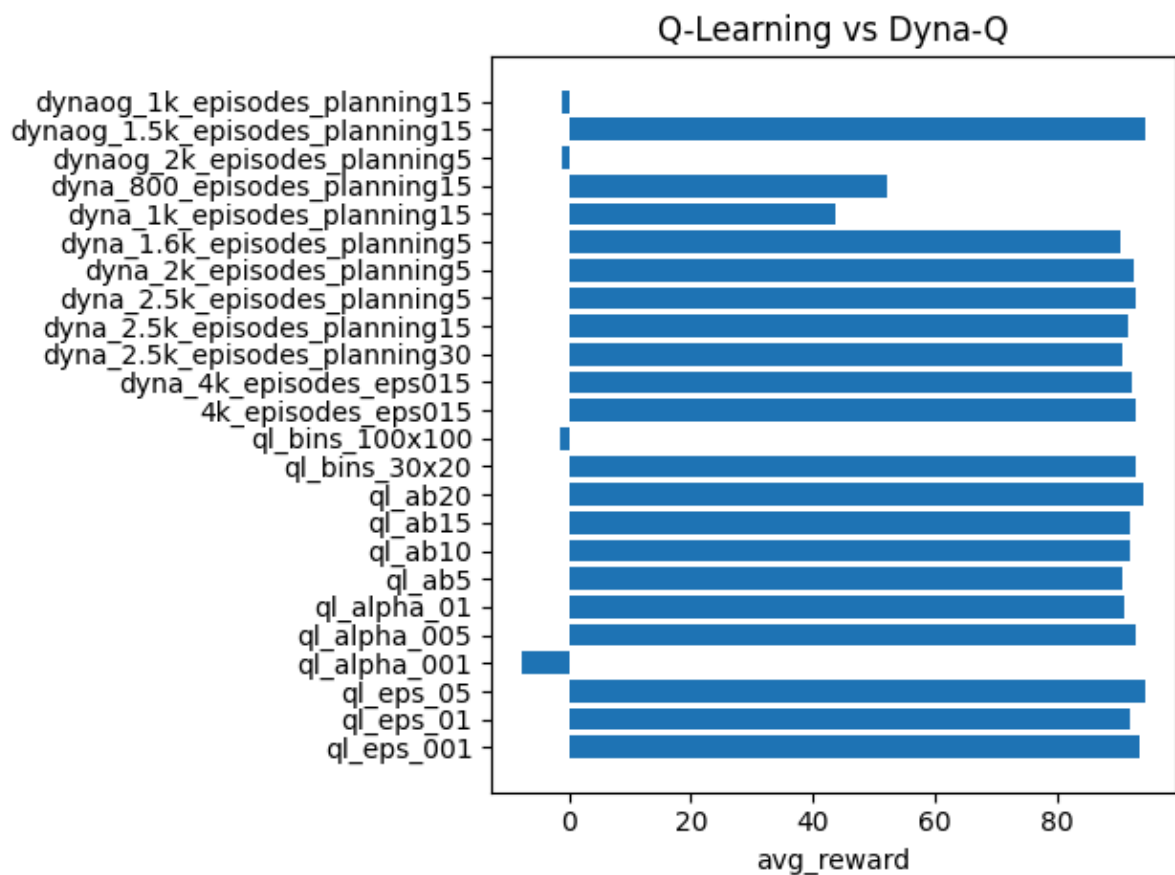
===== MEJOR CONFIGURACIÓN =====

run_id : dynaog_1.5k_episodes_planning15
Parámetros : {'x_bins': 100, 'vel_bins': 100, 'action_bins':
20, 'planning_steps': 15, 'alpha': 0.05}
avg_reward : 93.54 ± 1.46
success_rate : 100%
avg_steps : 1121
Convergencia : episodio 274
Estabilidad : std=2.032 (últimos 200 ep)

=====



Comparación Dyna Q y Q-Learning



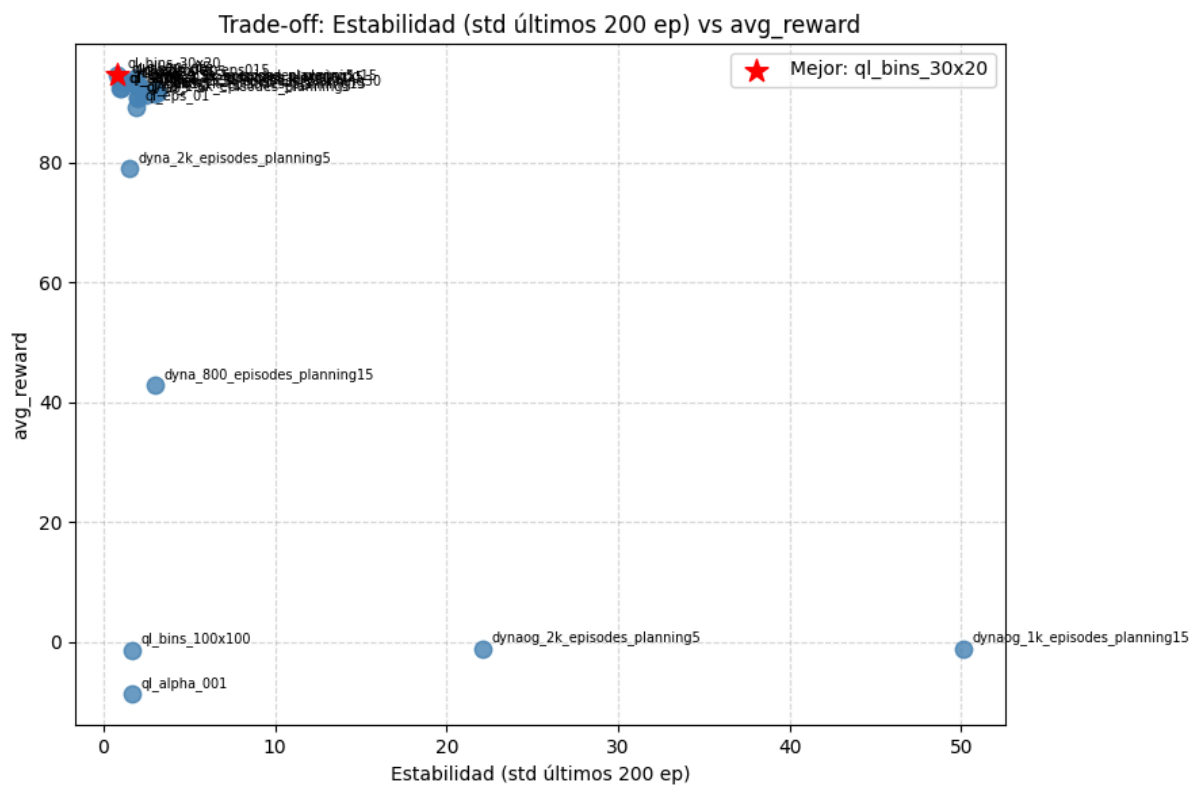
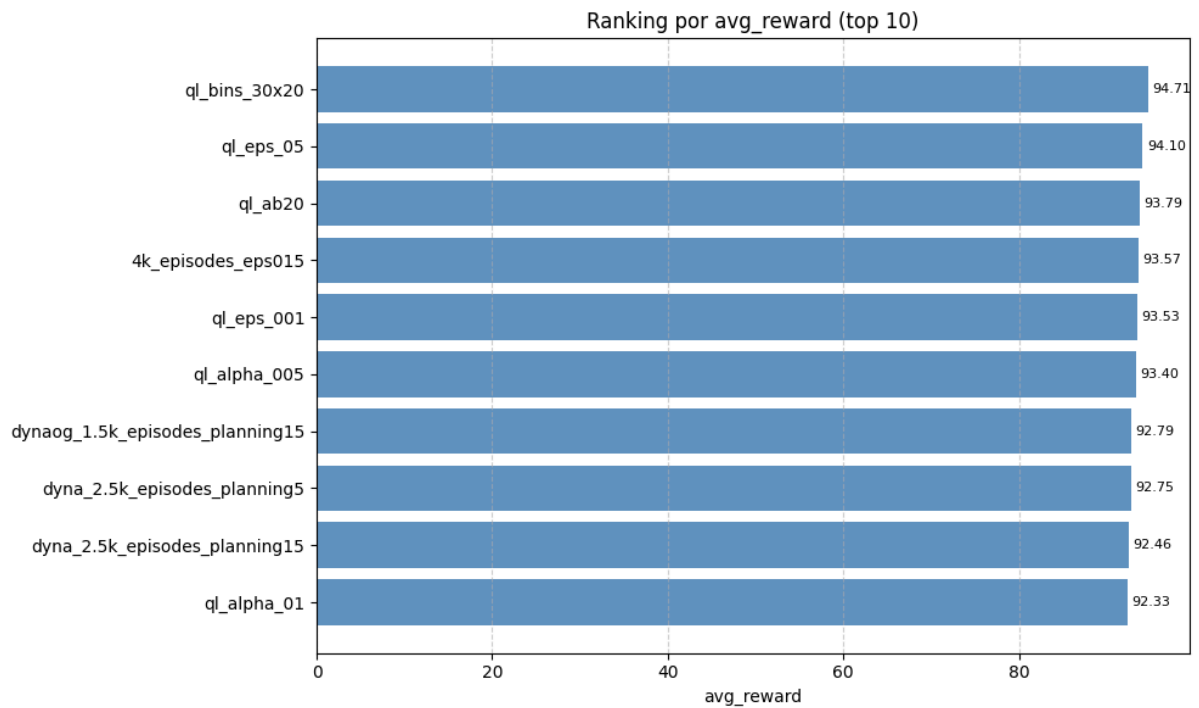
=== Evaluando 13 experimento(s), 10 episodios c/u ===

[4k_episodes_eps015] avg_reward=92.92 ± 2.81 | success=100% | avg_steps=170

[ql_eps_001] avg_reward=93.55 ± 0.42 | success=100% |
avg_steps=156
[ql_eps_01] avg_reward=92.15 ± 2.31 | success=100% |
avg_steps=200
[ql_eps_05] avg_reward=94.43 ± 0.47 | success=100% |
avg_steps=157
[ql_alpha_001] avg_reward=-7.58 ± 2.65 | success=0% |
avg_steps=4000
[ql_alpha_005] avg_reward=93.07 ± 0.94 | success=100% |
avg_steps=160
[ql_alpha_01] avg_reward=91.20 ± 1.26 | success=100% |
avg_steps=151
[ql_ab5] avg_reward=90.65 ± 1.38 | success=100% | avg_steps=122
[ql_ab10] avg_reward=92.02 ± 1.29 | success=100% | avg_steps=124
[ql_ab15] avg_reward=92.03 ± 1.49 | success=100% | avg_steps=153
[ql_ab20] avg_reward=94.08 ± 0.27 | success=100% | avg_steps=164
[ql_bins_30x20] avg_reward=92.91 ± 3.29 | success=100% |
avg_steps=194
[ql_bins_100x100] avg_reward=-1.33 ± 0.09 | success=0% |
avg_steps=4000
=== Evaluación completada ===

=== Evaluando 11 experimento(s), 10 episodios c/u ===

[dyna_4k_episodes_eps015] avg_reward=92.25 ± 1.20 | success=100%
| avg_steps=160
[dyna_2.5k_episodes_planning30] avg_reward=90.62 ± 1.45 |
success=100% | avg_steps=180
[dyna_2.5k_episodes_planning15] avg_reward=91.83 ± 1.73 |
success=100% | avg_steps=169
[dyna_2.5k_episodes_planning5] avg_reward=92.95 ± 1.12 |
success=100% | avg_steps=113
[dyna_2k_episodes_planning5] avg_reward=92.73 ± 1.66 |
success=100% | avg_steps=115
[dyna_1.6k_episodes_planning5] avg_reward=90.57 ± 0.51 |
success=100% | avg_steps=168
[dyna_1k_episodes_planning15] avg_reward=43.60 ± 147.64 |
success=90% | avg_steps=512
[dyna_800_episodes_planning15] avg_reward=52.24 ± 46.69 |
success=60% | avg_steps=1732
[dynaog_2k_episodes_planning5] avg_reward=-1.14 ± 0.02 |
success=0% | avg_steps=4000
[dynaog_1.5k_episodes_planning15] avg_reward=94.43 ± 0.47 |
success=100% | avg_steps=942
[dynaog_1k_episodes_planning15] avg_reward=-1.11 ± 0.00 |
success=0% | avg_steps=4000
=== Evaluación completada ===



REPORTE DE EVALUACIÓN DE HIPERPARÁMETROS

--- Top 5 por avg_reward ---

run_id	avg_reward	std_reward
success	avg_steps	params

```

-----
ql_bins_30x20                                94.71            0.26
100%          152  x_bins=30  vel_bins=20  action_bins=20  alpha=0.1
ql_eps_05                                94.10            0.53
100%          162  x_bins=30  vel_bins=20  action_bins=10
alpha=0.05
ql_ab20                                93.79            0.90
100%          170  x_bins=30  vel_bins=20  action_bins=20  alpha=0.1
4k_episodes_eps015                        93.57            1.36
100%          171  x_bins=30  vel_bins=20  action_bins=10
alpha=0.05
ql_eps_001                                93.53            0.40
100%          171  x_bins=30  vel_bins=20  action_bins=10
alpha=0.05

```

--- Velocidad de convergencia (umbral reward > 0) ---

```

4k_episodes_eps015                episodio 1802
ql_eps_001                        episodio 1703
ql_eps_01                         episodio 1812
ql_eps_05                         episodio 1539
ql_alpha_001                      episodio no convergió
ql_alpha_005                      episodio 1725
ql_alpha_01                       episodio 1755
ql_ab5                            episodio 1415
ql_ab10                           episodio 1840
ql_ab15                           episodio 1821
ql_ab20                           episodio 1652
ql_bins_30x20                     episodio 1644
ql_bins_100x100                   episodio no convergió
dyna_4k_episodes_eps015           episodio 1186
dyna_2.5k_episodes_planning30     episodio 781
dyna_2.5k_episodes_planning15    episodio 566
dyna_2.5k_episodes_planning5     episodio 372
dyna_2k_episodes_planning5        episodio 310
dyna_1.6k_episodes_planning5     episodio 280
dyna_1k_episodes_planning15       episodio 254
dyna_800_episodes_planning15     episodio 201
dynaog_2k_episodes_planning5     episodio 520
dynaog_1.5k_episodes_planning15  episodio 274
dynaog_1k_episodes_planning15    episodio 147

```

--- Estabilidad final (std últimos 200 ep de entrenamiento) ---

```

ql_bins_30x20                std=0.829
ql_ab15                      std=0.955
ql_alpha_01                  std=1.021
ql_ab20                      std=1.069
4k_episodes_eps015          std=1.235

```

dyna_2k_episodes_planning5	std=1.495
ql_alpha_005	std=1.570
ql_bins_100x100	std=1.644
ql_alpha_001	std=1.676
ql_eps_001	std=1.733
ql_eps_01	std=1.910
ql_eps_05	std=1.944
dyna_1.6k_episodes_planning5	std=1.969
ql_ab5	std=1.975
dynaog_1.5k_episodes_planning15	std=2.032
dyna_2.5k_episodes_planning5	std=2.100
dyna_2.5k_episodes_planning15	std=2.154
ql_ab10	std=2.473
dyna_800_episodes_planning15	std=3.002
dyna_1k_episodes_planning15	std=3.049
dyna_4k_episodes_eps015	std=3.094
dyna_2.5k_episodes_planning30	std=3.189
dynaog_2k_episodes_planning5	std=22.074
dynaog_1k_episodes_planning15	std=50.102

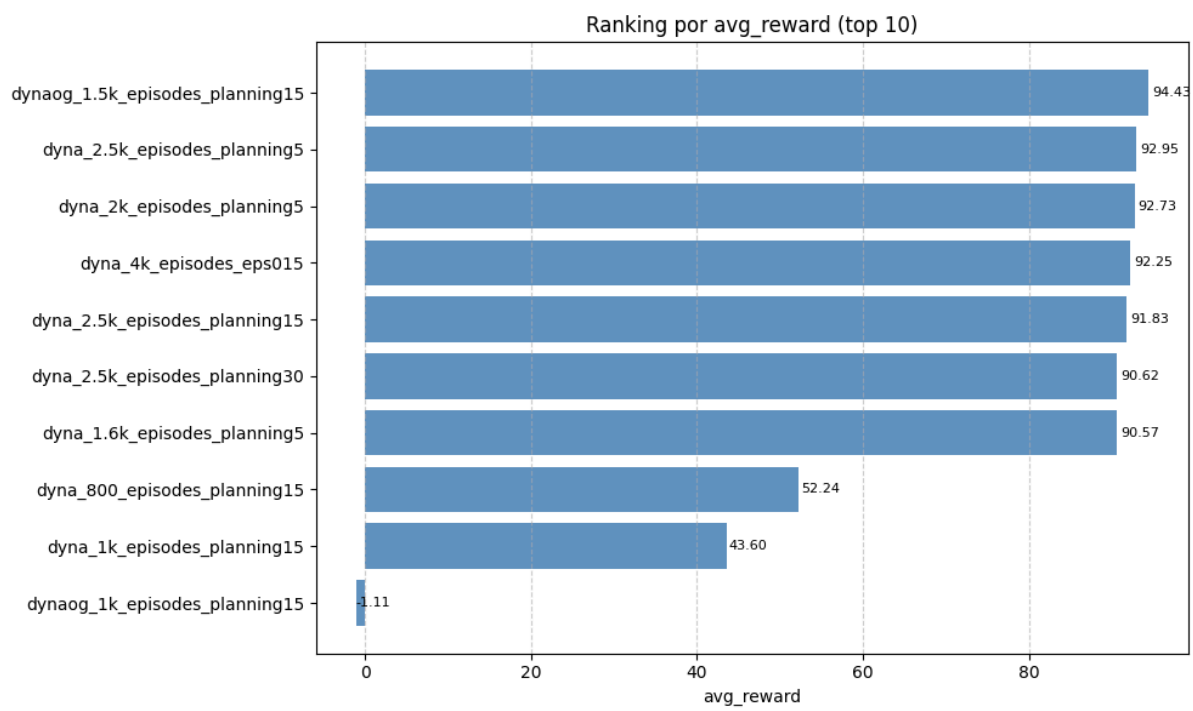
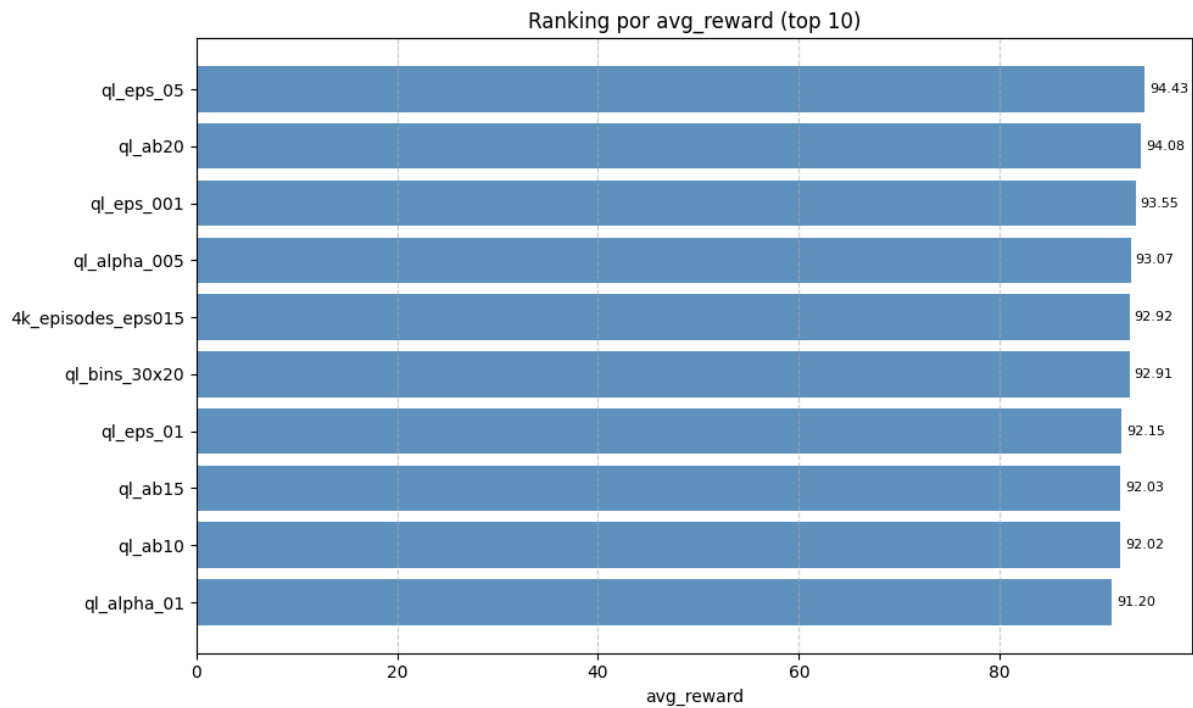
===== MEJOR CONFIGURACIÓN =====

```

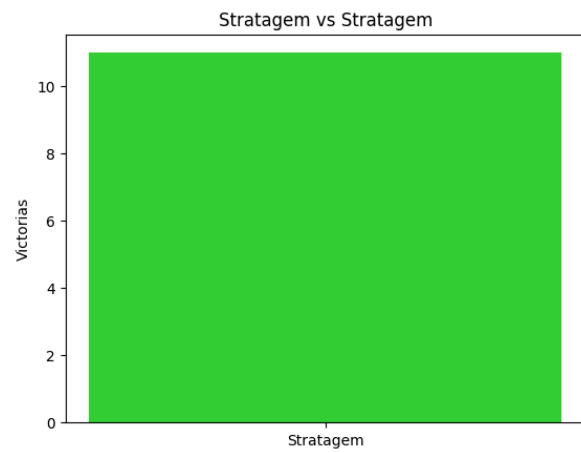
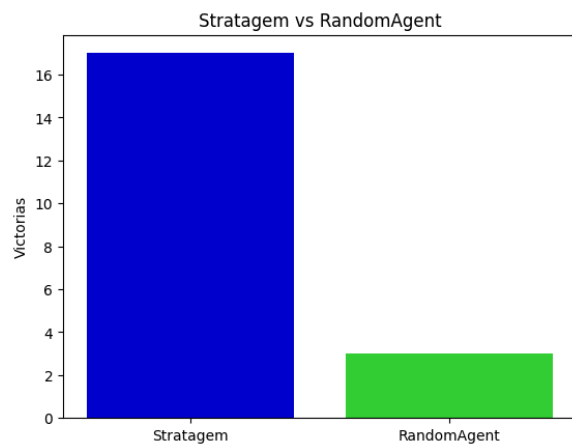
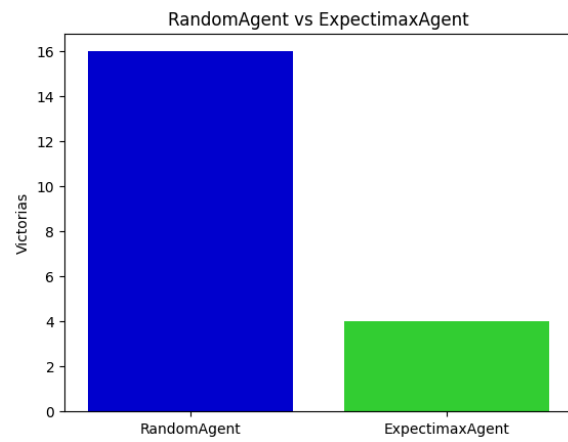
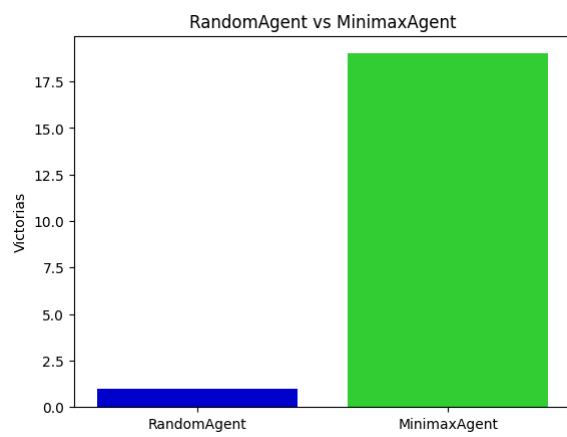
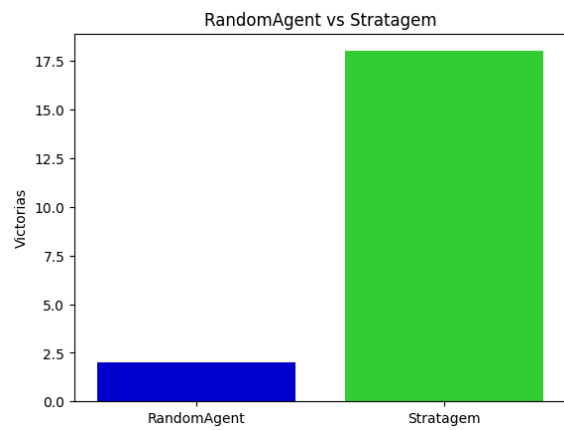
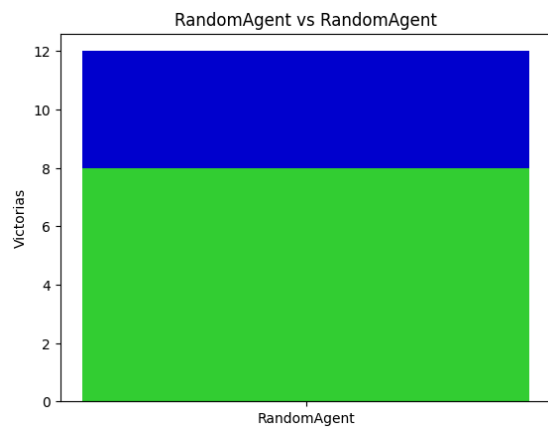
run_id      : ql_bins_30x20
Parámetros  : {'x_bins': 30, 'vel_bins': 20, 'action_bins': 20,
'alpha': 0.1}
avg_reward   : 94.71 ± 0.26
success_rate : 100%
avg_steps    : 152
Convergencia : episodio 1644
Estabilidad  : std=0.829 (últimos 200 ep)

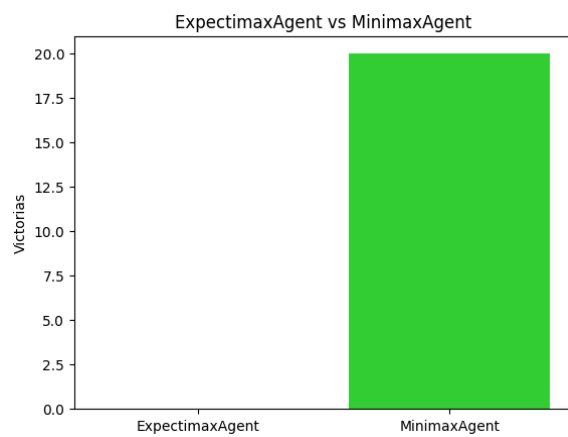
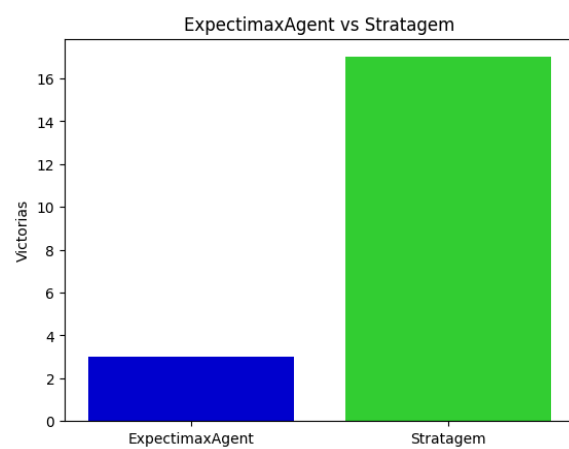
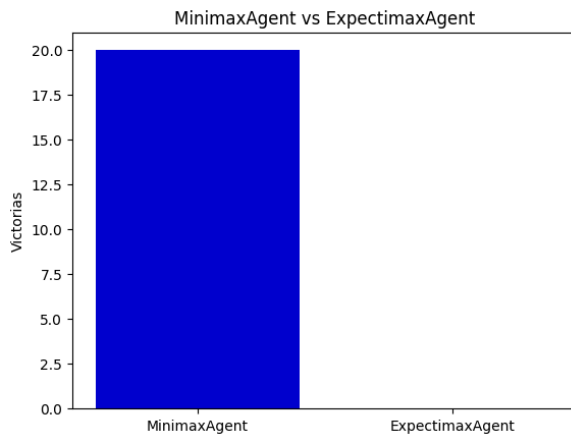
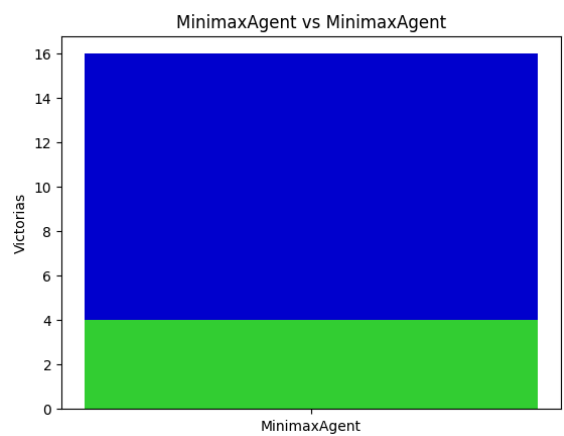
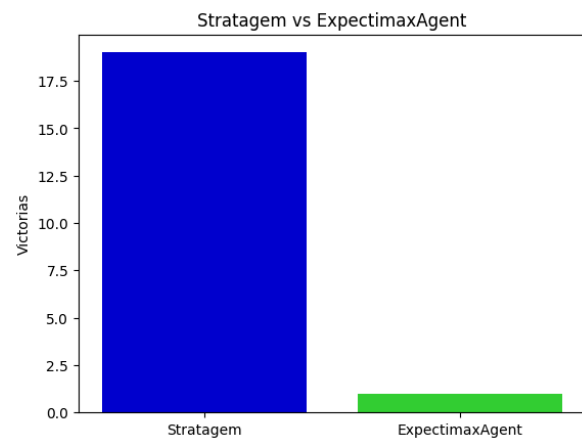
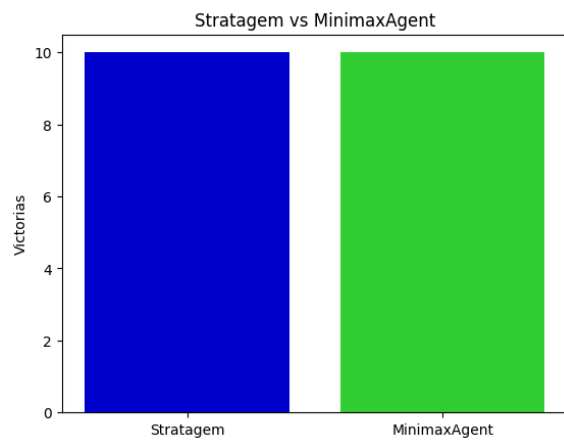
```

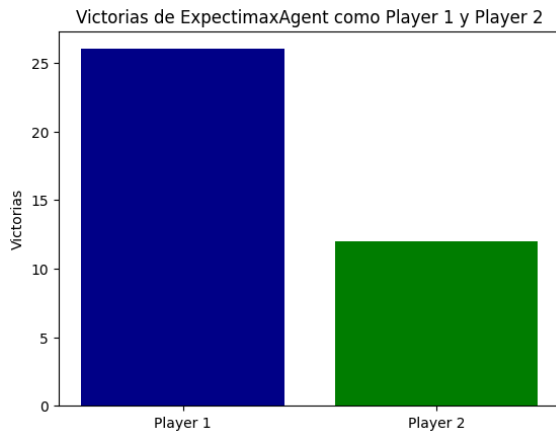
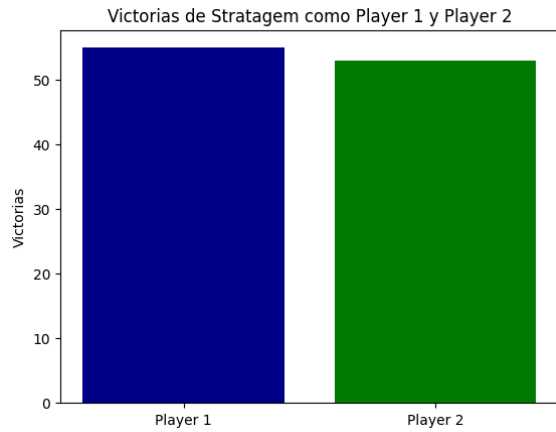
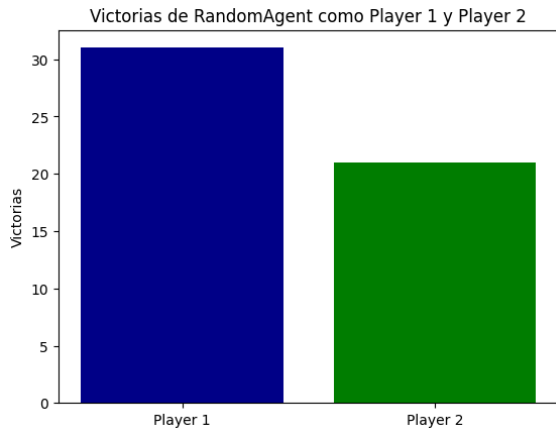
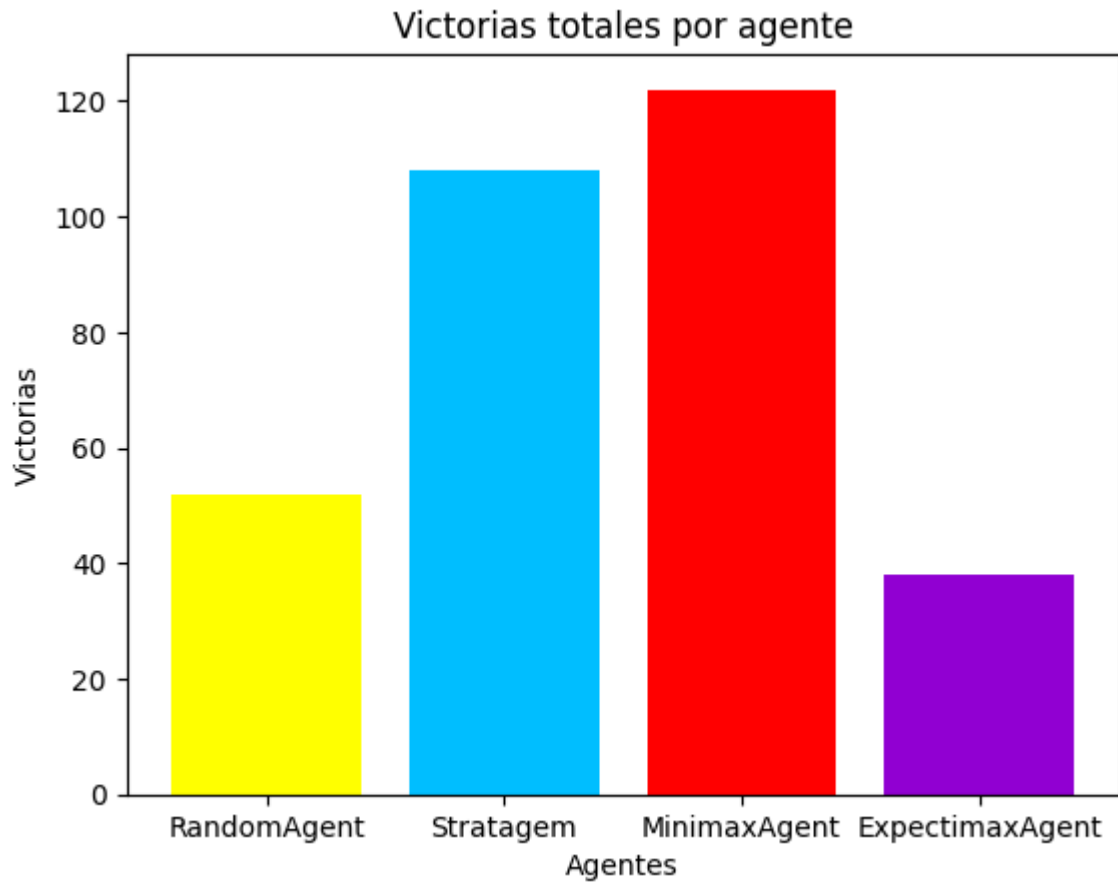
=====

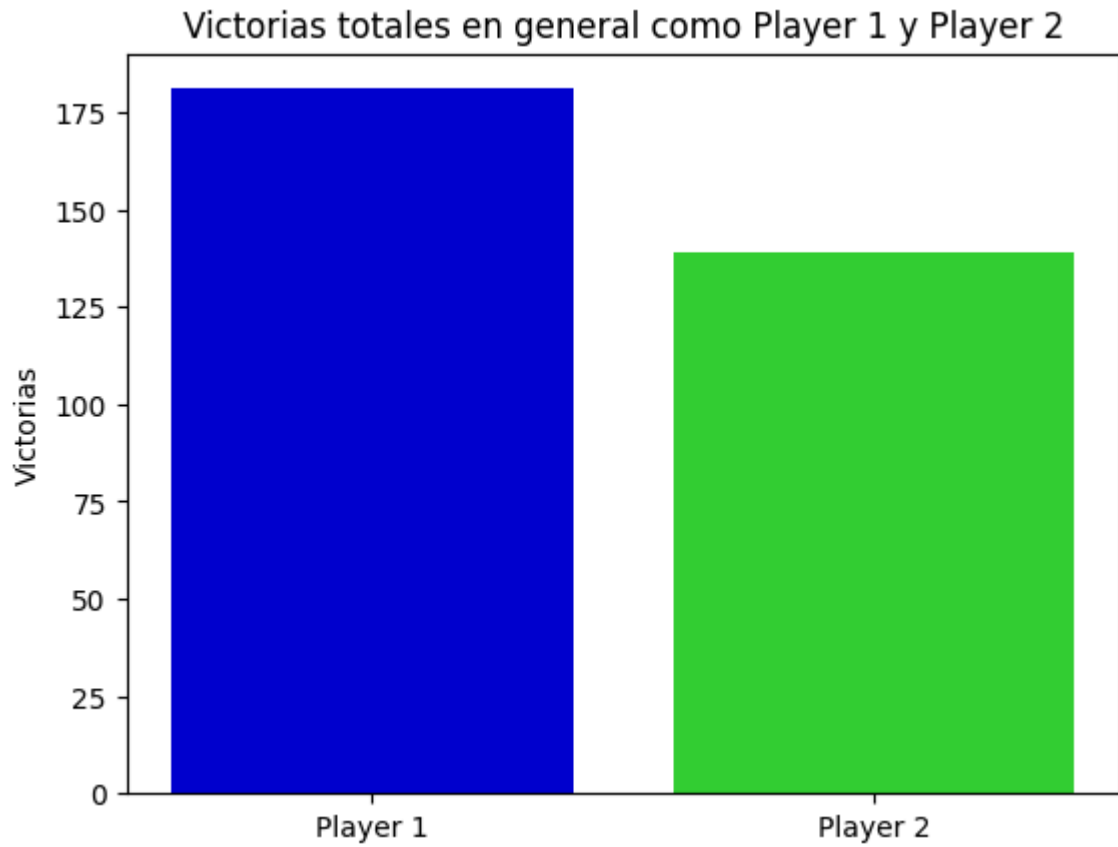


Anexo 2.1

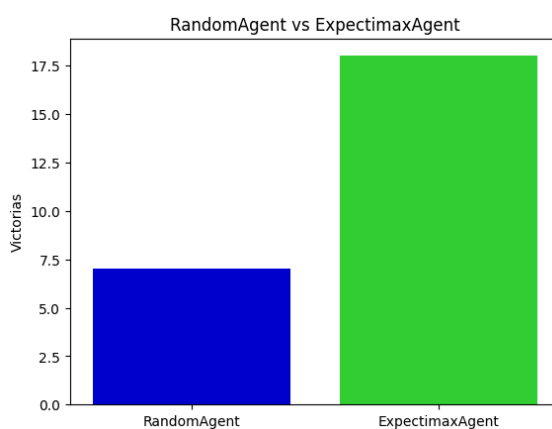
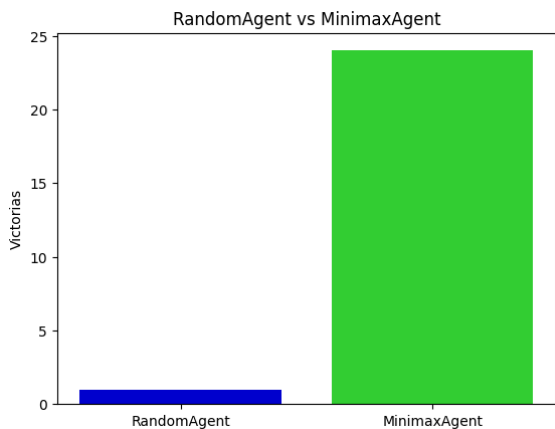
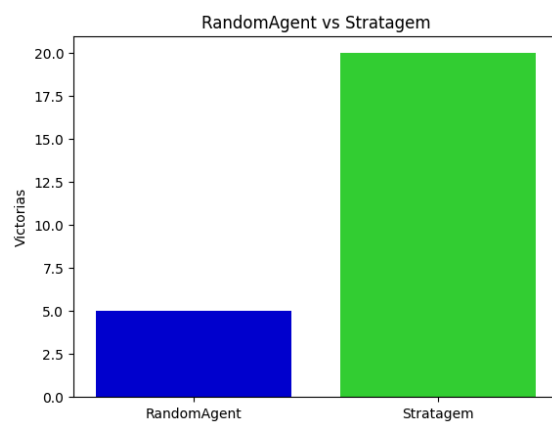
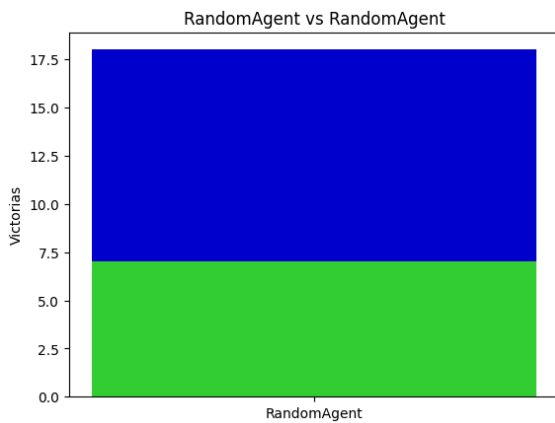


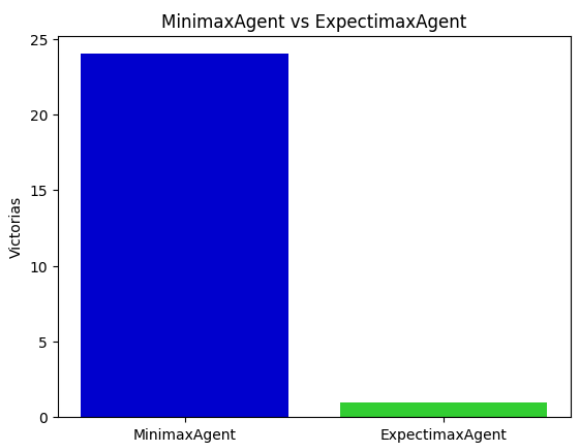
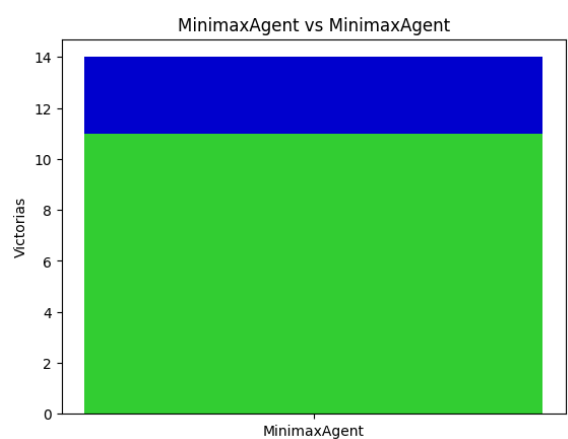
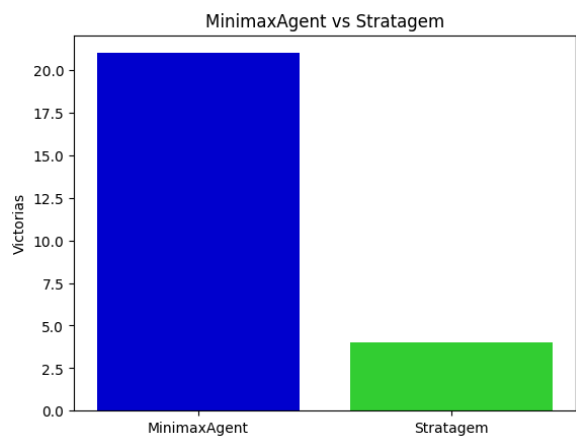
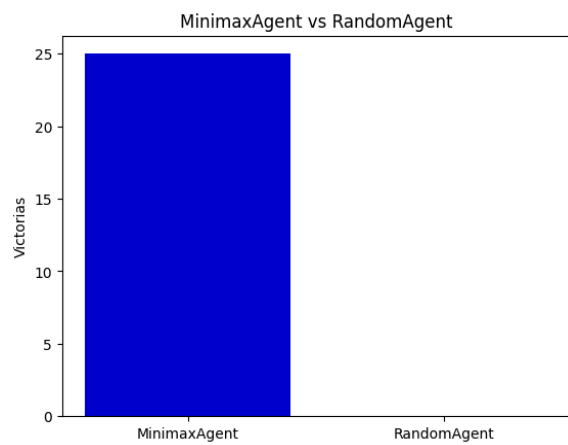
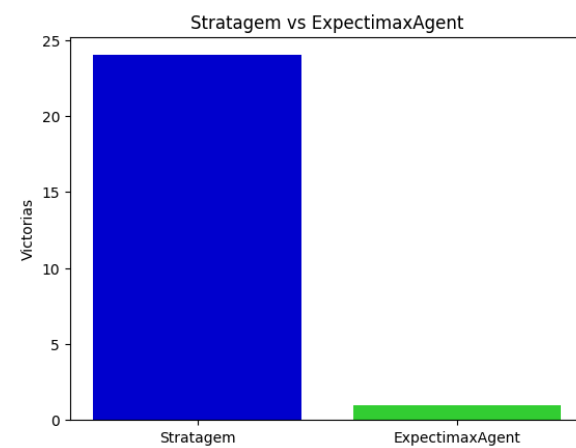
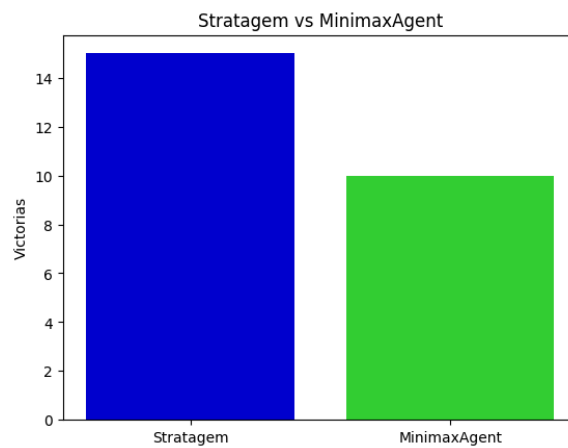
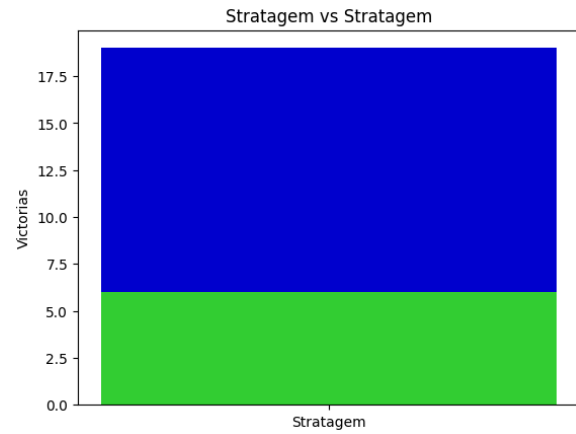
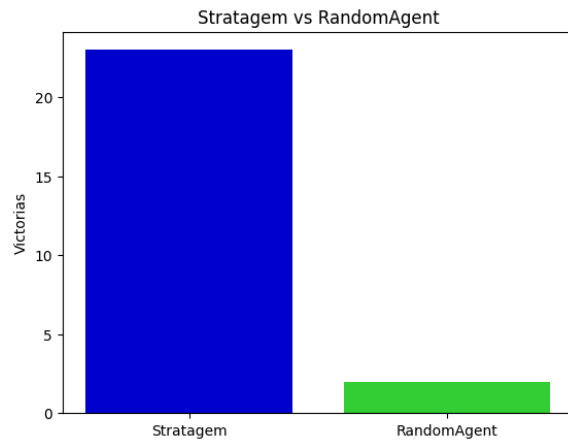


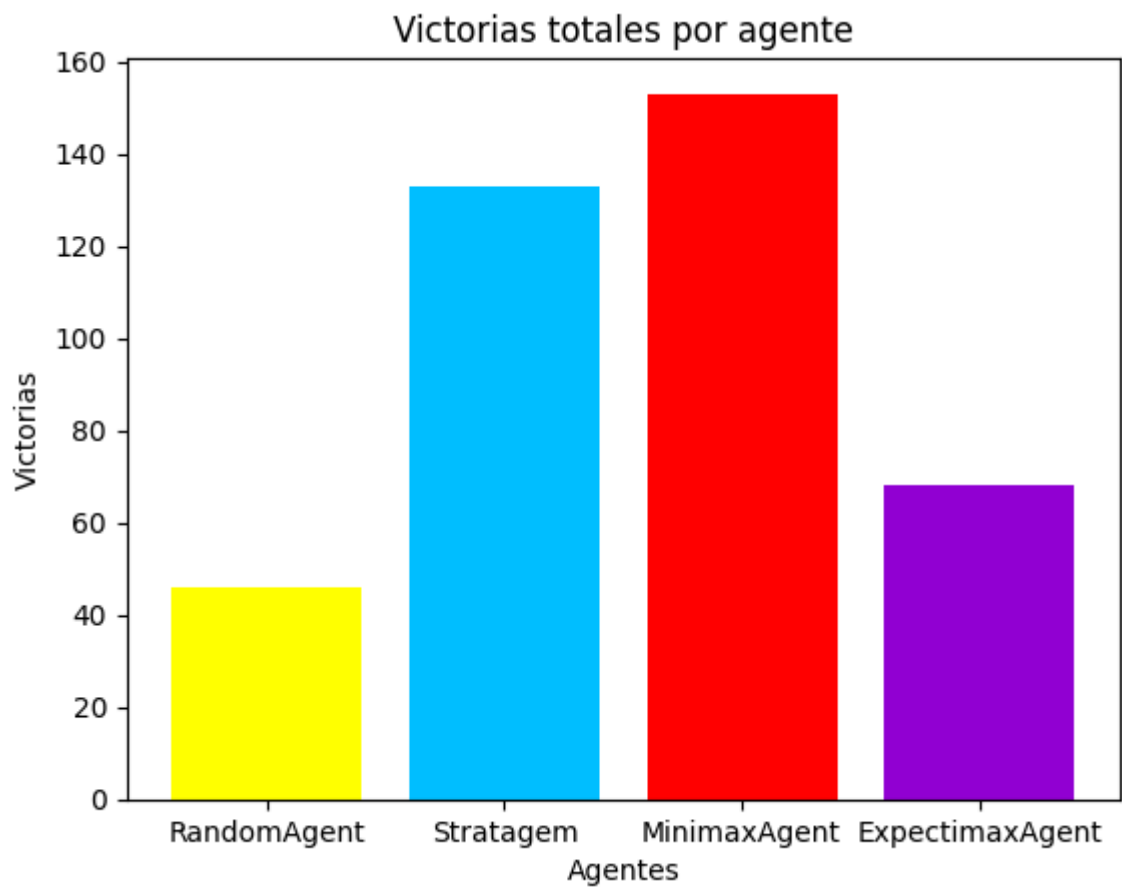
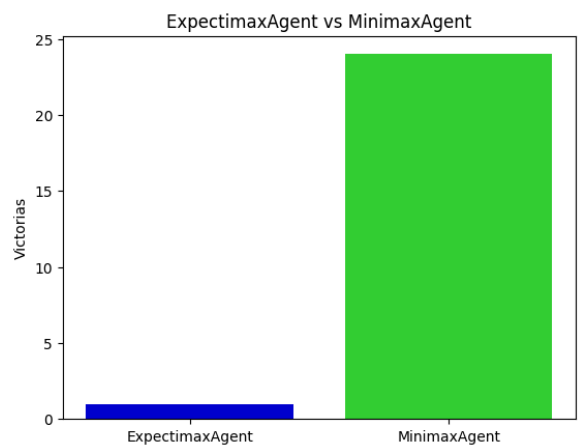
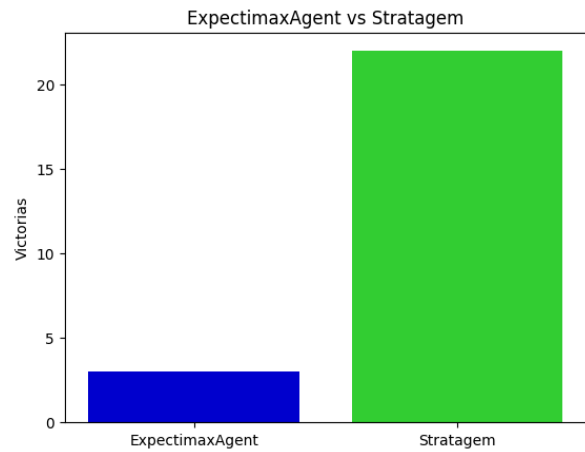
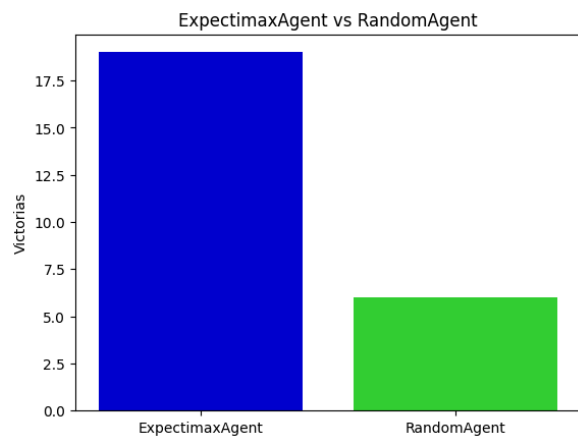


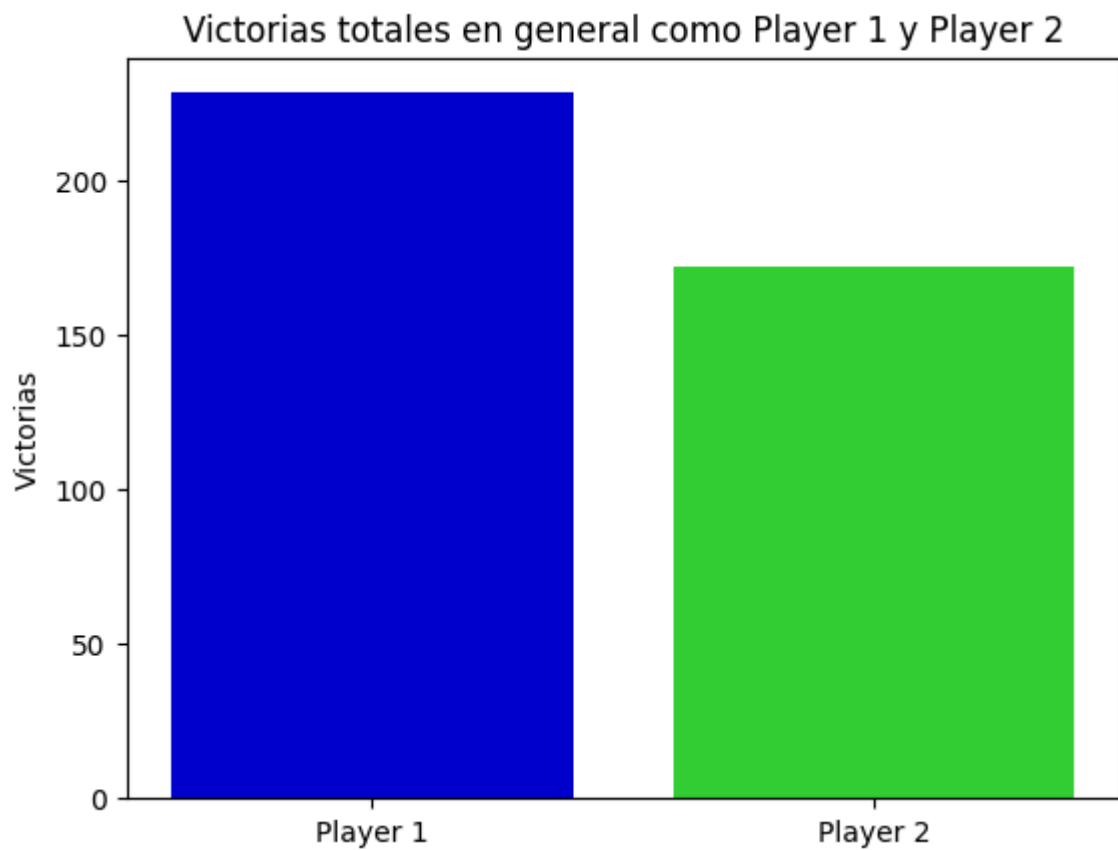
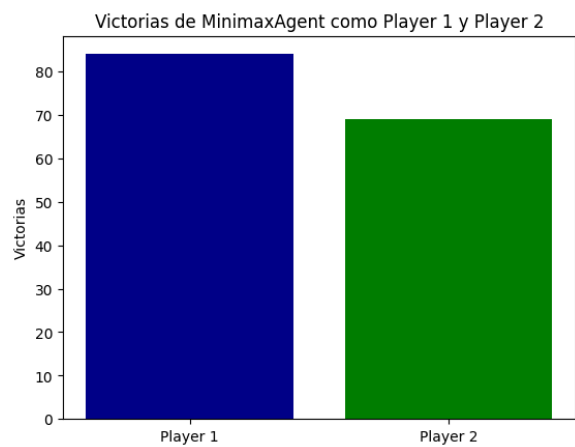
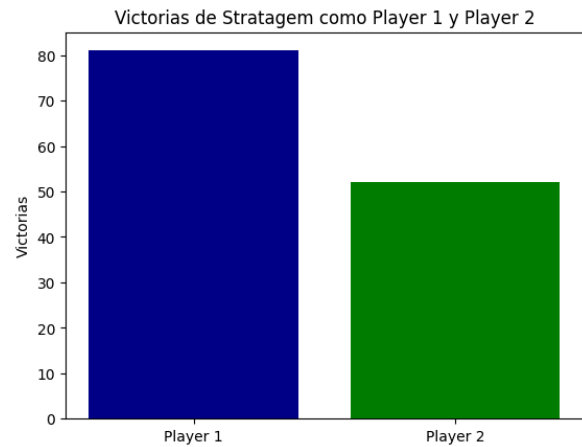
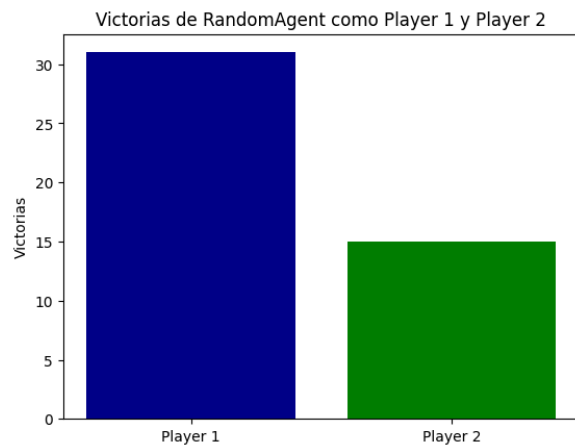


Anexo 2.2



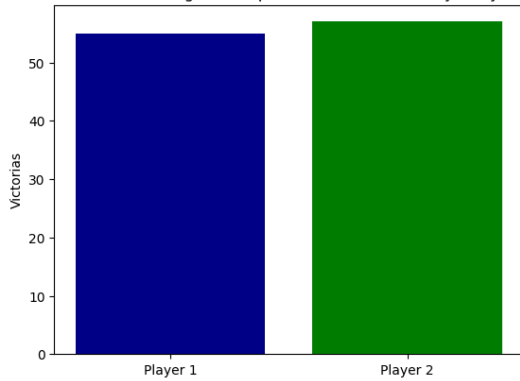




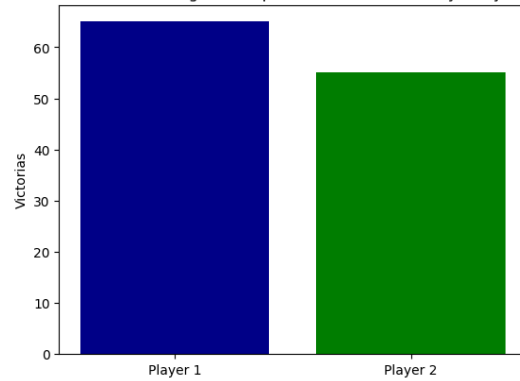


Anexo 2.3

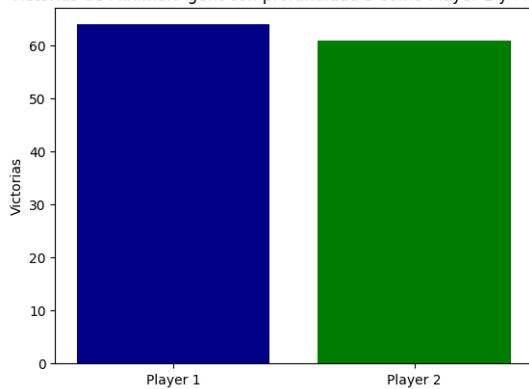
Victorias de MinimaxAgent con profundidad 1 como Player 1 y Player 2



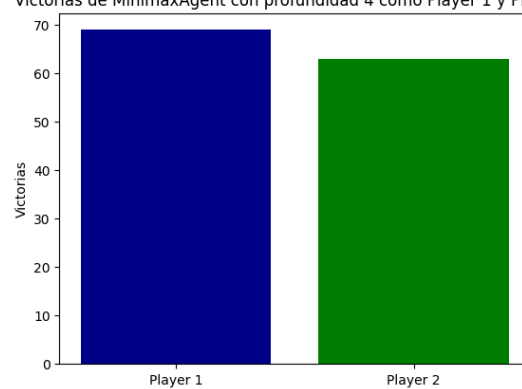
Victorias de MinimaxAgent con profundidad 2 como Player 1 y Player 2



Victorias de MinimaxAgent con profundidad 3 como Player 1 y Player 2



Victorias de MinimaxAgent con profundidad 4 como Player 1 y Player 2



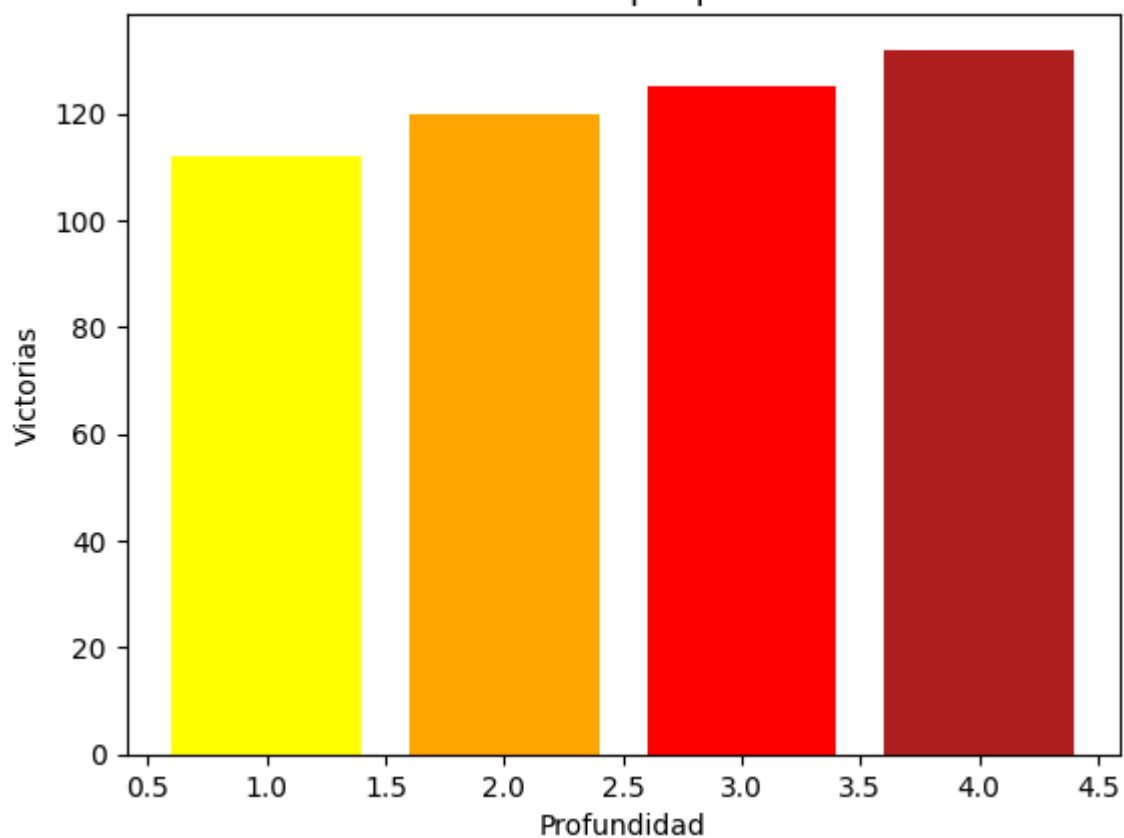
Victorias totales con profundidad 1 : 112

Victorias totales con profundidad 2 : 120

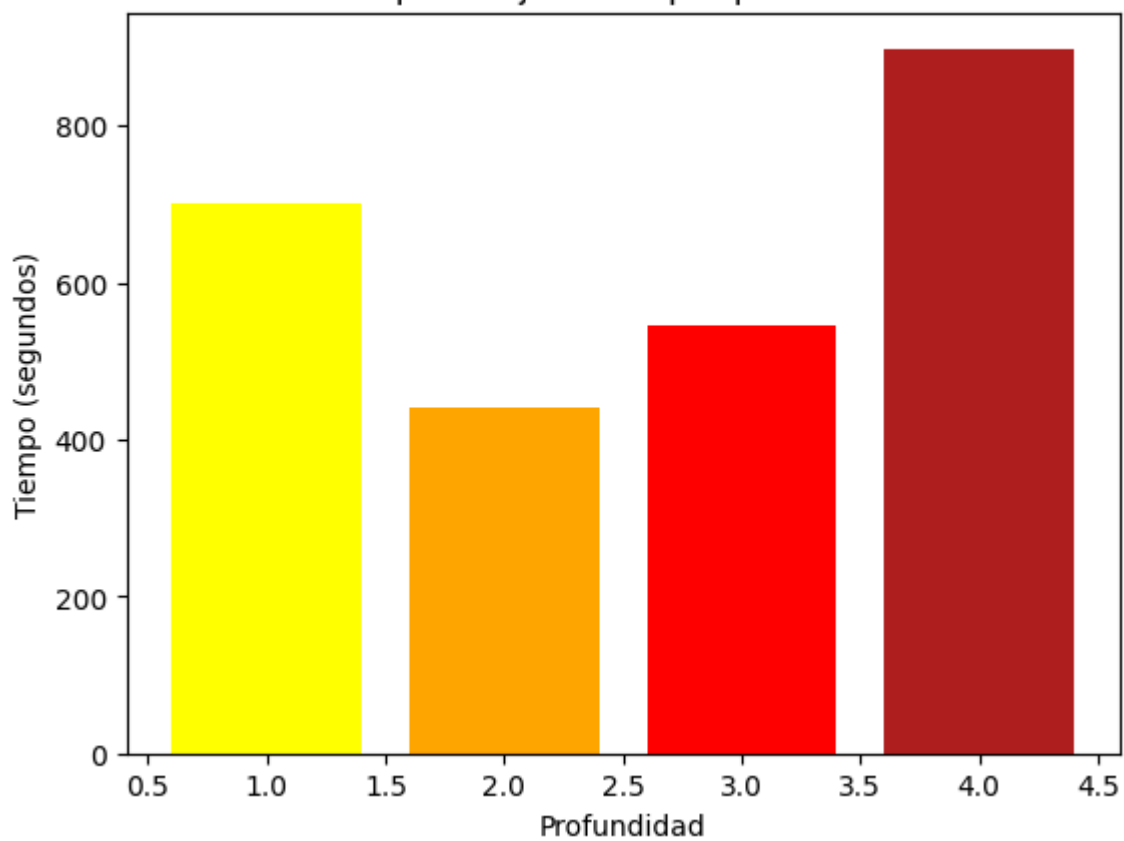
Victorias totales con profundidad 3 : 125

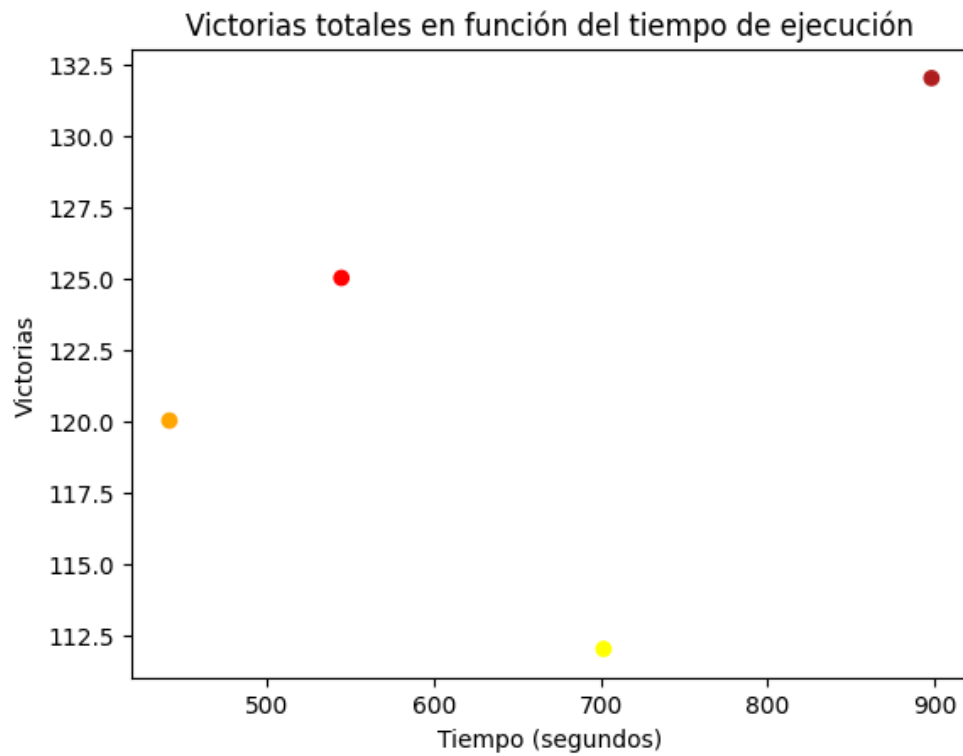
Victorias totales con profundidad 4 : 132

Victorias totales por profundidad



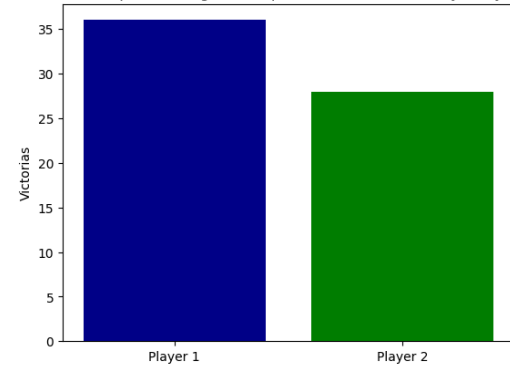
Tiempo de ejecución por profundidad



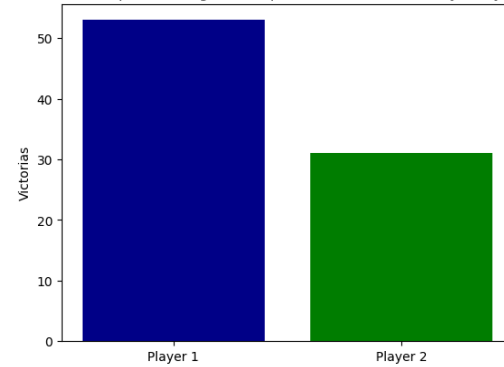


Anexo 2.4

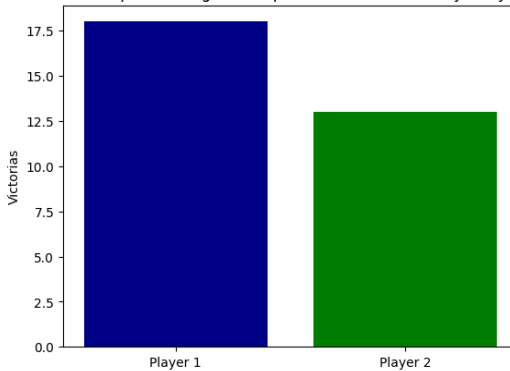
Victorias de ExpectimaxAgent con profundidad 1 como Player 1 y Player 2



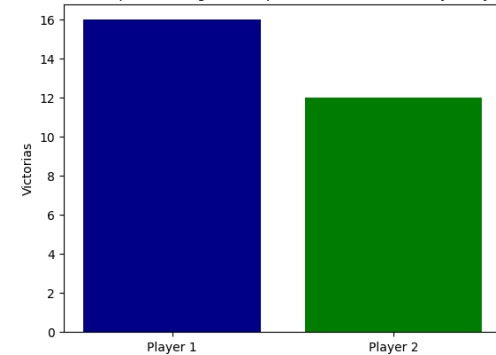
Victorias de ExpectimaxAgent con profundidad 2 como Player 1 y Player 2

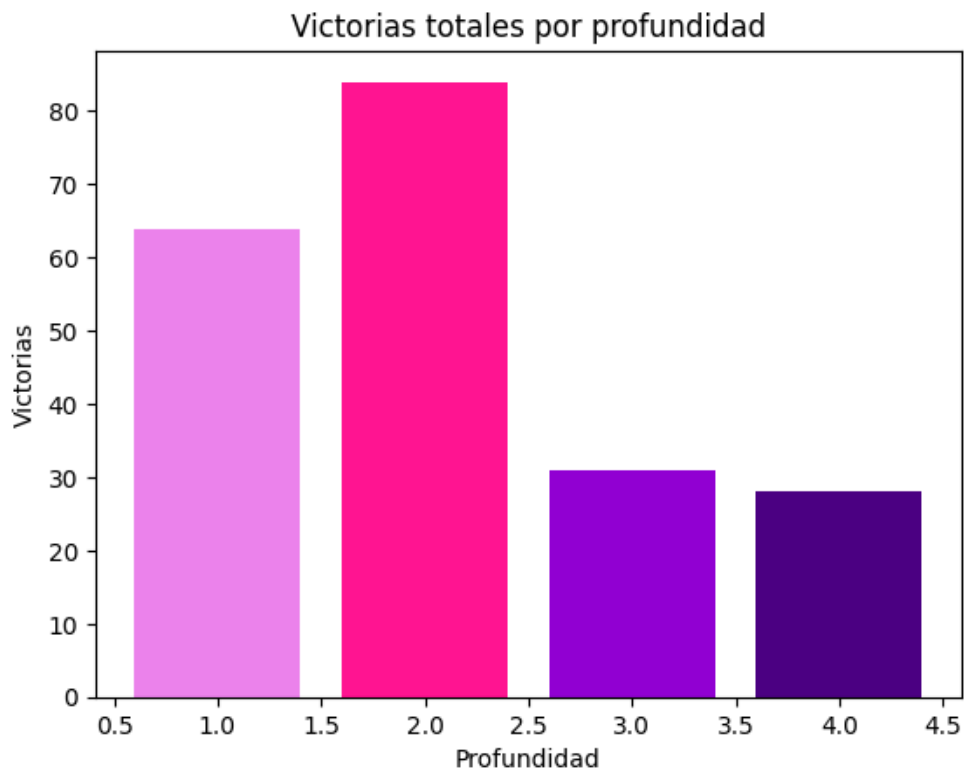


Victorias de ExpectimaxAgent con profundidad 3 como Player 1 y Player 2



Victorias de ExpectimaxAgent con profundidad 4 como Player 1 y Player 2



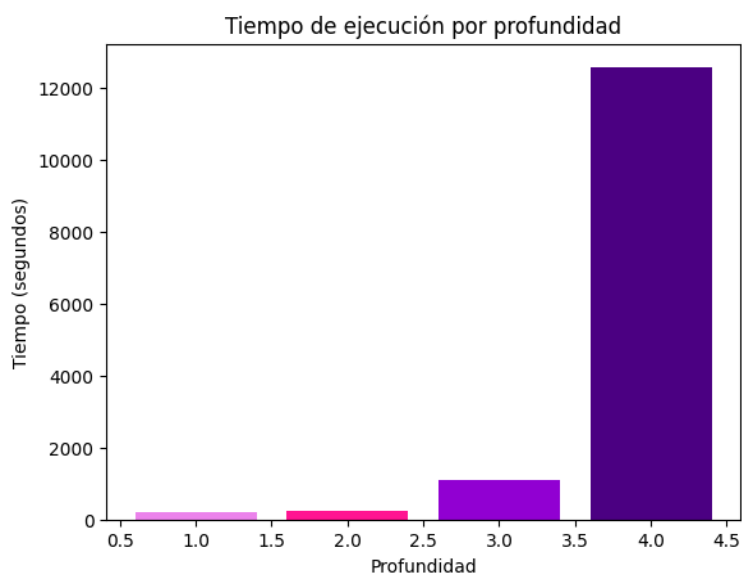


Victorias totales con profundidad 1 : 64

Victorias totales con profundidad 2 : 84

Victorias totales con profundidad 3 : 31

Victorias totales con profundidad 4 : 28

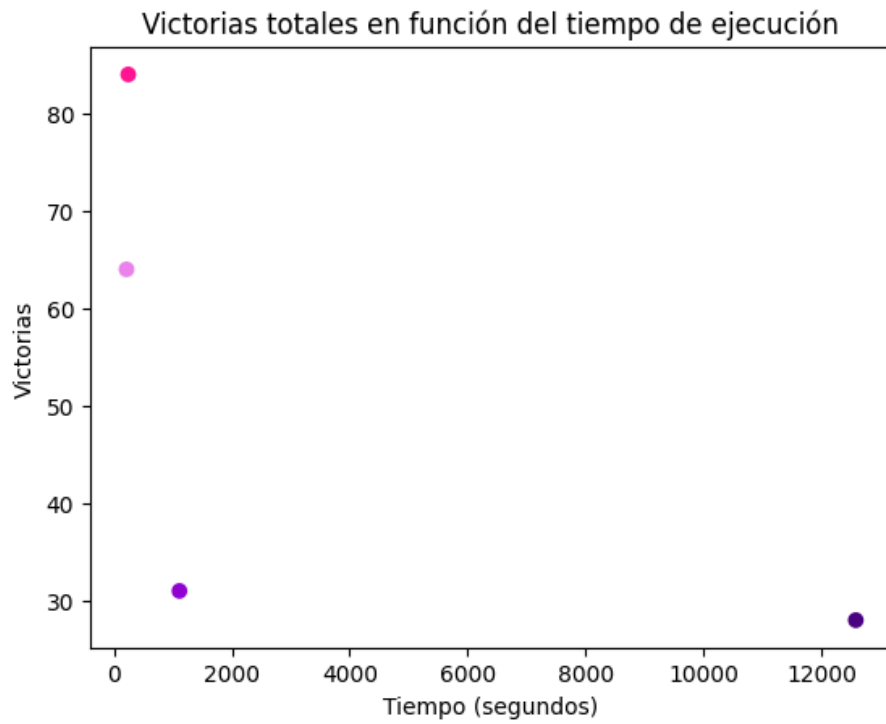


Tiempo de ejecución con profundidad 1 : 215.9470591545105 segundos

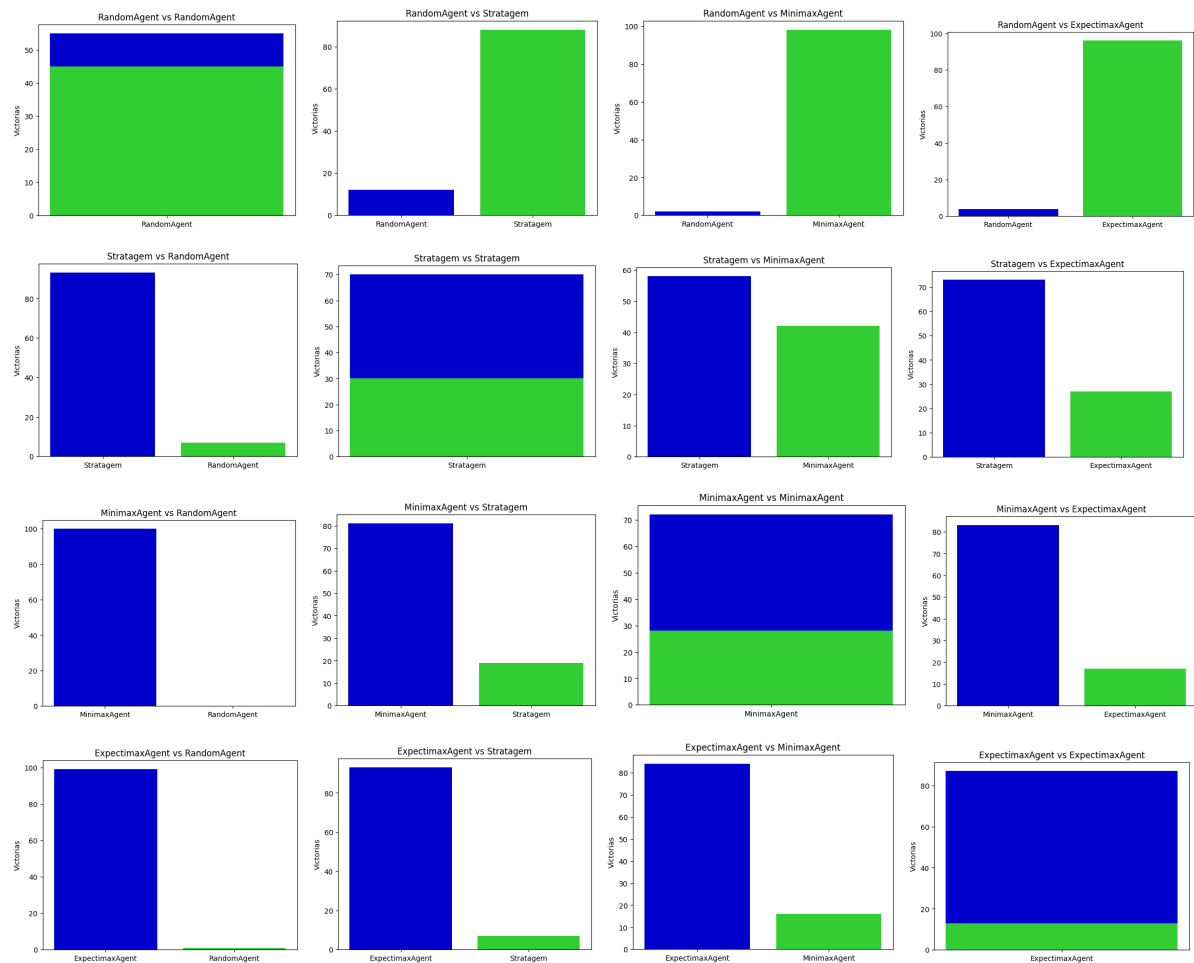
Tiempo de ejecución con profundidad 2 : 245.6399405002594 segundos

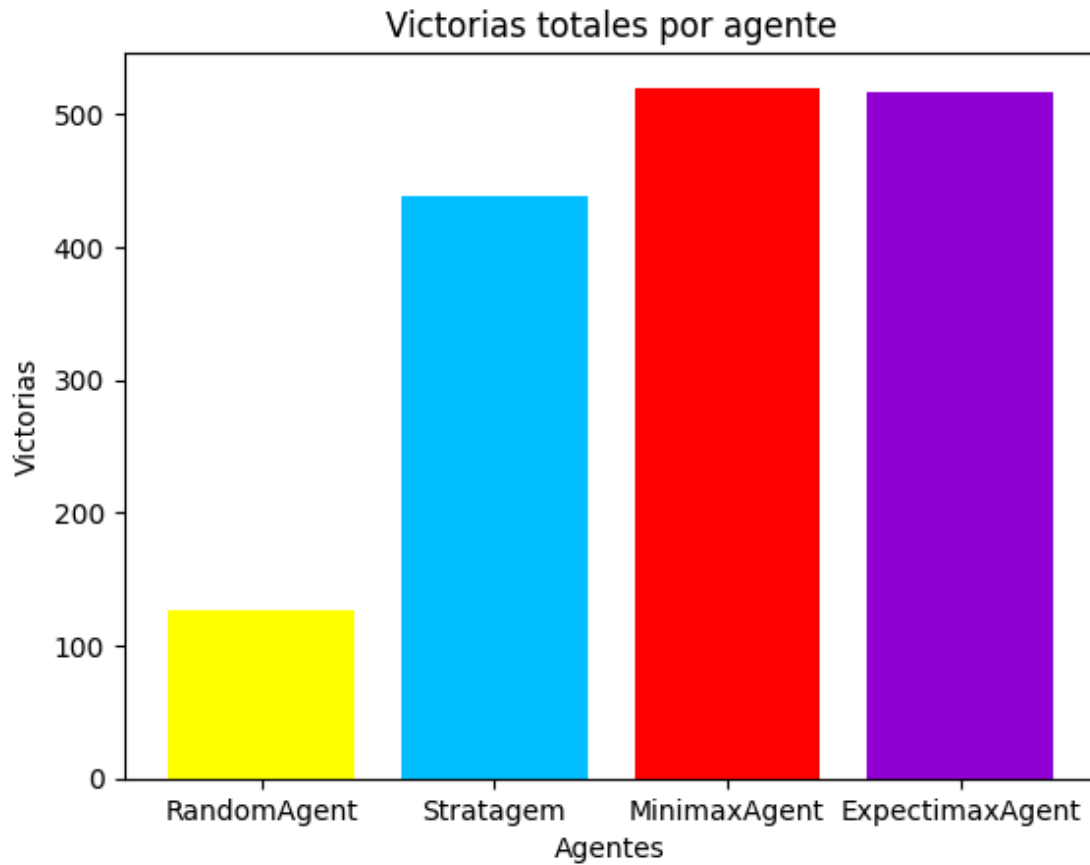
Tiempo de ejecución con profundidad 3 : 1113.8971092700958 segundos

Tiempo de ejecución con profundidad 4 : 12578.87403345108 segundos



Anexo 2.5



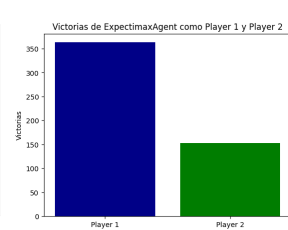
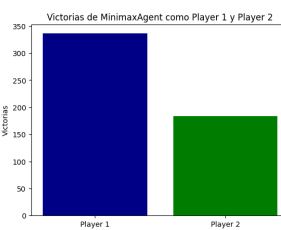
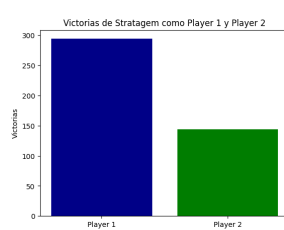
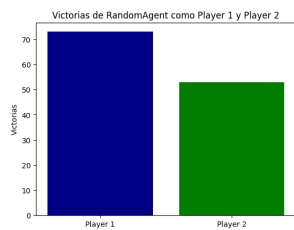


RandomAgent victorias totales: 126 victorias

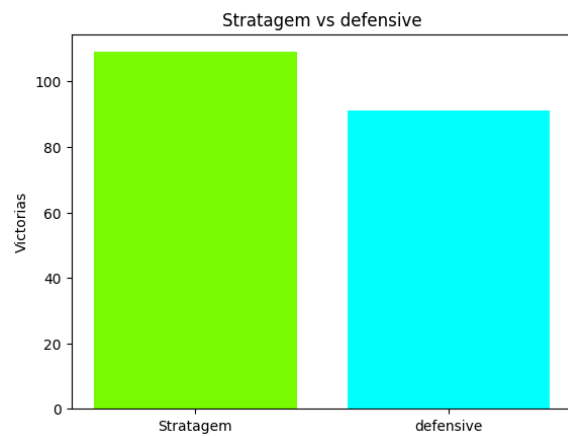
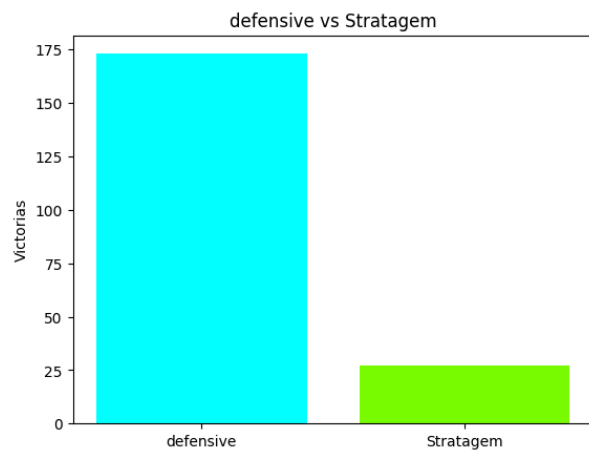
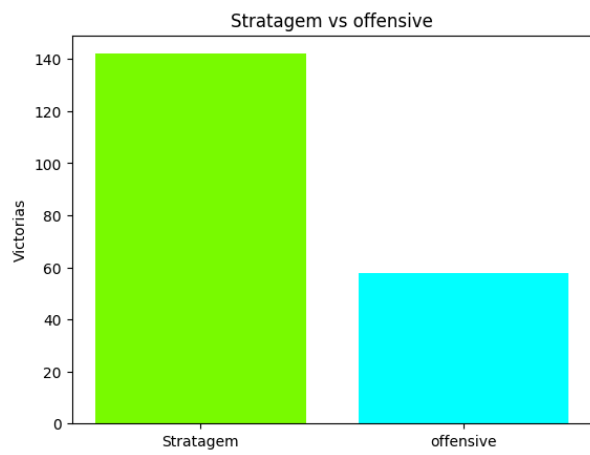
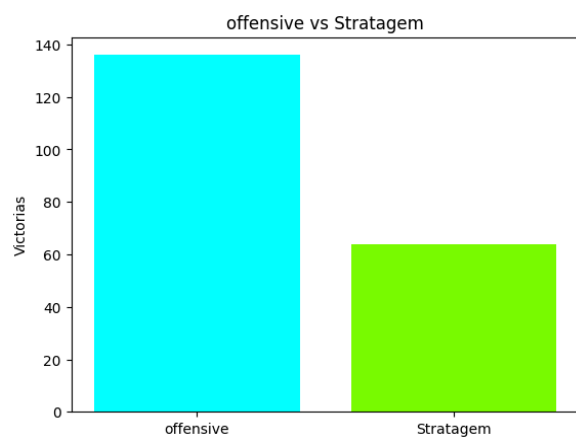
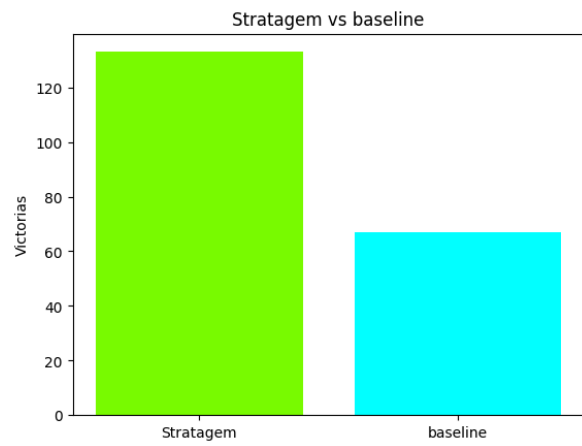
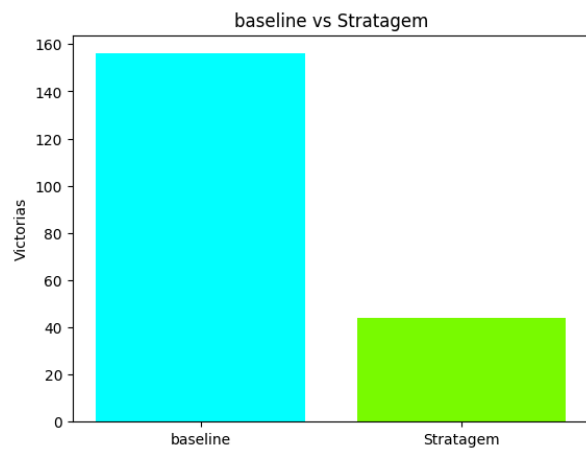
Stratagem victorias totales: 438 victorias

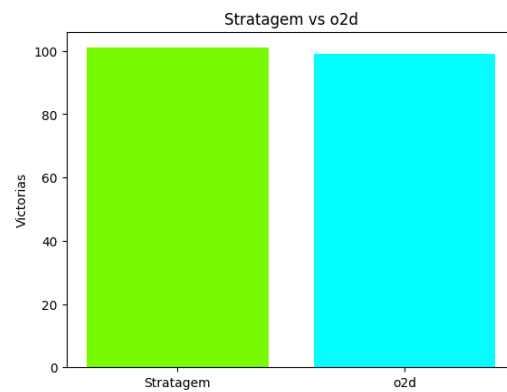
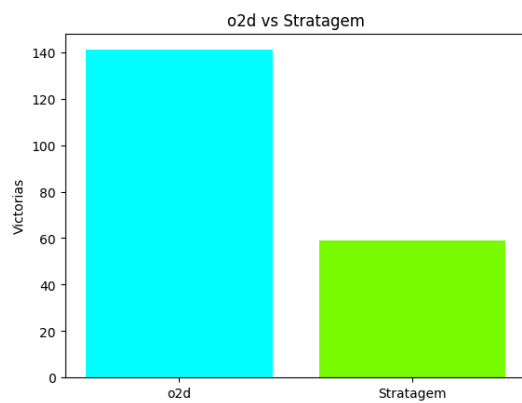
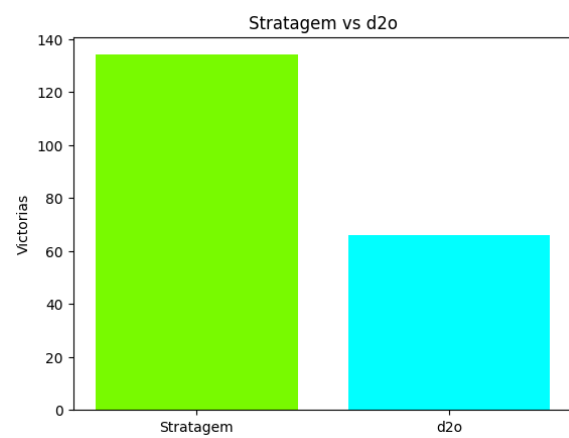
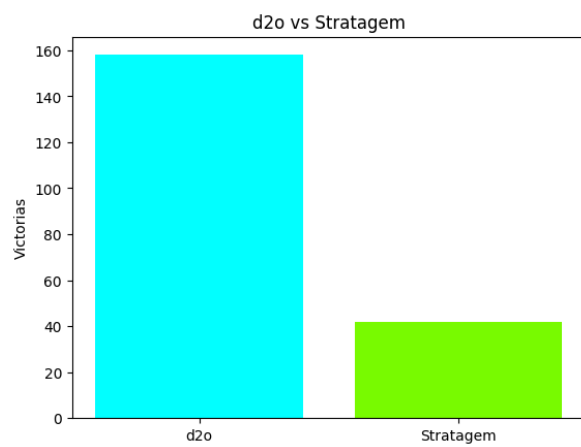
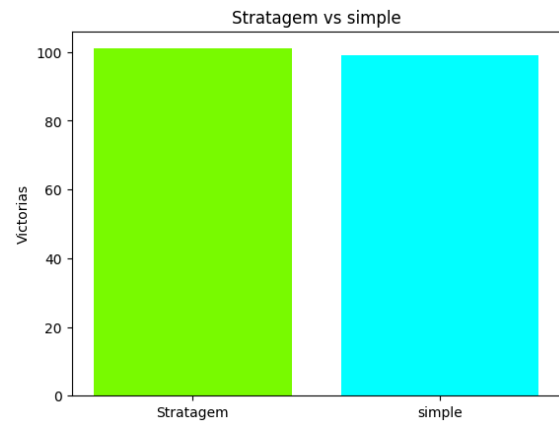
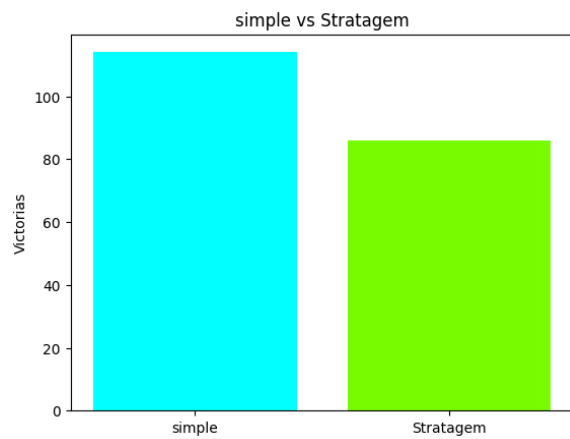
MinimaxAgent victorias totales: 520 victorias

ExpectimaxAgent victorias totales: 516 victorias

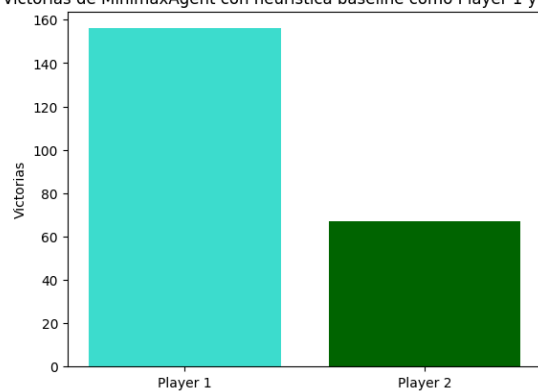


Anexo 2.6



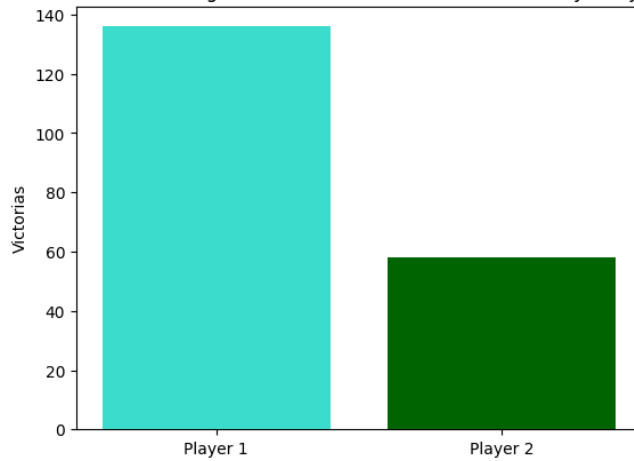


Victorias de MinimaxAgent con heuristica baseline como Player 1 y Player 2



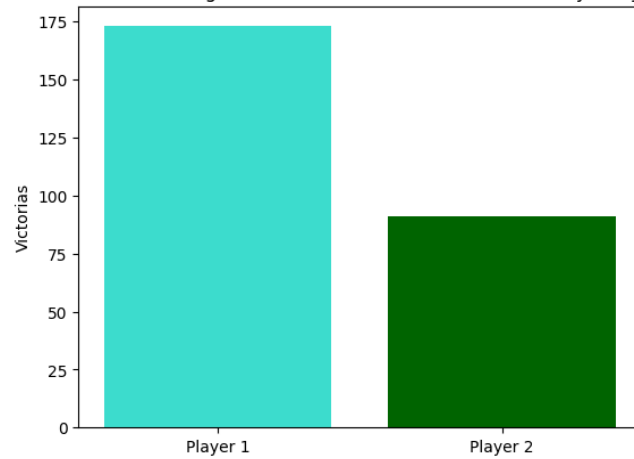
Victorias totales con heuristica baseline: 223

Victorias de MinimaxAgent con heuristica offensive como Player 1 y Player 2



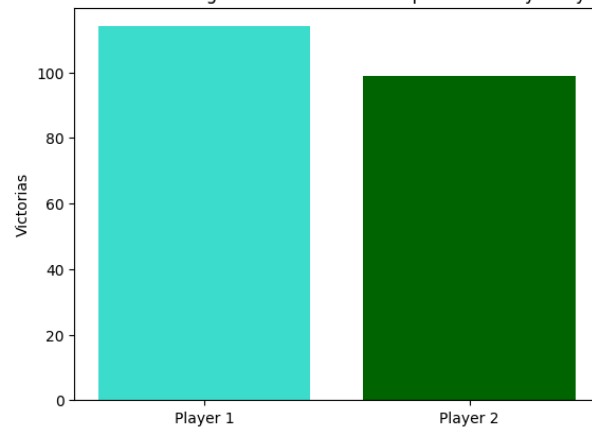
Victorias totales con heuristica offensive: 194

Victorias de MinimaxAgent con heuristica defensive como Player 1 y Player 2



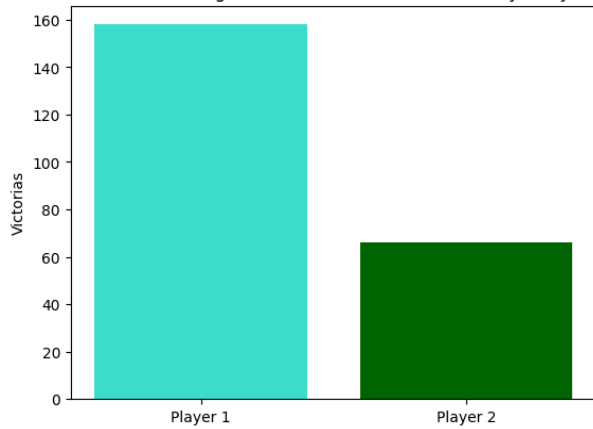
Victorias totales con heuristica defensive: 264

Victorias de MinimaxAgent con heuristica simple como Player 1 y Player 2



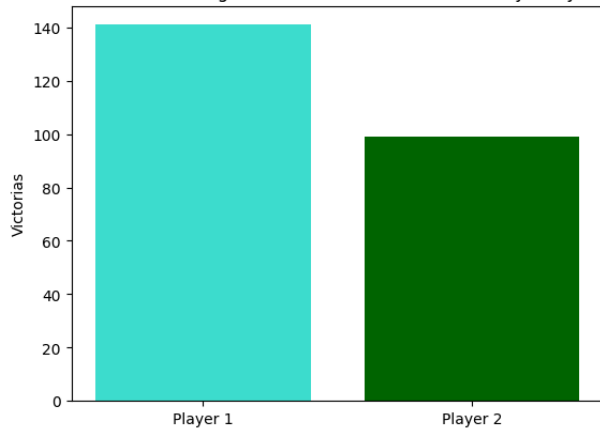
Victorias totales con heuristica simple: 213

Victorias de MinimaxAgent con heurística d2o como Player 1 y Player 2



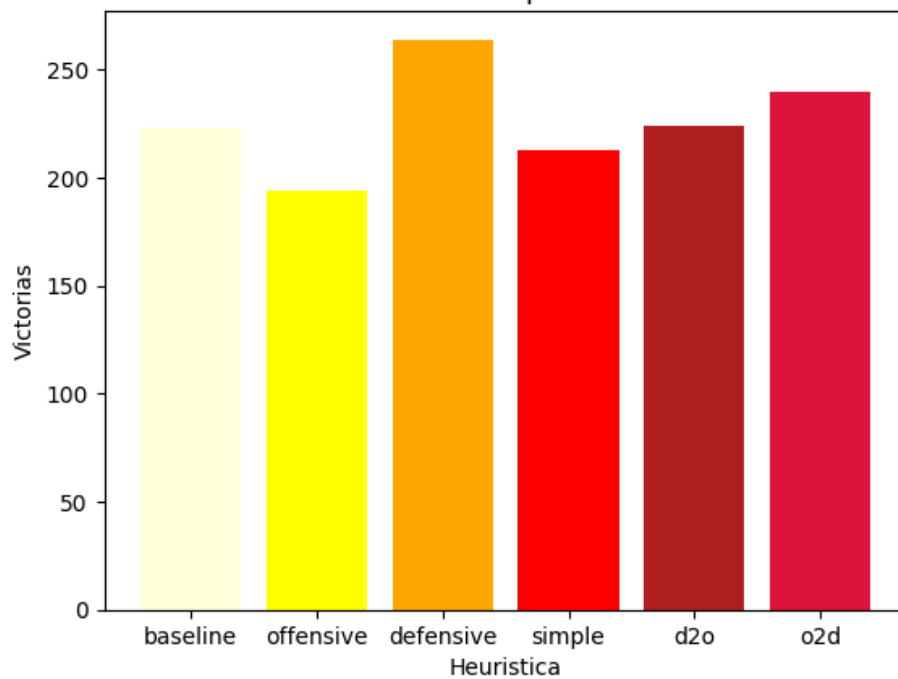
Victorias totales con heurística d2o: 224

Victorias de MinimaxAgent con heurística o2d como Player 1 y Player 2

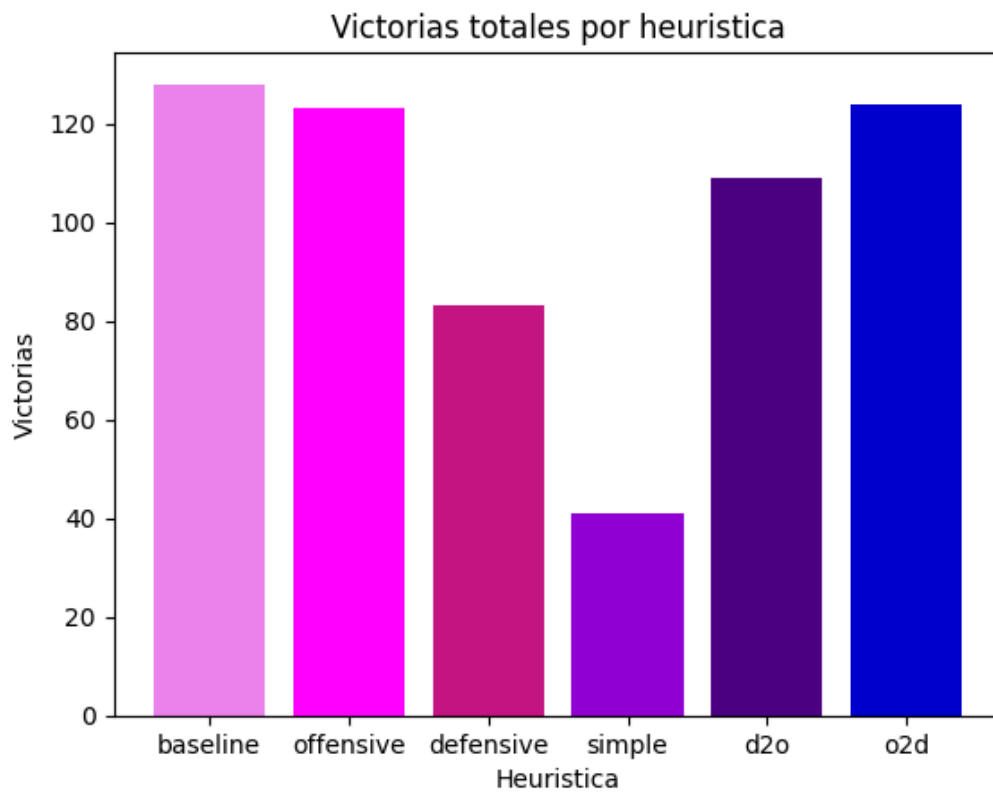


Victorias totales con heurística o2d: 240

Victorias totales por heurística

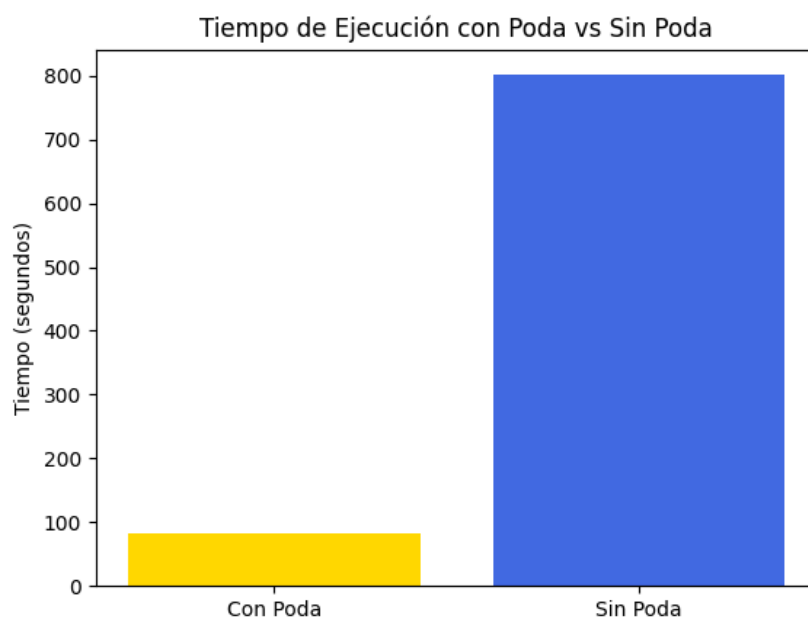


Anexo 2.7



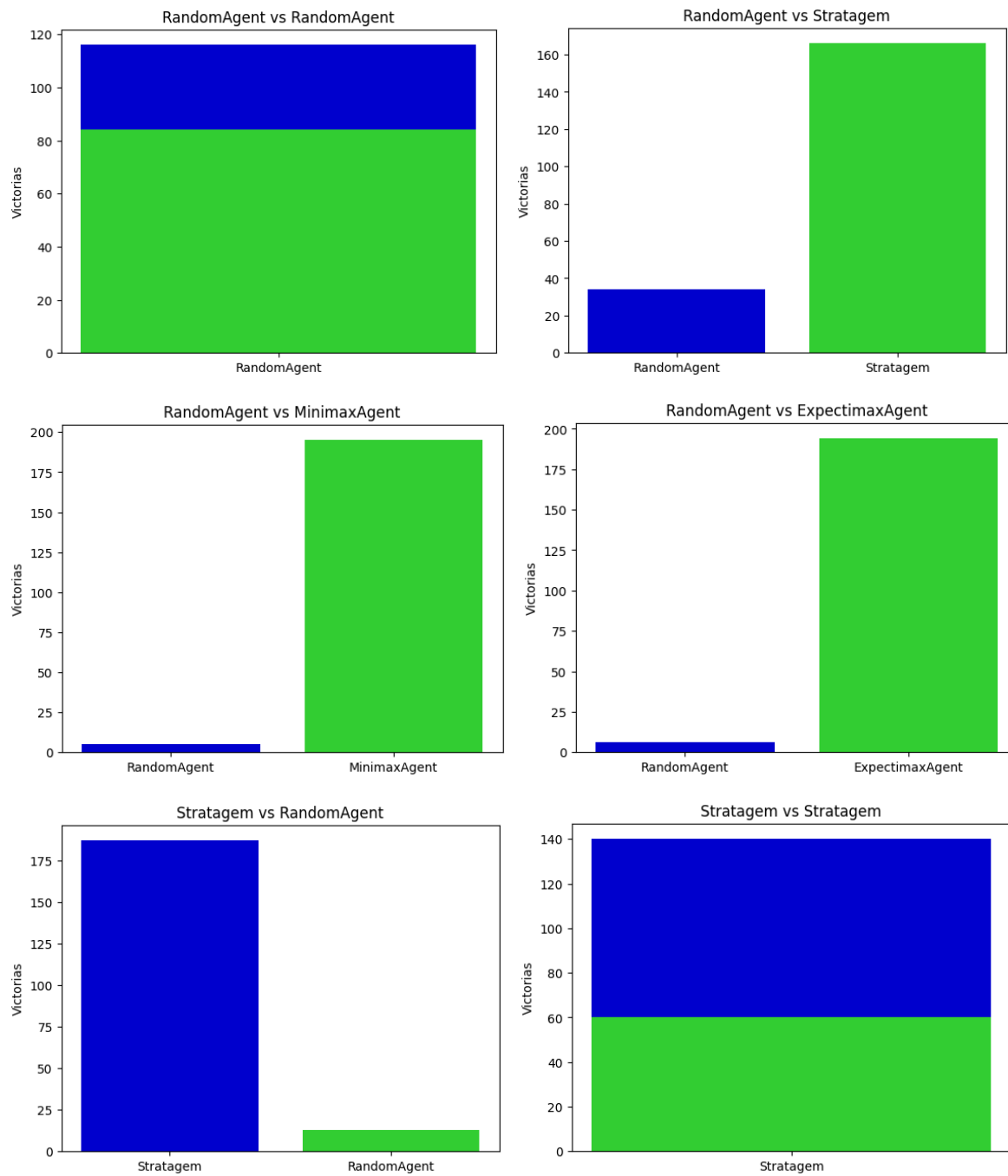
Victorias totales con heurística baseline: 128
Victorias totales con heurística offensive: 123
Victorias totales con heurística defensive: 83
Victorias totales con heurística simple: 41
Victorias totales con heurística d2o: 109
Victorias totales con heurística o2d: 124

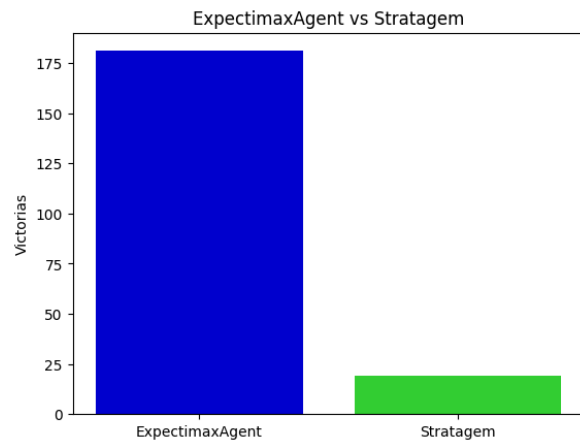
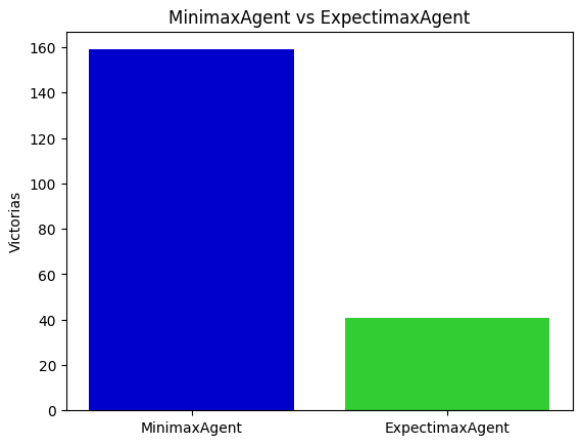
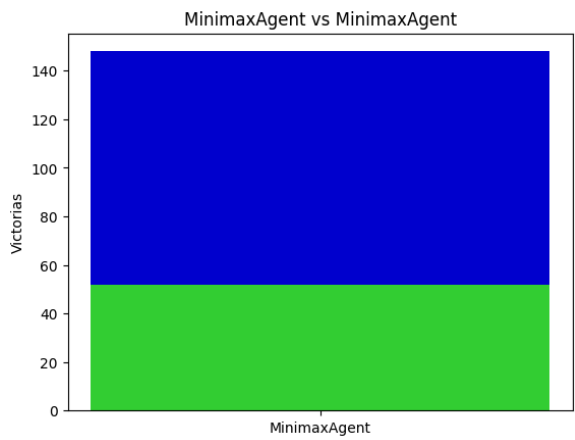
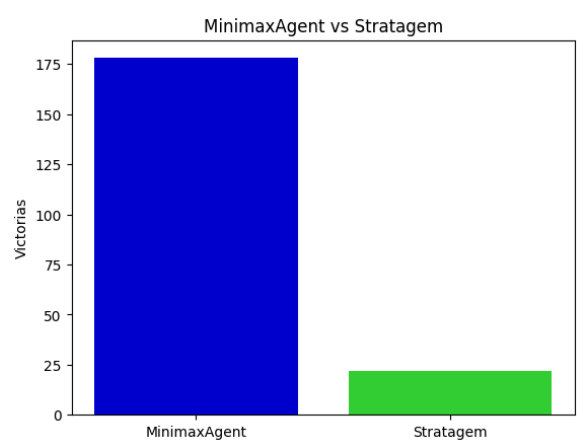
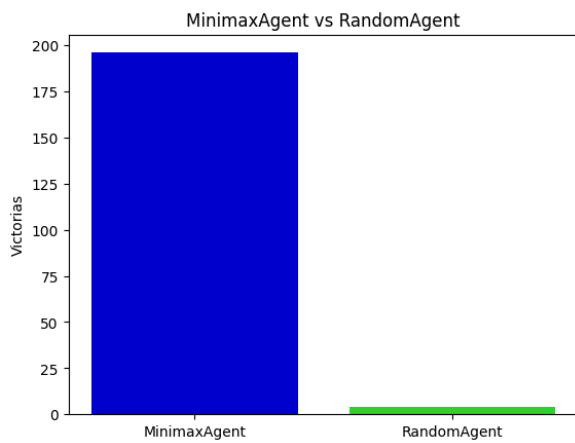
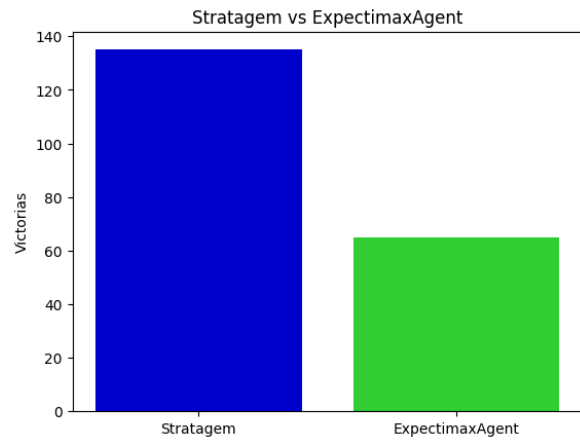
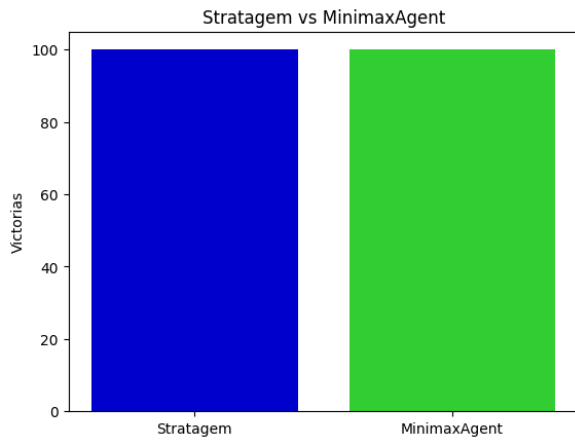
Anexo 2.8

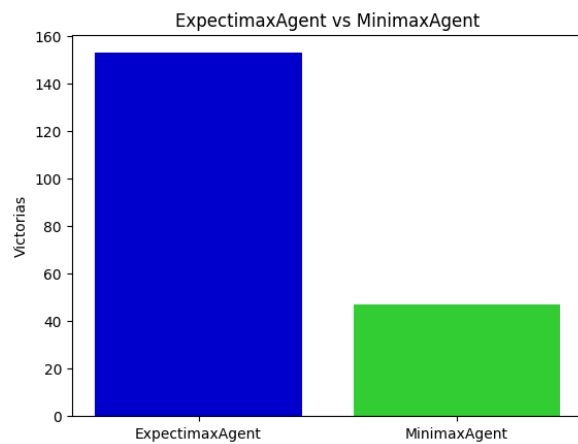


Victorias sin poda: 98, Tiempo sin poda: 800.8594064712524 segundos
Victorias con poda: 99, Tiempo con poda: 83.46517372131348 segundos

Anexo 2.9





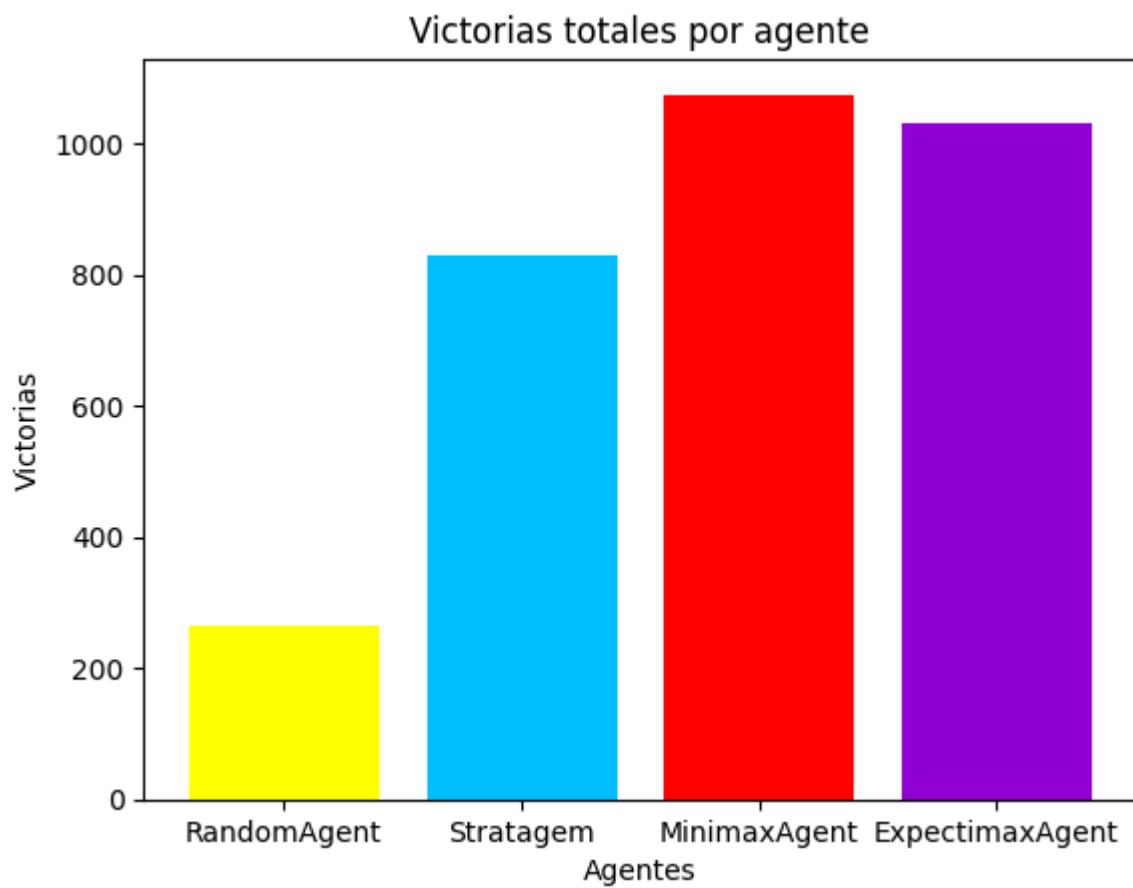


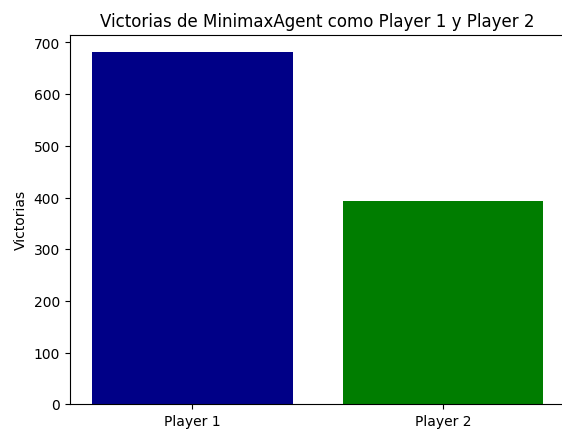
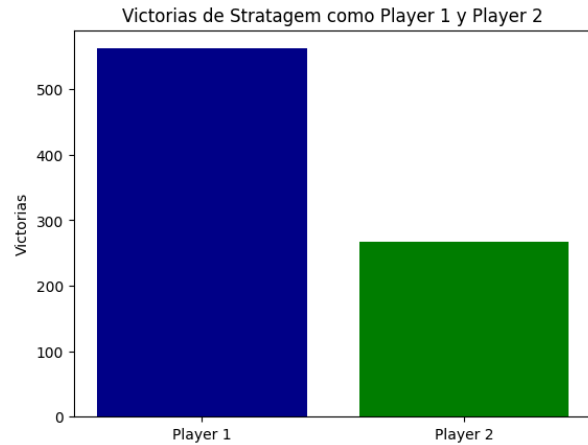
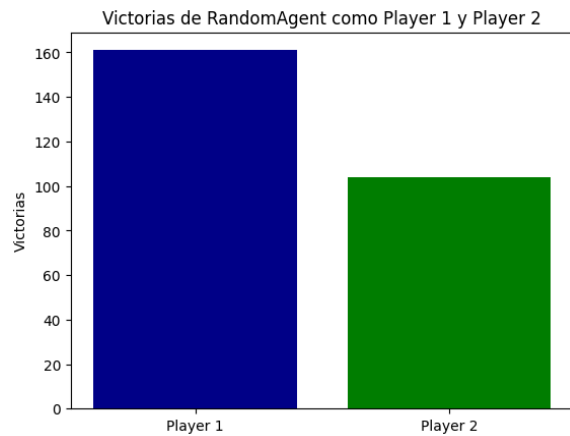
RandomAgent victorias totales: 265 victorias

Stratagem victorias totales: 829 victorias

MinimaxAgent victorias totales: 1075 victorias

ExpectimaxAgent victorias totales: 1031 victorias





RandomAgent como Player 1: 161 victorias
 RandomAgent como Player 2: 104 victorias
 Stratagem como Player 1: 562 victorias
 Stratagem como Player 2: 267 victorias
 MinimaxAgent como Player 1: 681 victorias
 MinimaxAgent como Player 2: 394 victorias
 ExpectimaxAgent como Player 1: 694 victorias
 ExpectimaxAgent como Player 2: 337 victorias